ҚАЗАҚСТАН РЕСПУБЛИКАСЫ АУЫЛ ШАРУАШЫЛЫҒЫ МИНИСТРЛІГІ
«С.СЕЙФУЛЛИН атындағы ҚАЗАҚ АГРОТЕХНИКАЛЫҚ ЗЕРТТЕУ УНИВЕРСИТЕТІ» КеАҚ

БЕКІТЕМІН

Бизнес және цифрлық
технологиялар институтының
директоры _____
М. Е.Сабырова
« 26 » 06 2026 ж.

НАЙЗАҒАРАЕВА АҚГҮЛ АМАНЖОЛҚЫЗЫ

«WEB БАҒДАРЛАМАЛАУ»

ЭЛЕКТРОНДЫҚ ОҚУ - ӘДІСТЕМЕЛІК КЕШЕН

Білім беру бағдарламасы: **6В06101-«Бағдарламалық инженерия»**

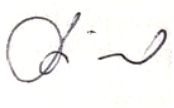
Астана, 2026ж

Электрондық оқу-әдістемелік кешенді құрастырған Найзағараева Ақгүл Аманжолқызы «Ақпараттық жүйелер» ББББТ-ның PhD, аға оқытушысы.

Электрондық оқу - әдістемелік кешен «Ақпараттық жүйелер» ББББТ отырысында қаралды және мақұлданды.
Хаттама № 10 «15» маусым 2026ж.

ББББТ жетекшісі  Шаушенова А.Г.

Бизнес және цифрлық технологиялар институтының Академиялық сапа жөніндегі кеңесінің отырысында қаралды және ұсынылды.
Хаттама № 11 « 24 » маусым 2026ж.

Академиялық сапа жөніндегі Кеңес төрайымы  Касимбекова М.А

ҚАЗАҚСТАН РЕСПУБЛИКАСЫ АУЫЛ ШАРУАШЫЛЫҒЫ МИНИСТРЛІГІ
«С.СЕЙФУЛЛИН атындағы ҚАЗАҚ АГРОТЕХНИКАЛЫҚ ЗЕРТТЕУ УНИВЕРСИТЕТІ» КеАҚ

БЕКІТЕМІН

Бизнес және цифрлық технологиялар институтының директоры _____
М. Е.Сабырова
« 26 » 06 2026 ж.

ПӘННІҢ ЖҰМЫС БАҒДАРЛАМАСЫ
(силлабус)

Пәні: «WEB бағдарламалау»
Білім беру бағдарламасы: 6В06101-«Бағдарламалық инженерия»
Білім беру бағдарламасы тобы: В057- «Ақпараттық технологиялар»
Дайындық бағыты: 6В061-Ақпараттық-коммуникациялық технологиялар
Білім беру саласы: 6В06 Ақпараттық коммуникациялық технологиялар

Курс __3____ Семестр__1___ Кредит саны __5__

	Сағат саны
Дәрістер	15
Зертханалық сабақтар	30
Практикалық/семинар сабақтары	
Өзіндік жұмыс (БӨЖ)	85
Өзіндік жұмыс оқытушымен (БОӨЖ)	20
Барлығы	150
Пәндер циклі (ЖББП (ЖООК), БП, ТК)	БП
Бақылау формасы	емтихан

Астана, 2026ж

Пәннің жұмыс бағдарламасы (Силлабус) «26» мамыр 2023 ж. университеттің Ғылыми кеңесі отырысының №16 хаттамасымен бекітілген «6В06101-Бағдарламалық инженерия» білім беру бағдарламасы негізінде жасалды.

Жұмыс бағдарламасы, «Ақпараттық жүйелер» ББББТ отырысында қаралды және мақұлданды.
Хаттама № 10 «15» маусым 2026ж.

ББББТ жетекшісі Шаушенова А.Г.

Жұмыс бағдарламасы, Бизнес және цифрлық технологиялар институтының Академиялық сапа жөніндегі кеңесінің отырысында қаралды және ұсынылды.
Хаттама № 11 « 24 » маусым 2026ж.

Академиялық сапа жөніндегі Кеңес төрайымы Касимбекова М.А.

1. ОҚЫТУШЫ ЖАЙЛЫ МӘЛІМЕТ

Найзағараева Ақгүл Аманжолқызы «Ақпараттық жүйелер» ББББТ-ның PhD, аға оқытушысы, 2408, 2410, 2406 сабақ кестесі бойынша.

2. КУРСТЫҢ ҚЫСҚАША СИПАТТАМАСЫ

ПӘНДІ ОҚЫТУ МАҚСАТЫ

Пәннің мақсаты - web- технологиялар негіздері, Интернет желісіне қатынас жасау технологиялары, Интернеттегі web-серверлер, статистикалық және динамикалық web- сайттар, қолданбалы бағдарламалар және программалау арқылы Web -бағдарламалау жобалау негіздері қарастырылады.

Код	ББ бойынша қалыптастырылатын оқыту нәтижелері	Пәнді оқыту нәтижелері
ON 7	Бағдарламалау технологиясы құралдарымен әртүрлі күрделіліктегі есептерді шешу, БҚ компоненттерін инспекциялауды жүргізу.	– IP протоколдарының 4 және 6 нұсқаларының ерекшеліктері; – IP туннелдері. Браузерлердің ерекшеліктері; – сайттарды жобалау.
ON 8	АТ жобасының жұмысын жоспарлау, БҚ және қолданбалы АЖ енгізуге, бейімдеуге және күйге келтіруге, сонымен қатар ғылыми зерттеулерге қатысу.	– гипермәтіндік ақпараттық жүйелерді құру принциптері; – клиенттік веб-технологиялар: HTML, CSS, JavaScript, HTML5, Ajax, JQuery, XML; JavaScript, jQuery сценарий тілі.
ON 10	Интернетті ұйымдастыру және жұмыс істеу технологияларын, қағидаттарын пайдалану, ақпараттық қауіпсіздік қатерлеріне талдау жүргізу.	– бағдарламалау PHP, PHP7; – Framework YII, Laravel. SQL сұрау тілі; – ДҚБЖ MySQL, PostgreSQL – академиялық адалдықты сақтауды; – академиялық адалдық мәдениетін сақтауды; – академиялық адалдық қағидаттарын және жасанды интеллект құралдарын этикалық пайдалану талаптарын сақтайды; – белгіленген талаптарға сәйкес жасанды интеллект құралдарын пайдалану арқылы алынған ақпараттың дұрыстығын бағалайды.

ПРЕРЕКВИЗИТТЕРІ

Берілген пәнді зерделеу үшін келесі пәндерді (бөлімдерді) меңгеру қажет:
«Ақпараттық жүйелер негіздер», «Алгоритмдеу және бағдарламалау негіздері», «Ақпараттық жүйелердегі деректер қоры», «Бағдарламалау технологиялары», «Объектіге-бағытталған бағдарламалау».

ПОСТРЕКВИЗИТТЕРІ

Мамандандыру пәндері және осы курстан алған білімдері мен дағдыларын студенттер курстық жобалау, диплом жұмыстарында және болашақ кәсіби мамандықтарында қолдана алады.

3. КУРС МАЗМҰНЫ

3.1 Дәріс сабақтарының тізімі

Модуль атауы	Тақырыбы	Оқыту әдісі	Апта	Көлемі, сағат
WEB-ТЕХНОЛОГИЯЛАРДЫ ҚОЛДАНУ ОРТАЛАРЫ Интернеттегі клиент-серверлік архитектура	Web – бағдарламалау негіздері. Web стандарттары және олардың дамуы	ауызша әдістер (әңгімелесу, пікірталастар)	**1-апта**	**1**
	Интернет коммуникациясының моделі. Коммуникацияның Pull- және Push-моделі. Интернет сервисінің коммуникациялық сипаттамалары	ақпараттық-хабарлау	**2-апта**	**1**
	Қолданушының Интернетке кіруі. «Соңғы миля». Коммутация каналының желісі. Модем	ақпараттық-хабарлау	**3-апта**	**1**
	Интернет жүйесіне қатынасу технологиялары. DSL, ISDN технологиялары	түсіндермелі-иллюстративтік әдіс	**4-апта**	**1**
	Интернеттегі web серверлер. Web – сервер жұмысының механизмі. Статистикалық және динамикалық беттер. CGI технологиясы. Скриптер	түсіндермелі-иллюстративтік әдіс	**5-апта**	**1**
	Web-ресурстарды алу технологиясы. Web-парақтарды кэштау. Прокси-сервер механизімінің жұмысы	эвристикалық	**6-апта**	**1**
	Интернеттің іздеу технологиялары	түсіндірмелік-иллюстрациялық	**7-апта**	**1**
	Интернет сервистері. Электрондық пошта. E-mail сервисінің жұмыс істеу механизмі. E-mail-дің екі протоколы.	эвристикалық	**8-апта**	**1**
	Интернет сервисі. Файлдармен алмасу. FTP сервисінің жұмыс механизмі. FTP – клиенттер	эвристикалық	**9-апта**	**1**
	Интернеттегі мәліметтерді қорғау. Цифрлық қолтаңба. Криптография және Интернет. Симметриялық және асимметриялық кілттер. Цифрлық қолтаңбаның механизмі. Қауіпсіздік проблемалары	түсіндермелі-иллюстративтік әдіс	**10-апта**	**1**

WEB ҚОСЫМШАЛАРЫН ҚҰРУ ТЕХНОЛОГИЯЛАРЫ Web-бағдарламалау ортасы	Интернетте пайдаланушыларды идентификациялау. Cookies механизмі. Cookies-ті қолдану. Cookies негізіндегі технологиялардың кемшіліктері	эвристикалық	**11-АПТА**	**1**
	Бір WEB-парақта фреймдер арқылы бірнеше құжаттарды орналастыру. HTML тілінің мультимедиялық мүмкіндіктері. HTML құжаттарындағы формалар	оқу пікірталас	**12-АПТА**	**1**
	СТИЛЬДЕРДІҢ САТЫЛЫ КЕСТЕЛЕРІ (CSS) Жеке бір тәг үшін жазылған стиль. Жеке HTML-файлына арналған стиль. Бірнеше HTML-файлдарға арналған стиль. CSS қасиеттеріне шолу. CSS құру негіздері. Объектіні белгіленген орынға қою, z-index	мәселелік баяндау, зерттеушілік, кейс-стади	**13-АПТА**	**1**
	JAVASCRIPT ТІЛІ. Жалпы мағлұматтар. Литералдар. Айнымалылар. Әрнектер. HTML-құжатындағы сценарий. Функцияларды сипаттау және пайдалану. Циклдер мен функциялар. JavaScript функциялары. Объект түсінігі. Мұралау. Браузер объектілері мен оқиғалары. Оқиғалар	мәселелік баяндау, зерттеушілік, кейс-стади	**14-АПТА**	**1**
	Web-қосымшалар интерфейсі. HTTP хаттамасы. Perl тіліне кіріспе. PHP ТІЛІ НЕГІЗДЕРІ. Тіл синтаксисі мен грамматикасы. PHP тілі операторлары	мәселелік баяндау, зерттеушілік, кейс-стади	**15-АПТА**	**1**
	Барлығы			**15**

3.2 ЗЕРТХАНАЛЫҚ САБАҚТАР ТАҚЫРЫБЫ

Модуль атауы	**ТАҚЫРЫБЫ**	**ЗС ТАПСЫРМАЛАРЫ МЕН МАҚСАТЫ**	**ОҚЫТУ ӘДІСІ**	**КӨЛЕМІ, САҒАТ**	**АПТА**
	WEB-сервермен байланыс орнату	Жұмыстың атауы: Visual Studio Code бағдарламасын орнату. Теориялық бөлім: Visual Studio Code (VS Code) - Microsoft компаниясы әзірлеген бастапқы код редакторы. Ол Windows,	хабарлау әдісі	**2**	**1-апта**

		Linux және macOS операциялық жүйелерінде жұмыс істейді.			
web-технологиялардың қолдану орталары мен негіздері қарастырылады. Web-қосымшалар интерфейсі Web-бағдарламалау ортасы	Қолданушы идентификациясы	Жұмыстың атауы: HTML және оның негізгі элементтеріне кіріспе. **Теория:** HTML (Hypertext Markup Language) - веб-беттердің құрылымы мен мазмұнын жасау үшін қолданылатын стандартты белгілеу тілі. Ол тақырыптар, абзацтар, тізімдер, сілтемелер, суреттер және тағы басқа элементтерден тұрады.	хабарлау әдісі	**2**	**2-апта**
	Клиент - сервер технологиясының WEB- ресурстарын қолдау	Эксперимент атауы: Кестелерге арналған HTML бағдарламасын жазу. Теория: Кесте - жолдар мен бағандардың орналасуынан тұрады. HTML негіздерін біле отырып, кез келген адам кесте жасай алады. HTML тілінде кесте <table> тегі арқылы анықталады.	эвристикалық әдіс	**2**	**3-апта**
	WEB – қосымшалардағы пайдаланушының тіркеу формасы	HTML: тізімдер Теория: HTML тізімдері ақпарат тізімдерін көрсету үшін қолданылады. Барлық тізімдер бір немесе бірнеше тізім элементтерінен тұруы мүмкін. HTML тізімдерінің үш негізгі түрі бар: 1. Реттелген тізім (ol) 2. Реттелмеген немесе маркерленген тізім (ul) 3. Сипаттамалар тізімі немесе анықтамалар тізімі (dl).	мәселелік оқыту	**2**	**4-апта**
	WEB-қосымшаның ақпаратты қорғау элементтері бөлімі	HTML: формалар, мультимедиа, фреймдер және iFrame Теория: HTML-форма - веб-беттің әртүрлі элементтерді, мысалы, енгізу өрістерін, батырмаларды,	зерттеушілік әдіс	**2**	**5-апта**

		жалаушаларды, радио батырмаларды және тағы басқаларды қамтитын, пайдаланушыларға өңдеу үшін деректерді веб-серверге жіберуге мүмкіндік беретін бөлімі.			
	WEB-қосымшадағы мәліметтер базасы бар серверлер	HTML мультимедиялық тегтеріне кіріспе. Теория: HTML мультимедиялық тегтері веб-беттерге кескіндер, аудио және бейне сияқты әртүрлі мультимедиялық контент түрлерін кірістіру және басқару үшін қолданылады. Мұндай тегтерге кескіндерге арналған <img>, дыбысқа арналған <audio> және бейнені ойнатуға арналған <video> тегтері жатады. Олар мультимедиялық элементтерді веб-бетке біркелкі кіріктіруге мүмкіндік береді.	бинарлық әдістер	**2**	**6-апта**
	Web-қосымшаларды жасау технологиялары. Гипермәтінді белгілеу тілдері. HTML, DHTML тілдері	CSS-ке кіріспе. Теория: CSS (Cascading Style Sheets - каскадты стильдер кестесі) - веб-беттерді безендіру және олардың орналасуын реттеу үшін қолданылатын тіл. Ол HTML элементтерінің сыртқы көрінісін, мысалы, түстерді, қаріптерді, аралықтарды және жалпы орналасуды басқарады. Контентті (HTML) дизайннан (CSS) бөлу арқылы әзірлеушілер сыртқы көрінісі тартымды, қолдауға және жаңартуға ыңғайлы веб-сайттар жасай алады. CSS сонымен қатар адаптивті дизайн жасауға мүмкіндік береді, яғни веб-беттер әртүрлі экран өлшемдері мен құрылғыларда әдемі көрінеді.	зерттеушілік әдіс	**2**	**7-апта**

	Сілтемелерді қолдану. Статикалық айнымалыларды қолдану	CSS позициялау Теория: CSS позициялау веб-беттегі элементтердің орналасуын басқару үшін қолданылады. Ол әзірлеушілерге статикалық (static), салыстырмалы (relative), абсолютті (absolute), бекітілген (fixed) және жабысқақ (sticky) сияқты қасиеттерді пайдаланып, элементтерді қажетті орынға орналастыруға мүмкіндік береді.	эвристикалық әдіс	**2**	**8-апта**
	Style, Location обьектілерін қолдану	CSS Flexbox және Grid Теория: Flexbox: Flexbox - элементтерді негізінен жол (row) немесе баған (column) түрінде орналастыруға арналған бір өлшемді макет моделі. Ол қолжетімді бос кеңістікке байланысты контейнер ішіндегі элементтердің өлшемі мен орналасуын динамикалық түрде өзгертуге мүмкіндік береді және күрделі макеттерді құруды жеңілдетеді.	бинарлық әдістер	**2**	**9-апта**
	.MySQL деректер қорымен жұмыс	JavaScript-ке кіріспе Теория: JavaScript - веб-беттерді интерактивті және динамикалық ететін танымал бағдарламалау тілі. Ол HTML элементтерін басқара алады, пайдаланушының батырманы басу сияқты әрекеттеріне жауап береді және бетті қайта жүктемей-ақ контентті жаңарта алады. JavaScript формаларды тексеру, анимациялар жасау және күрделі веб-қосымшаларды әзірлеу сияқты тапсырмалар үшін қажет. Ол	эвристикалық әдіс	**2**	**10-апта**

		тікелей браузерде жұмыс істейді, сондықтан заманауи веб-әзірлеудің маңызды құралы болып табылады.			
	Cookies технологиясы	JavaScript операторлары, циклдері, функциялары және объектілері Тәжірибенің тақырыбы: JavaScript тіліндегі операторларға, циклдерге және пайдаланушы функцияларына кіріспе.	мәселелік оқыту	**2**	**11-апта**
	web-қосымшаларын құру технологиялары сипатталып, HTML гипермәтінді белгілеу тілінің мнмкіндіктері келтірілген және оның web-парақтар жасай алатын басқа тілдермен байланысу жолдары жайлы айтылады.	JS тілінде DOM-мен өзара әрекеттесу және манипуляция жасау Эксперимент атауы: JavaScript тілінде DOM-мен өзара әрекеттесу және манипуляция жасау.	зерттеушілік әдіс	**2**	**12-апта**
	стильдердің сатылы кестелерін (Cascade Style Sheets) қолдану мәселелері туралы жазылған, онда CSS сипаттамаларының синтаксисі, стильдерді құжат ішінде немесе сыртында беру, CSS атрибуттарының жиі қолданылатын атрибуттары, элементтерді позициялау тәсілдері	PHP және веб-сервер: кіріспе және баптау Эксперимент атауы: PHP-ке кіріспе және XAMPP серверін орнату. Теория: PHP - веб-әзірлеуге бағытталған әмбебап скрипттік тіл. Оны 1993 жылы даниялық-канадалық бағдарламашы Расмус Лердорф жасаған және 1995 жылы шығарған. Қазіргі уақытта PHP-тің эталондық жүзеге асырылуын PHP Group әзірлейді. PHP бастапқыда Personal Home Page сөздерінің қысқартылған атауы болған, ал қазір ол рекурсивті инициализм болып табылатын PHP: Hypertext Preprocessor дегенді білдіреді. PHP - бэкенд үшін кеңінен қолданылатын ашық бастапқы кодты	эвристикалық әдіс	**2**	**13-апта**

		скрипттік тіл. PHP файлдарының кеңейтімі - .php.			
	динамикалық web-парақтар жасай алатын JavaScript программалау тіліне арналып, World Wide Web ортасында осы тілді пайдалану жолдарынан мысалдар мен программалық кодтау	PHP және веб-сервер: кіріспе және баптау Эксперимент атауы: PHP-ке кіріспе және XAMPP серверін орнату. Теория: PHP - веб-әзірлеуге бағытталған әмбебап скрипттік тіл. Оны 1993 жылы даниялық-канадалық бағдарламашы Расмус Лердорф жасаған және 1995 жылы шығарған. Қазіргі уақытта PHP-тің эталондық жүзеге асырылуын PHP Group әзірлейді. PHP бастапқыда Personal Home Page сөздерінің қысқартылған атауы болған, ал қазір ол рекурсивті инициализм болып табылатын PHP: Hypertext Preprocessor дегенді білдіреді. PHP - бэкенд үшін кеңінен қолданылатын ашық бастапқы кодты скрипттік тіл. PHP файлдарының кеңейтімі - .php.	практикалық тапсырмалар, зерттеушілік әдіс, кейс-стади	**2**	**14-апта**
	желілердегі сервер жағынан әр тнрлі іс-әрекеттер атқара алатын Perl тілінің мүмкіндіктері	Деректерді жинауға және оларды деректер базасында сақтауға арналған PHP формасы. Жұмыс атауы: XAMPP серверін пайдаланып, PHP және MySQL көмегімен HTML-де тіркелу және жүйеге кіру формасын жасау.	практикалық тапсырмалар, зерттеушілік әдіс, кейс-стади	**2**	**15-апта**
	Барлығы			**30**	

3.3 ПӘН БОЙЫНША СӨЖ ТАПСЫРМАЛАРЫН ОРЫНДАУ ЖӘНЕ ТАПСЫРУ КЕСТЕСІ:

№	Модуль атауы	Сабақтың тақырыбы	БӨЖ тапсырмалары	Тапсырманы бақылау формасы	Тапсыр-маны тапсыру мерзімі
1	2	3	4	6	7
1	Интернеттегі клиент-серверлік архитектура Веб құрылымы және принциптері. PHP тілі. PHP тілінің басқарушы конструкциялары. PHP тілінде масссивтер, файлдар.	Интернетпен жұмыс істеуге арналған қызметтік утилиталар. HTTP хаттамаларын оқып білу.	«Web-технологиялардың даму эволюциясы». Web, WWW, HTML, CSS, JavaScript ұғымдарын зерттеңіз. Web 1.0, Web 2.0 және Web 3.0 ерекшеліктерін салыстырыңыз. W3C және Web стандарттарының маңызын сипаттаңыз. HTML5, CSS3 және JavaScript-тің қазіргі Web-қосымшалардағы рөлін мысалдармен түсіндіріңіз.	Бағалау қарастырылмаған	1,2
2		PHP бағдарламалау тілі. Негізгі операторлар.	PHP бағдарламалау тілі. Негізгі операторлар.	Әдебиеттердің библиографиялық тізімін тексеру; Дереккөздердің сенімділігін бағалау	3
3		Тармақталған және цикл алгоритмдерін бағдарлама алгоритмдерін бағдарлама лау	PHP бағдарламалау тілі. Басқарушы конструкциялар	Жеке тапсырманың орындалуын тексеру;	4,5
5		Массивтерді өңдеу алгоритмде рін бағдарлама лау	Массивтермен жұмыс.	Жеке тапсырманың орындалуын тексеру;	6

6		Екі өлшемді массивтерді өңдеу алгоритмдерін бағдарламалау және файлдармен жұмыс	PHP-де екі өлшемді массивтермен жұмыс. Файлдар	Жеке тапсырманың орындалуын тексеру; Бағдарламалық құралдар ортасында орындалған жұмысты тексеру	7,8
7	PHP-де деректер қорымен жұмыс істеу	Деректер қорымен жұмыс	MySQL деректер қорымен жұмыс істеуге арналған PHP функциялары	Жеке тапсырманың орындалуын тексеру; Бағдарламалық құралдар ортасында орындалған жұмысты тексеру	10, 11
8		GET және POST әдістерін қолдану	GET және POST әдістері. Деректер қорымен жұмыста POST әдісін қолдану	Есептеулердің дұрыстығын тексеру; Модельдеу нәтижелерін талдау; Болжамдардың дәлдігін бағалау	12
9		Функциялар, сессиялар және кукилар құру	PHP-де функциялар. Сессиялар (сеанстар) және кукилар	Есептеулердің дұрыстығын тексеру; Модельдеу нәтижелерін талдау; Болжамдардың дәлдігін бағалау	13
10	Web-бағдарламалау ортасы	Объектіге-бағытталған бағдарламалау әдістерін қолдану	PHP-де объектіге-бағытталған бағдарламалау. Мұрагерлік. Конструктор және деструктор	Есептеулердің дұрыстығын тексеру; Модельдеу нәтижелерін талдау; Болжамдардың дәлдігін бағалау	14
11		Объектіге-бағытталған бағдарламалау әдістерін қолдану	Әдістерді қайта жүктеу. Класстарды автоматты түрде жүктеу	Есептеулердің дұрыстығын тексеру; Модельдеу нәтижелерін талдау; Болжамдардың дәлдігін бағалау	15

4. ОҚЫТУДЫҢ НЕГІЗГІ ФОРМАЛАРЫ МЕН ӘДІСТЕРІ

Курсты оқу кезінде оқытудың келесі әдістері қолданылады:

Ауызша немесе вербальды әдіс. Бұл әдіс дәріс, әңгімелеу, әңгіме т.б. қамтиды.

Пән бойынша оқыту білім берудің дәрістер, практикалық сабақтар, өзіндік жұмыс (БОӨЖ және БӨЖ) түріндегі ұйымдастырушылық нысандары арқылы жүргізіледі.

Дәрістер– түсіндірмелік-иллюстрациялық, мәселелік баяндау, эвристикалық, зерттеушілік дәрістер түрінде өткізіледі.

Практикалық сабақтар– практикалық тапсырмаларды шешу, пікірталастар, кейстерді талдау, жобалық әдіс, эвристикалық пікірлесу, топтық жұмыс және т.б. әдістерді қолдану негізінде жүргізіледі. Тақырыптардың меңгерілу деңгейін анықтау және бағалау үшін жазбаша бақылау жұмыстары (жалпыға ортақ не жеке нұсқа түріндегі тапсырма), Platonus жүйесінде ағымдық тест түріндегі бақылау, коллоквиум өткізіледі.

Өзіндік жұмыс– әр тақырыпқа сәйкес дәріс материалдарын қайталау, тақырыптың мазмұнын терең түсіну үшін қосымша әдебиеттерді оқу (алдын-ала оқу), практикалық сабақ тапсырмаларын орындау, өзіндік жұмысқа берілген арнайы тақырыптарды меңгеру, жеке жобалық, зерттеу тапсырмаларын орындау, презентациялар дайындау, ағымдық бақылау және тест тапсырмаларына дайындалу, ғылыми-зерттеу жұмысын жүргізу және т.б. бағыттарды қамтиды.).

5. ӘДЕБИЕТТЕР ТІЗІМІ

Негізгі әдебиеттер

1. Ибрагимов О.М., Нысанов Е.А., Құрақбаев Ж.С. Web-бағдарламалау негіздері. – Алматы: TechSmith, 2024. – 248 б.
2. Асылбеков У.Б., Исмаилова А.А. Web-технологиялар. 1-бөлім. – Алматы: Бастау, 2019. – 324б.
3. Асылбеков У.Б., Исмаилова А.А. Web-технологиялар. 2-бөлім. – Алматы: Бастау, 2019. – 248б.
4. Бөрібаев Б., Абдрахманова М. Web-технологиялар: HTML және PHP тілдері. – Алматы: Қазақ университеті, 2015. – 241 б.
5. Бөрібаев Б., Абдрахманова М. Web-технологиялар: JavaScript және PHP тілдері. – Алматы: Қазақ университеті, 2023. – 183 б.

Қосымша әдебиеттер

1. Фримен Э. Изучаем HTML, XHTML и CSS. – СПб.: Питер, 2010. – 656 с.
2. Фримен Э., Робсон Э. Изучаем программирование на JavaScript. – СПб.: Питер, 2015. – 640 с.
3. Флэнаган Д. JavaScript. Подробное руководство. – СПб.: Символ-Плюс, 2013. – 1080 с.
4. Морган Н. JavaScript для детей. Самоучитель по программированию. – М.: Манн, Иванов и Фербер, 2016. – 288 с.
5. Макфарланд Д. Новая большая книга CSS. – СПб.: Питер, 2016. – 720 с.
6. Кит Дж. HTML5 для веб-дизайнеров. – СПб.: Питер, 2014. – 176 с.
7. Мейер Э. CSS. Каскадные таблицы стилей. Подробное руководство. – СПб.: Символ-Плюс, 2010. – 1088 с.
8. Кастро Э., Хайсмит Б. HTML5 и CSS3. Веб-разработка по стандартам нового поколения. – М.: Эксмо, 2014. – 368 с.
9. Симпсон К. Вы не знаете JS. Познание JavaScript. – М.: Вильямс, 2016. – 352 с.
10. Симпсон К. Вы не знаете JS. Области и замыкания. – М.: Вильямс, 2016. – 192 с.
11. Хавербеке М. Выразительный JavaScript. Современное программирование для Интернета. – СПб.: Питер, 2019. – 480 с.
12. МакФарланд Д. JavaScript и jQuery. Исчерпывающее руководство по программированию. – СПб.: Питер, 2015. – 880 с.

13. Никсон Р. Создаем динамические веб-сайты с помощью PHP, MySQL, JavaScript, CSS и HTML5. – СПб.: Питер, 2019. – 816 с.
14. Лабрс Б., Пенна М. HTML и CSS. Разработка веб-сайтов. – М.: ДМК Пресс, 2018.
15. Фримен Э. Head First. HTML5. Программирование для веб-дизайнеров. – СПб.: Питер, 2014.

Электронные ресурсы
16. MDN Web Docs. HTML, CSS, JavaScript, Web APIs. – электронный ресурс.
17. W3Schools. HTML, CSS, JavaScript Tutorials. – электронный учебный ресурс.
18. JavaScript.info. Современный учебник JavaScript. – электронный ресурс.
19. HTML Academy. Интерактивные курсы по HTML, CSS и JavaScript. – электронный образовательный ресурс.
20. Learn JavaScript. Учебные материалы по современному JavaScript. – электронный ресурс.

6. КУРСТЫҢ САЯСАТЫ

Курс саясаты «С. Сейфуллин атындағы ҚАТЗУ» КеАҚ Академиялық саясатының талаптарына сәйкес ұйымдастырылған. Осы пәнді оқу барысында оқытушы мен білім алушылар университетте қабылданған білім беру процесіне қатысушылардың Академиялық адалдық кодексінің (7-тармақ) талаптарын ұстануы тиіс. Жасанды интеллект құралдарын пайдалану университеттің ЖИ пайдалану саясатына және оқытушының талаптарына сәйкес жүзеге асырылады.

Барлық білім алушыларға сабақтардағы оқу тақырыптарын талқылауға тең мүмкіндік беріледі. Силлабуста көрсетілген оқу тақырыптары бойынша барлық білім алушылар кеңес алуға, сұрақтар қоюға және жауаптар алуға құқылы. Оқытушының тапсырмаларын орындау кезінде ойлау ерекшелігі, білім алушылардың шығармашылық көзқарасы құпталады. Ерекше білім беру қажеттіліктері бар білім алушылар оқытуда жеке көзқарас құқығын пайдалана алады.

Барлық білім алушылардан мінез-құлықтың академиялық мәдениетін сақтау, бір-біріне өзара құрмет көрсету, академиялық адалдық ережелерін ұстану, сабақтарды себепсіз босатпау, сабақтарға кешікпеу талап етіледі.

Білім алушылардың ағымдық бақылауы пәннің жұмыс оқу жоспарына (силлабус) және "Білім алушылардың үлгерімін ағымдық және аралық аттестаттауды жүргізу" СМЖ талаптарына сәйкес жүргізіледі және «Платонус» ААЖ-де электрондық журнал толтырылады:

–берілген тапсырма уақытында себепсіз орындалмағаны үшін тиісті балл әр аптасына *10% кемітіледі*, тапсырманы беліленген уақыттан 2 аптаға кешіктіруге болмайды;

–тапсырылмаған тапсырма үшін *«0» балл қойылады*;

–жоспарланған ағымдық бақылау жұмыстарына, тестілеуге, коллоквиумдарға себепсіз қатыспаған білім алушыға ағымдық бақылауға *«0» балл қойылады*.

Білім алушылардың оқу жетістіктерін бақылаулар кезінде бағалау келесідей критерийлер бойынша жүргізіледі:

– тапсырмаларды орындау мерзімдерін сақтау;

– ғылыми-зерттеу тапсырмасының толықтығы мен қойылған талаптарға сәйкестігі;

–зерттеу нәтижелерін ғылыми тұрғыдан негіздеп, түсіндіре білу;

–зерттеу мәселесін сыни тұрғыдан талдау және алынған нәтижелер бойынша қорытынды жасай білу;

–зерттеу нәтижелерін кестелер, графиктер, диаграммалар және басқа да көрнекі құралдар арқылы ұсына білу;

–зерттеу нәтижелерін нақты қаржылық көрсеткіштермен және практикалық мысалдармен байланыстыра білу;

–зерттеу барысында академиялық адалдық қағидаларын және ЖИ-ді этикалық пайдалану талаптарын сақтау.

7. БІЛІМ АЛУШЫЛАРДЫҢ ОҚУ ЖЕТІСТІКТЕРІН БАҒАЛАУ САЯСАТЫ

Білім алушылардың оқу жетістіктерін бағалау саясаты университет сайтының 9-тармағындағы академиялық саясатта ұсынылған талаптарға сәйкес келеді (сайтқа сілтеме: *https://kazatu.edu.kz/ru/pages/obucenie/akademiceskaa-politika*).

Білім алушылардың оқу жетістіктері (білімі, іскерліктері, дағдылары мен құзыреттері) халықаралық практикада қабылданған цифрлық эквиваленті бар әріптік жүйеге сәйкес келетін 100 балдық шкала бойынша бағаланады.

Білім алушылардың үлгерімін ағымдағы бақылау пәнннің әрбір тақырыбы бойынша жүргізіледі және аудиториялық және аудиториядан тыс сабақтардағы білімді бақылауды қамтиды.

Білім алушының үлгерімін ағымдық және аралық бақылау бағалары негізінде қорытынды емтиханға жіберуді бағалау қалыптастырылады.

Ағымдық бақылауды (АБ) есептеу мына формула бойынша жүргізіледі:

1АБ, 2АБ (сабақ түрі) = дәріс/практикалық / сем /зерт/БӨЖ /БОӨЖ (1+2+n)/n

Білім алушының білімін аралық (рейтингтік) бақылау есебі әр семестрдің 7 (1 АБ) және 15 (2АБ) апталарының соңында автоматты түрде есептеледі.

Рейтинг (1Р, 2Р) ағымдық (1 АБ жалпы, 2 АБ жалпы) және аралық бақылаудың (1 АБ, 2 АБ) орташа арифметикалық сомасы ретінде айқындалады.

1АБ (2)=(1АБ (2)жалпы+1 АБ (2))/2

Қабылдау рейтингі (ҚР) аралық бақылаудың орташа арифметикалық сомасы ретінде айқындалады (1 ҚР, 2 ҚР).

ҚР=(1Р+2Р)/2

Білім алушы міндетті түрде 50% - дан кем емес рұқсат беру рейтингін (ҚР) алған кезде емтиханға жіберіледі.

7.1 БІЛІМ АЛУШЫЛАРДЫҢ ОҚУ ЖЕТІСТІКТЕРІН БАҒАЛАУ КРИТЕРИЙЛЕРІ.

Білім алушының оқу жетістіктерін бағалау критерийлері

Әріптік жүйе бойынша бағалау	**Баллдардың сандық баламасы**	**%-дық мазмұны**	**Дәстүрлі жүйе бойынша бағалау**	**Бағалау критерийлері**
A	4,0	95–100	Өте жақсы	Web бағдарламалау технологияларын толық әрі терең меңгерген. HTML5, CSS3 және JavaScript мүмкіндіктерін кәсіби деңгейде қолданады. Семантикалық HTML құрылымын, адаптивті және заманауи интерфейс дизайнын өз бетінше әзірлейді. JavaScript арқылы DOM элементтерімен, оқиғалармен, массивтермен, объектілермен және функциялармен тиімді жұмыс істейді. Асинхронды бағдарламалауды, Promise, async/await және API-мен өзара әрекеттесуді дұрыс қолданады. Күрделі, функционалды және пайдаланушыға ыңғайлы Web-қосымшаларды дербес әзірлейді.

Әріптік жүйе бойынша бағалау	Баллдардың сандық баламасы	%-дық мазмұны	Дәстүрлі жүйе бойынша бағалау	Бағалау критерийлері
				Кодтың құрылымын, өнімділігін және қауіпсіздігін талдап, оңтайландырады. Қабылданған техникалық шешімдерді негіздейді және күрделі практикалық есептерді шығармашылықпен шешеді.
A-	3,67	90–94	Өте жақсы	Web бағдарламалаудың негізгі және заманауи технологияларын толық меңгерген. HTML, CSS және JavaScript көмегімен функционалды Web-беттер мен қосымшалар әзірлейді. Семантикалық құрылым, CSS стильдері, Flexbox/Grid және адаптивті дизайнды дұрыс қолданады. JavaScript арқылы DOM, оқиғалар, функциялар, массивтер және объектілермен жұмыс істейді. API, Promise және async/await технологияларын қолдана алады. Web-қосымшаның құрылымын дұрыс ұйымдастырып, кодтағы қателерді өздігінен анықтап, түзетеді.
B+	3,33	85–89	Жақсы	Web бағдарламалаудың негізгі технологияларын жақсы меңгерген. HTML құжатының құрылымын дұрыс жасайды, CSS арқылы Web-беттерді безендіреді және адаптивті интерфейс әзірлейді. JavaScript көмегімен интерактивті элементтер, оқиғаларды өңдеу және динамикалық мазмұн құрады. DOM элементтерімен жұмыс істейді, функциялар, массивтер және объектілерді қолданады. Қарапайым API сұраныстарын орындайды және алынған деректерді Web-бетте көрсетеді. Практикалық тапсырмаларды негізінен дұрыс орындайды.
B	3,0	80–84	Жақсы	Web бағдарламалаудың негізгі ұғымдары мен принциптерін түсінеді. HTML көмегімен Web-беттің құрылымын құрады, CSS арқылы оның сыртқы көрінісін қалыптастырады. JavaScript тілінің синтаксисін, айнымалыларды,

Әріптік жүйе бойынша бағалау	Баллдардың сандық баламасы	%-дық мазмұны	Дәстүрлі жүйе бойынша бағалау	Бағалау критерийлері
				деректер типтерін, шартты және циклдік конструкцияларды, функцияларды қолданады. DOM және оқиғалармен жұмыс істей алады. Қарапайым интерактивті Web-беттер әзірлейді. Жекелеген елеусіз қателер болғанымен, негізгі практикалық тапсырмаларды дұрыс орындайды.
B-	2,67	75–79	Жақсы	Web бағдарламалаудың негізгі ұғымдарын меңгерген. HTML тегтерін дұрыс қолданып, қарапайым Web-беттер жасайды. CSS көмегімен мәтіндерді, блоктарды, түстерді және басқа элементтерді безендіреді. JavaScript арқылы қарапайым есептерді шешеді, функциялар мен шартты операторларды қолданады. DOM элементтерін таңдап, қарапайым оқиғаларды өңдей алады. Практикалық жұмыстарды орындау кезінде жетекші сұрақтар арқылы түзетілетін кішігірім қателер болуы мүмкін.
C+	2,33	70–74	Қанағаттанарлық	Web бағдарламалау туралы негізгі түсініктері бар. HTML құжатының негізгі құрылымын құра алады және CSS көмегімен қарапайым безендіру орындайды. JavaScript тілінің негізгі конструкцияларын, айнымалыларды, шарттарды және циклдерді қолданады. Қарапайым интерактивті элементтерді жасай алады. DOM және оқиғалармен жұмыс істеуде кейбір қиындықтарға кездеседі. Практикалық тапсырмаларды орындау кезінде жекелеген қателер жібереді.
C	2,0	65–69	Қанағаттанарлық	Пән бойынша базалық білімді меңгерген. HTML, CSS және JavaScript тілдерінің негізгі қызметін біледі. Қарапайым Web-бетті құрып, оны CSS арқылы безендіре алады. JavaScript көмегімен қарапайым бағдарламалық конструкцияларды

Әріптік жүйе бойынша бағалау	Баллдардың сандық баламасы	%-дық мазмұны	Дәстүрлі жүйе бойынша бағалау	Бағалау критерийлері
				қолданады. DOM, оқиғалар және интерактивтілік ұғымдарын біледі, бірақ оларды практикалық қолдануда қиындықтарға кездеседі. Практикалық тапсырмаларды орындау кезінде оқытушының көмегін қажет етеді.
C-	1,67	60–64	Қанағаттанарлық	Web бағдарламалау бойынша толық емес білім көрсетеді. HTML тегтерін және CSS қасиеттерін қолдануда елеулі қателер жібереді. JavaScript-тің негізгі синтаксисін толық меңгермеген. Айнымалылар, шарттар, циклдер және функцияларды қолдануда қиындықтарға кездеседі. DOM және оқиғалармен жұмыс істеу дағдылары жеткіліксіз. Қарапайым Web-беттерді әзірлеу кезінде оқытушының тұрақты көмегін қажет етеді.
D+	1,33	55–59	Қанағаттанарлық	Пәннің негізгі тақырыптары бойынша үстірт білім бар. HTML құжатының құрылымы, CSS және JavaScript туралы түсінігі шектеулі. Қарапайым Web-бетті құру кезінде көптеген синтаксистік және құрылымдық қателер жібереді. JavaScript кодын жазу және DOM элементтерімен жұмыс істеуде елеулі қиындықтарға кездеседі. Практикалық тапсырмаларды орындау үшін оқытушының елеулі көмегін қажет етеді.
D	1,0	50–54	Қанағаттанарлық	Web бағдарламалау бойынша фрагментті және жүйесіз білім көрсетеді. HTML, CSS және JavaScript тілдерінің негізгі ұғымдарын анықтауда елеулі қателер бар. Қарапайым Web-беттің құрылымын өздігінен құруда, стильдерді қолдануда және JavaScript кодын жазуда қиындықтарға кездеседі. Практикалық тапсырмалардың тек қарапайым бөлігін ғана орындай алады.

Әріптік жүйе бойынша бағалау	Баллдардың сандық баламасы	%-дық мазмұны	Дәстүрлі жүйе бойынша бағалау	Бағалау критерийлері
FX	0,5	25–49	Қанағаттанарлықсыз	Пәннің негізгі материалын жеткіліксіз меңгерген. HTML, CSS және JavaScript бойынша білімінде елеулі олқылықтар бар. Web-беттің құрылымын құру, CSS арқылы безендіру және JavaScript кодын жазу кезінде көптеген қателер жібереді. DOM және оқиғалармен жұмыс бойынша практикалық тапсырмалардың едәуір бөлігін орындай алмайды немесе принциптік қателер жібереді.
F	0	0–24	Қанағаттанарлықсыз	Пәннің негізгі оқу материалын меңгермеген. HTML, CSS және JavaScript тілдерінің негізгі ұғымдарын білмейді. Web-беттің қарапайым құрылымын құра алмайды, CSS стильдерін және JavaScript конструкцияларын қолдану дағдылары қалыптаспаған. DOM, оқиғалар және Web-қосымша ұғымдарын түсінбейді. Практикалық тапсырмаларды орындай алмайды.

ҚАЗАҚСТАН РЕСПУБЛИКАСЫ АУЫЛ ШАРУАШЫЛЫҒЫ МИНИСТРЛІГІ
«С.СЕЙФУЛЛИН атындағы ҚАЗАҚ АГРОТЕХНИКАЛЫҚ ЗЕРТТЕУ УНИВЕРСИТЕТІ» КеАҚ

«WEB БАҒДАРЛАМАЛАУ»

ДӘРІСТЕР ЖИНАҒЫ

Астана, 2026ж

Дәріс №1.
Web - технологиялар негіздері. Web стандарттарыжәне олардың дамуы

Дәрістің мақсаты: байланысқан желілер, клиент-сервер түсініктерімен таныстыру.

Дәрістің мазмұны: «Желі» терминінің түсінігі. Клиент-сервер архитектурасы. Провайдерлер және олардың желілері. Компьютерлер-серверлер және компьютерлер-клиенттер. Қосымша-серверлер және қосымша-клиенттер. Провайдерлер желілерінің иерархиясы.

Қазіргі кезде Internet те телефон, факс, сияқты адам өмірінің ажырамас бөлігі болып табылады. Адамдардың барлығы қандай да бір ақпарат алу, бір-бірімен сөйлесу мақсатында Internet деп аталынатын дүниежүзілік өрмекте саяхат жасайды.

Интернет дегеніміз глобальді желі. Интернет физикалық мағынасында бір-бірімен байланыс линиялары арқылы байланысқан бірнеше миллиондаған компьютерлер деп қарастыруға болады. Бірақ Интернеттіфизикалық мағынада емес, оны ақпараттық кеңістік деп қарастырған дұрыс.Интернетті ішінде мәліметтердің үздіксіз циркуляциясы жүретін кеңістік ретінде қарастыруға болады. Интернеттің теле және радиоэфирден айырмашылығы – эфирде ешқандай ақпарат сақталынбайды, ал Интернетте ол желінің түйіндерінде болатын компьютерлер арасында орын ауыстырады
да, қандай да бір уақыт аралығында қатты дискіде сақталады.

Екі немесе одан да көп компьютерлер арасындағы физикалық байланысу компьютерлік желі деп аталынады. Жалпы компьютерлік желілерді құру үшін арнайы аппараттық қамсыздану және арнайы бағдарламалық қамсыздану қажет.

Компьютерлік желілер аппаратты ресурстардан басқа қосымша программалық ресурстарды да пайдалануға мүмкіндік береді.Компьютерлік желілер локальді және глобальді болып бөлінеді.

Интернет – глобальді желі. Қазіргі кезде әлемде 200-ден астам глобальді желі бар. Барлық әлемдегі глобальді есептеуіш желілер өзараИнтернет көмегімен байланысқан.

ҒЕЖ-де жұмыс істеу үшін сәйкес аппаратты және программалық қамсыздану болу керек. Барлық қамсыздану 2 топқа бөлінеді:

1) Бағдарлама-серверлер - қолданушының компьютеріне қызмет көрсететін желі түйінінде орналасатын бағдарламалық қамсыздану.

2) Бағдарлама-клиенттер – қолданушының компьютерінде орналасады және сервердің қызметін пайдаланатын бағдарламалыққамсыздану.

Заманауи интернет технология бір орында тұрмайды. Қазіргі кезде адамның күнделікті өмірін интернеттің көмегінсіз елестету мүмкін емес. Интернет ол әлемдегі ең үлкен желі. Оның бірегей басқару орталығы болмайды, бірақ ол белгілі бір ереже бойынша жұмыс жасайды және өзінің қолданушыларына қызметтер жиынтығының бірегейлігін ұсынады. Интернетті «желілердің желісі» ретінде қарастыруға болады, олардың әрқайсысы бір-біріне тәуелсіз операторлармен басқарылады – Интернет қызметінің провайдерлері (ISP, Internet Service Provider).

1. Өзара әсерлесетін және бір-бірімен байланысқан компьютерлер тобы өрмекті құрайды. Интернет – бүкіл әлем бойынша таратылған мыңдаған кіші желілерден тұратын компьютердің үлкен өрмегі.

Интернеттегі компьютерлер бір ортақ тілде сөйлеседі, ал бұл тіл протоколдарда бекітілген.

Протокол деген желідегі компьютерлердің өзара әрекеттесу механизмін анықтайтын жалпыға бірдей ережелер жиынтығы. Интернеттегі мәліметтермен алмасуға арналған стандартты протоколдар: TCP және IP. Осы екі протоколдың комбинациясы TCP/IP түрінде болады.

2. Интернетке қатысты қолданылатын терминдер:

1) Айна - алыста орналастырылған атақты сервердің көшірмесіболатын сервер.

2) Прокси-сервер - желідегі жүктемені төмендететін және желідегі іс-әрекетті тездететін компьютер.

3) Шлюз - әртүрлі протоколдармен жұмыс жасайтын әр текті екі желіні біріктіретін бағдарламалық-аппараттық құрал.

4) Брандмауэр – қорғауға алынған желіге санкциясыз енуді тоқтатын аппараттық құрал.

5) Маршрутизатор – бірнеше каналдармен жұмыс істейтін құрылғы.

6) Маршрутизациялау – ақпараттың жіберілу жолын таңдау процедурасы.

3. Интернет көптеген әртүрлі ұйымдардың компьютерлерінбіріктіретін болса, Интранет бір ғана ұйымның ішіндегі компьютерлерді біріктіреді. Интранеттегі ақпарат тек қана сол ұйымның адамдары пайдалана алады. Интранет желілері электронды пошта секілді бизнес үшін өте тиімді коммуникациялық құрал болып табылады.

Қазіргі таңдағы Интернет желісі, өзінің қарқынды дамып келе жатқаны үшін ARPANET біріктірілген желісіне қарыздар. Бұл желі коммутация пакеттері технологиясының қарапайым эксперименті ретінде басталған.

Интернет өте күрделі желі болып табылғандықтан, сәйкесінше, желі құрылғылары арасында қарым-қатынасты ұйымдастыру тапсырмалары да күрделі болып келеді. Мұндай күрделі тапсырманы шешу үшін декомпозиция қолданылады. Бұл дегеніміз күрделі тапсырмаларды бірнеше қарапайым тапсырма-модельдерге жіктеу.

Интернет сөзі Interconnected networks (байланысқан желілер) терминінен шыққан, яғни техникалық көзқараспен – бұл кіші және ірі желілер бірлестіктері. Кең мағынасында – бұл, бір-бірімен мәліметтер алмасатын жер жүзіндегі миллиондаған компьютерлер арасында бөлінген ақпараттық кеңістік. Көбіне Интернет сөзімен Желінің ақпараттық құрамынтүсінеді. Интернет – бұл өзіне уникальды жетістіктерді толығымен жинаған технология. Интернет сонымен қатар, ең күшті және тәуелсіз ақпарат қоры, байланыстың сенімді және оперативті тәсілі, жер жүзіндегі миллиондаған адамдардың шығармашылық түрде өзін-өзі көрсету және ақпараттық технологияларды дамыту негізі болып табылады.

Интернеттің басты тапсырмасы бұл – әр тәуліктік, жоғары сенімді байланыс. Интернетке қосылған кез келген екі компьютер (немесе басқа құрылғылар) бір-бірімен кез келген уақытта хабарласа алады. Ары қарай

«Желі» сөзін қолданғанда Интернет сөзіне синоним ретінде Желімен интернет арқылы екі компьютерді байланыстыру мүмкіндігін және олардың өзара байланысын қамтамасыз етуді түсінеміз. Интернетке қосылған әрбір компьютер – бұл Желінің бір бөлігі.

Компьютерлер-серверлер және компьютерлер-клиенттер.

Интернетке қосылған барлық компьютерлерді екі типке бөледі, олар

:серверлер және клиенттер. Бір компьютерде серверді де және клиентті де орнату мағынасында бөлу онша қатаң жүргізілмейді. Жергілікті компьютерде Web-сервер орнатылуы мүмкін және осыған қарамастан, дәл осы компьютерде браузермен және почталық клиентпен де жұмыс жасауға болады.

Басқа компьютерлерге анықталған сервис ұсынатын компьютерлерді *серверлер* (ағыл., *to serve* – қызмет көрсету) деп атайды, ал осы сервисті қолданатындар - *клиенттер.*

Көп жағдайларда үйдегі клиенттік компьютерлерде әр уақытта Интернетке кіруге мүмкіндігі болмайды, сондықтан Желіге тек керек уақытта ғана қосылады. Керісінше, компьютер-серверлер мәліметтерді берудің жоғары жылдамдықты арналары арқылы Интернетпен байланысқан, сондықтан оларға сұраныс арқылы хабарласуға болады.

Қосымша-серверлер және қосымша-клиенттер.

Компьютерлерді серверлер мен клиенттер деп атағаннан гөрі, оларды бағдарламалық қамсыздандыру деңгейіндегі клиенттер немесе серверлердеп атаған жөн. Бір бағдарлама клиент есебінде, ал екіншісі сервер есебінде іске қосылатын қосымшалардың өзара байланысы *клиент-сервер архитектурасы* деп аталады.

Сервердің басты тапсырмасы – сервиске қайсыбір клиент сұранысжібермейінше әр кезде жұмыс жасап және күту жағдайында болу болып табылады.

Серверде сұраныстардың көптігінен оның жұмысы баяулап және белгілі бір сұраныстарға қызмет көрсетуді тежейді. Серверге сұраныс белгілі бір протокол шегінде болады – бұл Желіде компьютерлер арасында байланысты қамтамасыз ететін стандарттар жиыны. Серверлік бағдарламалар клиенттік бағдарламаларға қызмет көрсету үшін компьютердің аппаратты ресурстарын қолданады. Клиент-бағдарлама сұраныс құрып, оны желі арқылы белгілі бір адреске жібереді және алдын ала белгіленген протокол арқылы сервер-бағдарламамен өзара байланысады. Сол бір компьютерде бірнеше серверлік бағдарламалар

орналаса алады. Клиенттік қосымша серверлік қосымша орналасқан компьютерде де, сонымен қатар, серверден керегінше жойылған компьютерде де орналаса алады, бірақ олар Желімен байланысса, бұл айырмашылық тек уақыт бойынша жауаптың кідіруіне сәйкестеледі.
Әрбір сервер-бағдарламаның типі үшін өзіндік клиент-бағдарламасы бар. Осылай, Web-клиент Web-серверге, почталық клиент – почталық серверге хабар береді және т.б. Серверлік бағдарлама әрқашан сұранысты орындауға дайын болу керек және сондықтан да сервер-бағдарлама жұмыс жасайтын компьютерлерге сенімділікке және өнімділігіне байланысты жоғары шарттар қойылады. Клиенттік компьютердің жұмысының тұрақтылығы бір адамның жұмысына әсер ететіндіктен, олардың жұмысынасенімділігіне байланысты аз талаптар қойылады, ал аппаратты сервердің жұмысының сенімділігіне байланысты көптеген клиенттердің жұмысының жүргізілуі тәуелді болады. Жоғарыда көрсетілген тәсіл (клиент-серверлік архитектура) дербес компьютердің қолданушысына өзінің жұмыс үстелінен Интернетке қосылған миллиондаған серверлердің ресурстарына қол жеткізуге мүмкіндік береді.
Провайдерлер және олардың желілері.
Интернетке қарап, біз интернет-провайдерлердің қызметтерін пайдаланамыз және ISP(Internet Service Provider – Интернет қызметін жеткізуші). Көбіне ISP – бұл өзіндік желісі бар арнайы ұйым (магистральдыдеп аталады), оған клиенттердің көптеген саны қосылады. Провайдердің желісі ғаламның кез келген нүктесімен байланысуды қамтамасыз ететін жержүзінің басқа да желілерімен байланысуы мүмкін. Қалыпты жағдайда ISP- провайдерлер – бұл белгілі бір аймақтарда өзіндік орналасу нүктесі (POP - Point of Presence) бар ірі компаниялар, бұл нүктелерде клиенттерінің Интернетке қосылуын қамтамасыз етуге арналған провайдердің аппаратты қамсыздығы. Ірі провайдердің әртүрлі қалаларда өзінің орналасу нүктесі мен мыңдаған клиенттері болады. Бірнеше қалаларда орналасу нүктелерібар провайдерлермен қатар, бір қалада орналасу нүктесі бар провайдерлердіде атап көрсетуге болады. Телефон линиясы арқылы ISP мен байланысуды ұйымдастыру: ДК қолданушысы драйверге хабарласады және модем жинақтарының ішіндегі провайдер модемдерінің бірімен байланыс орнатады (модемді пул деп атауға болады). Қолданушы өзінің ISP не қосылғаннан кейін, ол оның желісінің бір бөлігі болып табылады. Провайдер өзінің серверінде клиенттерге әртүрлі қызмет көрсете алады: электрондық почта (e-mail), желілер жаңалықтары (Usenet) және т. б. Провайдердің магистральды желісін көбіне тіректі желі немесе бэкбоун деп атайды (ағыл. Backbone-қырат). Провайдер желілері көптеген клиенттерге қызмет көрсететіндіктен, оның жоғары жылдамдықты желісі болуы және жоғары трафикті қамтамасыз етуі керек (желі бойымен берілетін мәліметтер көлемі). Өзінің барлық орналасу нүктелерін біріктіру үшін, провайдер ірі коммуникациялық компаниялардан жоғары жылдамдықты арналарды жалғаала алады, сонымен қатар, өзінің арналарын тарта алады. Ірі коммуникациялық компаниялардың өздерінің жоғары жылдамдықтыканалдары бар.
Провайдерлердің желілерін біріктіру.
Кейбір провайдерлердің клиенттері, мысалы, ISP-A бір біріменөздерінің жеке желілері арқылы өзара байланысады, ал басқа ISP-B компаниясының клиенттері өздерінің, бірақ егер ISP-A және ISP-B желілерінің арасында байланыс болмаса, онда A компаниясының клиенттері және B компаниясының клиенттері бір бірімен байланыса алмайды. Өздерінің клиенттерін бір желіде біріктіру мақсатында A және B әр қалада желілік кіруді (NAP - Network Access Points) қамтамасыз ететін нүктелер арқылы өз араларында тікелей байланысты орнатады. Осылайша, басқа провайдерлердің магистральды желілеріне қосылуы құрылады, нәтижесінде жоғары деңгейлі көптеген желілердің бірігуі болады.
Интернетте жүздеген ірі интернет-провайдерлер орналасады және олардың магистральды желілері NAP арқылы әртүрлі қалаларда жасалады, және мәліметтердің үлкен ағыны NAP-түйіннің әртүрлі желілері арқылы таралады.
Үлкен және кіші желілердің бірігуі (Интернетті құрайтын) негізінде шартты келісімдер жатады. Әрбір клиенттің белгілі бір ISP пен өзінің компьютерін немесе жергілікті желісін провайдер желісіне қосу туралы келісім шарты бар. Кейбір ISP-A провайдерлердің клиенттері ISP-Aжелісіне қосылу туралы келісім құрайды, өз кезегінде ISP-A ISP-B менжелілерін біріктіру туралы келіседі және солай жалғаса береді.
Провайдерлер желілерінің иерархиясы.
Әртүрлі елдерде халықаралық, ұлттық және аймақтық болып бөлінетін жүздеген провайдерлер бар.

Аймақтық провайдерлердің желілері (екіншілік) ұлттық провайдерлердің желілерімен (біріншілік) жоғары жылдамдықты каналдар арқылы байланысады. Мысалы, АҚШ-та Т1 стандартты мәліметтерді беру жылдамдығы 1,544 Мбит/с арна немесе жылдамдығы 44,74 Мбит/с жететін Т3 арнасы.

Осыдан он жылдай уақыт бұрын Microsoft корпорациясы веб-стандарттарды құруды қолға алды. Бұл веб-стандарттар қорыта келе, барлық веб-бағдарламалау бойынша берілетін технологияларды бір бүтін етіп жинақтауға және көшірілетін веб-платформаны құруға бағытталған.

Бақылау сұрақтары:

1) Клиент-серверлік архитектураны құрайтын негізгі желілерқандай?
2) Желіні бағдарламалық қамсыздандыру қандай принциппенжүргізіледі?
3) Провайдердің магистральды желі термині нені білдіреді?
4) Провайдерлер желілерінің бірігуі қалай орындалады?
5) Интернетті түзетін желілер иерархиясы қалай аталады?

Дәріс №2.

Интернет коммуникациясының моделі. Коммуникацияның Pull- және Push-моделі. Интернет сервисінің коммуникациялық сипаттамалары

Дәрістің мақсаты: интернет коммутациясының модельдерін, интернет сервисінің коммуникациялық сипаттамаларын таныстыру.
Дәрістің мазмұны: интернет коммуникациясының Pull- және Push- модельдері. Интерактивтілік. Гипермәтін. Мультимедиа. Болу эффектісі. Желілік навигация. Internet сервисінің коммуникативті сипаттамасы.

Интернет коммуникациялық сипаттамаларын қарастырайық. Дәстүрлі коммуникациялық модель негізінде «бірге бір» ережесі жүреді. Коммуникациялық құрылымға байланысты ақпарат статикалық (мәтін, графика) және/немесе динамикалық (аудио, бейне, анимация) күйінде көрсетіледі.
Бұл модельден ерекше Интернет негізінде екі басқа принциптер жатыр. Біріншіден, Байланысқа өз үлесін қосатын Интернетке коммуникация кезінде оның арнасы арқылы өтеді. Бұл модель бастапқы байланыс жіберуші мен қабылдаушы арасында емес, негізінде қолданушы мен коммуникациялық ортадағы бір кеңістік арасында болатынын көрсетеді, сонымен байланыстың екі қатысушысы да жіберуші де, қабылдаушы да болып табылады.
Интернет өзімен әрбір желі абоненті басқа абоненттерге немесе топтармен топ атымен немесе өз атымен хабарласа алтын «көпке көп» көп бағыттылы коммуникациялық модельді көрсетеді. Бұл модельде коммуникация құралы болып таратылған компьютерлік желі айтылады, ал ақпарат гипермедиялық түрде көрсетілуі мүмкін. Бұл модель интерактивті байланыс Интернеттің басқа қолданушыларымен қатар ортаның өзімен де бола алатынын, тіпті соңғы байланыстың соңғы түрі басымдылық көрсететінін білдіреді. Осындай байланыстың бар болуынан ақпарат жіберуші құралы кейде оның қолданушысы да бола алады. Бұндай модельде ақпарат және мазмұн жіберушіден қабылдаушыға беріліп қана қоймай, ортаның өзі де оның қатысушыларымен де құрылады.
Интернет ортасының моделі ашық көрсетілген кері байланыстарды көрсеткенде, жалпы коммуникацияның дәстүрлі әдістері үшін коммуникациялық модельде кері байланыс контуры болмайды. Тұтынушымен кері байланысты жүзеге асыруға мысал ретінде электронды почтаны, қолданушылар тіркеу туралы мәліметтер, «cookie» файлдары, Web-серверлерде жазылу немесе тіркелу. Кері байланыстың болуы өзара байланыстың коммуникациялық әдістерін қолдану тиімділігін және ішкіжәне сыртқы ортада болатын оқиғаны қалыпты қабылдауды жоғарылатады.
Интернет коммуникациясының Pull- және Push-модельдері.
Жалпыға бірдей қызмет көрсетудің дәстүрлі әдістері тұтынушыларға ақпаратты жеткізуші push-модельді жүзеге асырады, бұнда тұтынушылар пассивті роль атқарады және ақпараттың каналдарын таңдаудың шектеулі мүмкіндіктерінің болуы.
Ақпаратты жеткізудің дәстүрлі push-моделіне қарсы Интернет негізінде, ақпаратты сұраныс бойынша жеткізетін (demand pull) pull-моделі жатыр. Бұл Интернет ортасының ерекшелігі ақпаратты іздеуде бақылаумен негізделген және іздеу механизмін қолдануда навигация бейнесі немесе басқа да URL (uniform resource locator) бастауларына байланысты тұтынушылардың тікелей қатысуымен байланысты. Интернетте сонымен қатар push-моделін іске асыру мүмкіндігі бар. Интернет бұл модельмен push-технологиясы арқылы жүре алады, соған байланысты қолданушыларға Интернетте ақпаратты іздеп жатудың қажеті жоқ, ол тек керекті тақырып бойынша қажетті каналға жазылса болғаны, кейін ақпарат канал жазылушыларының компьютерлеріне автоматты түрде жеткізіліп тұрады.
Интернет ортасының технологиялары push- және pull-модельдерінің тығыз байланысы бағытында дамиды. Интернеттің жоғары функционалдылығына байланысты керекті ақпаратты немесе басқа да қорларды табу мақсатында қолданушының навигация мүмкіндіктері әрқашан да болады. Басқа жағынан, қолданушылардың әрқашан ақпаратты жеткізушіні және қабылданатын push-каналдар тематикасын таңдауға мүмкіндіктері бар.
Интерактивтілік.
Internet ортасының басты сипаттамасы оның интерактивтілігі болып табылады. Интерактивтілік – бұл коммуникациялық хабарламалардың бір біріне қатысты орналасуын білдіретін,

коммуникация процесінің жүзеге асуын көрсететін сипаттама. Интерактивтік өзара байланыс үшін ақпараттың немесе хабарламаның келуіне жауапты іс-әрекеттің болуы тән, сонымен қатар жауап алдындағы хабарламаның контекстінде болуы қажет.

Жоғарыда айтылғандай, Интернет ортасы үшін интерактивтілікті қолданушыға «жауап қайтару» мүмкіншілігі есебінде түсінуге болады. Осылайша, Интерактивтілік компьютердің қолданушының қимылын бағалап және осы бағаға байланысты жауап бере алатын мүмкіндігі бар, диалогқа қатысушы есебінде функциялық көрінісін кеңейтеді және толықтырады. Компьютерлік гипероратны қолдану арқылы өзара байланысу процесінде клиентпен хабарласу қатынасты орнатудың ең бастапқы кезеңінде сол компьютерлік гипероратмен, клиенттің алдыңғы іс-әрекетке қалай қарағандығына байланысты жасалуы мүмкін. Internet негізінде жатқан құрамдық модельді қайта қарау барысында, Internet ортасындағы интерактивтілік енді орта арқылы жеке қатынас деңгейінде жүзеге аспай, нақты ортаның өзімен өзара байланысу деңгейінде жасалатынын көруге болады. Өзара байланыстың интерактивті сипаты коммуникация мүшелерінің өзара байланысу әсерін жоғарылатуға мүмкіндік береді. Бұл, мысалы тәжірибе жүзінде, келісім немесе сатып алу үшін қажетті ақпаратты алу үшін кететін уақытты қысқартуға керек. Сонымен қатар, ортаның интерактивті сипаты жеке клиенттің ерешеліктеріне қатысты байланыстың қалыптасу мүмкіндігін ұсынады.

Гипермәтін. Internet ортасының гипермәтіндік табиғатының ерекшелігі аз емес. Гипермәтінге негізделген дүние жүзіндегі ең алғашқы жүйені бірнеше жылдан астам уақыт бұрын ЭЕМ-нің алғаш құрушыларының бірі (Ванневар Буш) ұсынған. Бұл жүйе Метех деп аталады, ол қолданушының жеке кітаптарын, жазбалар мен коммуникацияларын сақтайтын құрылғы, оған жүктелу тез және оңай. 1967 жылы Нельсон (Nelson) гипертексті жүйелер көрсеткіштері мен ассоцация ретінде бейнелеп, адам ойындағы жеке мәліметтер бөліктерін ұйымдастыру және байланыстыру мүмкіндіктеріне теңеді. 1993 жылы Бомман (Bomman) гипертекстке мынандай анықтама берді: «Гипертекст концепцияның ретсіз жазба мәліметтерін құрайды, осыған орай қолданушы мәліметтер бөліктерін бір-бірімен көрсеткіштер мен байланыстар көмегімен байланыстырады. Гипертекстік жүйеде мәліметтүйін және байланыс түрінде беріледі».

Гипертекст ұйымдастыру құралының бөліктерге бөлінген сызықты емес желілер формасымен сипатталады. Мұнда әр бөлік (фрагмент) келесі бөлікке белгілі бір байланыс типі арқылы көшеді. Гипертекст мәліметтерді ғана емес, сонымен бірге қолайлы іздеу қондырғысын құрайды. Сонымен, гипертекстік көрініс мәліметтеріқарапайым әдіске қарағанда, мәліметтерді қорытады.

Мультимедиа. Мультимедияның пайда болуына байланысты динамикалық (аудио, видео, анимация) және статикалық (текст, графикалар, суреттер) құрамыбар кмпьютерлік интеграция мүмкіндігі туды. Түйіндер мен байланыстар комбинациясы гипертекстік жүйе мультимедиасында жаңа ортаны құруға мүмкіндік берді.

Гипермедиа – бұл мәліметтер көрінісі және оған кіру әдісі. Оның концепциясы граф түрінде берілген мәліметтер кеңістігінің моделінде орналасады, түйіндерінде мәліметтер, ал семантикалық байланыс граф доғасында қамтылады. Ақиқатты гипермедия жүйесіндегі сақталатын мәлімет осы заманғы компьютер шығара алатын барлық мүмкін формалармен көрсетіледі. Сонымен қатар, гипермедиа өзінде радио (аудио),теледидар (динамикалық бейне), пресса (текст, суреттер, фотосуреттер) және компьютер (видеотерминал) компьютерлік гипероратда құрамында жататын гипертекстік байланыстарды орнатады.

Болу эффектісі. Интернеттің коммуникациялық моделінің басты мәселесі «болу эффектісі» болып табылады, қоршаған ортамен клиенттің араласу процесі: жағдай, бөлме, компьютерлік жұмыс орын және т.б., компьютерлік гиперәдіспен құралатын орта құрамына кіреді. Дәреже қатынасы мен клиентке әсер ету «болу эффектінің» дәрежесін анықтайды. Айта кеткен жөн, «болу эффектісі» өзінің табиғатына сай, интерактивтік араласу мен компьютерлік гипероратмен тығыз байланысты.

Желілік навигация. Желілік навигация компьютерлік гипероратда өзіндік қозғалыс процесі болып табылады. Сызықты емес іздеуді және мәліметтерді талап ету құрамындағы әдісті клиентке еркін түрде береді.

Internet сервисінің коммуникативті сипаттамасы. Интернет комплексті сервис болғандықтан, оның жеке сипаттамаларын таңдап және қолдану кезінде көрсету керек. Кестеде коммуникация түріндегі Internet коммуникациялық сервис анализінің нәтижесі, коммуникация моделінің құрамында жатқан, көрсетілген мәліметтер түріне, симметриялық кері байланыс және

интерактивті қарым- қатынасы көрсетілген.

2.1 кесте - Internet сервисінің коммуникативті сипаттамасы

Сервис	Аралық қарым-қатынас	Машиналық қарым-қатынас	Коммуникация Моделі	Мәліметтер түрінің көрінісі	Кері байланыс симметриясы	Интерактивті қарым-қатынас мүмкіндігі
Электрондық пошта	Иә	Иә	Бірге-бір Бірден – көпке	Мәтін, графика, дыбыс	Иә	Жоқ
Жіберу тізбегі	Иә	Иә	Көпке-көп	Мәтін	Иә	Жоқ
Конференциялар	Иә	Иә	Көпке-көп	Мәтін	Иә	Жоқ
Сөйлесу бөлімдері	Иә	Иә	Көпке-көп	Мәтін	Иә	Иә
WWW	Иә	Иә	Көпке-көп	Мәтін, графика, дыбыс, видео	Жоқ	Иә
Дауыс байланысының программалары	Иә	Иә	Бірге-бір	Дыбыс	Иә	Иә
Аудио- және видео - конференциялары	Иә	Жоқ	Көпке-көп	Дыбыс, бейне	Иә	Иә

Көрсетілген мәліметтер Интернет ортасының қолдану әдістерінің коммуникациялық кең диапазонды түрлері көрсетілген. Нақты есептердішығару кезінде әрбір Интернет сервисінің жеке анализдері қажет.

Бақылау сұрақтары:

1) Қандай коммуникативті модельдер Интернетте шығарылады?
2) Гипертекст ұғымы нені білдіреді?
3) Internet ортасындағы интерактивтік ұғымы нені білдіреді?
4) Pull- және Push-моделдерінің Internet коммуникациясының ерекшеліктері қандай?
5) «Болу эффектісі» термині Internet ортасында нені білдіреді?

ДӘРІС №3. ҚОЛДАНУШЫНЫҢ ИНТЕРНЕТКЕ КІРУІ. «СОҢҒЫ МИЛЯ». КОММУТАЦИЯ КАНАЛЫНЫҢ ЖЕЛІСІ. МОДЕМ

Дәрістің мақсаты: қолданушының интернетті пайдалану кезіндегі «соңғы миля», «коммутация каналының желісі» және «модем» түсініктерімен, интернетке кірудің түрлі нұсқаларымен таныстыру.

Дәрісті мазмұны: «Соңғы миля» технологиясы. Желінің коммутациялы каналдары. Модем функциясы. ASCII-кодировкасы.

Бірнеше компьютерлер арасында жобалау және есептерді шығаруды жеңілдету үшін ол жүйені бірнеше бір-бірімен байланысқан топтарға бөлінеді. Жүйе арқылы алмасу жүріп жатқанда екі торап келесі ережелерді сақтауы тиіс:

- электрлік сигналдардың түрін және деңгейлерін келістіру;
- берілістердің ұзындығын анықтау әдістері;
- бақылау әдістері.

Келісімдер мәліметтерді интерпретациясын анықтайтын, биттарды беру ең кіші деңгейден үлкен деңгейлерге дейін барлық деңгейлерге бірдей болу керек.

Бір деңгейдегі берілістердің форматын және тізбегін анықтайтын ондай формалданған ережелерді протоколдар деп атайды.

Иерархиялық жүзеге асырылған протоколдардың жиынтығын коммуникациялық протоколдардың стекі деп атайды.

Желілік деңгей біріншіден келесі есептерді шығару үшін құралдарменқамтамасыз ету керек:

- еркін топологиясы бар желілерге пакеттерді жеткізу;
- желіні сенімді локалды трафиктың арқасында құрылымдау;
- әртүрлі протоколдардың каналдық деңгейледің келісуі.

Желідегі трафик кездейсоқ ретімен орналасады, бірақ онда кейбір заңдылықтар көрсетіледі. Кейбір жалпы есеппен жұмыс істейтін қолданушылар (мысалы, бір бөлімнің қызметкерлері) көбінесе бір-біріне немесе жалпы серверге сұраумен қатынасады. Сол кезде олардың басқа бөлімдердің компьютерлерінің ресурстарымен қолдануы қажет болады. Сондықтан жүйенің құрылымы мәліметтердің ағынының құрылымына сәйкес келу керек. Жүйедегі трафигіне байланысты компьютерлер бірнеше бөлімдерге (сегменттерге) бөлінуі мүмкін. Егер бір-бірімен қатынасатынкомпьютерлердің сұраулары мен жауаптары осы топтың компьютерлеріне арналса, онда олар бір бөлімге жинақталады.

Жүйені сегменттерге бөлу үшін мосттар мен коммутаторлар қолданылады. Олар сегменттің ішіндегі локалды трафикті сыртқа ешқандай кадрларды бермей экрандайды, тек қана басқа сегменттерде орналасқан компьютерлерге арналған кадрлардан басқа. Сонда жүйе бірнеше жүйелерге бөлінеді. Ол бір топтың ішіндегі трафиктің жиілігін және топтардың арасындағы алмасудың активтігін ескере отырып байланыс бөлімдерінің өткізу мүмкіндігін рационалды түрде таңдау мүмкіндігін тудырады.

Бір жағынан жүйенің мосттардың арасында орналасқан логикалықсегменттері бір-бірінен дұрыс оқшауланбаған, өйткені олар кеңдік толқындарынан қорғалмаған. Егер қандай да бір станция кеңдіктолқындары арқылы мәліметтерді жіберсе, онда ол мәліметтер жүйенің станцияларының барлық логикалық сегменттеріне беріледі. Мосттардың көмегімен салынған кеңдік толқындарынан қорғану үшін: администратор тек бір түйінге генерирлеуге рұқсат етілген кеңдік толқындарының пакетін шектейді.

Екінші жағынан коммутаторда жасалған локалды жүйелердің виртуалды сегменттердің механизмдерін пайдалану трафиктың толық локалдауына әкеліп соғады. Ондай сегменттер бір-бірінен толық оқшауланған. Сонда мосттар мен коммутаторлардың көмегімен салынған жүйелердегі әртүрлі виртуалды сегменттерге жататын компьютерлер біркелкі жүйені құрамайды. Мосттармен коммутаторлардың келтірілген кемшіліктері олардың каналдық деңгейлердің протоколдарымен жұмыс істеуіне байланысты. Ол протоколдарда үлкен жүйені құрылымдау кезінде пайдалануға болатын жүйенің бөлігі немесе сегментті анықталмайды. Жүйелік технологияларды жобалаушылар каналдық деңгейлерді жетілдірудің орнына ол есепті жүйелік деңгейлерге жүктеді.

Қолданушылардың көп бөлігінде жоғарғы жылдамдықпен желіге кірумүмкіндіктері провайдерге қарағанда жоқ. Абонентпен провайдердің болу нүктесі арасындағы болатын байланыс технологиясы *технологияның соңғы милясы деп аталады*. Бұл атау шартты түрде, ал практика жүзінде осы айтылған қашықтық бір миляға сәйкес келу қажет емес. Көп жағдайларда «соңғы миля» желіден компьютер қолданушысына ақпаратты жіберу тар жол болады. Интернетке жіберу соңғы миля технологиясы әртүрлі бола алады, бірақ «кіру жылдамдығы жоғары болған сайын, каналдарды қолдану ақысы жоғары болады» деген принцип әмбебапты болып келеді. Барлық инфраструктураны қолдануға мүмкіндік беретін көптеген технологиялар бар, олар - Интернетке кіру мүмкіндігіне арналған телефонды линиялар, кабельді теледидар желілері және басқалар.

Интернетке кіру әртүрлі технологиялары: телефондық линия арқылы модеммен қосылу, радиоканал арқылы қосылу, кабельді теледидардың желісімен қосылу және де спутникті канал көмегімен қосылу. Интернетке қосылудың әртүрлі әдістеріне қарамастан, қолданушы үшін тек қосылу тұрақтылығы, мәліметтерді жіберу жылдамдығы және жауап беру уақытында ерекшеліктер бар. Интернеттің барлық ресурстарына кіруге мүмкіншілік бар. Желіге қосылудың әртүрлі нұсқаларын қарастырайық.

Интернетке кірудің әртүрлі нұсқалары. Коммутаторлық телефон линиясы арқылы қосылу. Коммутативті телефонды линияның модем арқылы қосылуы – бұл әзірше үйдегі Интернет қолданушылардың ТМД-ға таралған қосылу әдісі.

Желінің коммутациялы каналдарымен. Коммутациялы телефонды желінің немесе желінің коммутациялы каналдарымен жұмыс принципін айтайық. Сіз біреуге қоңырау шалған кезде, сіздің аралығыңызда белгілі бір физикалық байланыс каналы орналасады, олар коммутатор арқылы байланысатын жеке учаскелерден тұрады. Абоненттің әрбір учаскесі бірінші коммутаторға дейін бөлінбейтін болып келеді. Яғни, әрбір абонент осы учаскелерді жеке өзі қолданады. Желілердің қалған учаскелері коммутатор арасында ажыратылған болып келеді, дәлірек айтқанда әртүрлі уақытта әртүрлі қолданушылармен қолданылады. Бірақ, егер қайсыбір участок бір қолданушымен бос емес болса, онда келесі біреу оны осы уақытта қолдана алмайды. Коммутативті телефонды линиямен қосылу - бұлуақытша (сеансты) қосылу дейді. Телефонды линия телефонды әңгімемен немесе мәліметтерді модем арқылы жіберумен бос болмауы мүмкін.

«Модем» сөзі «МОДулятор» және «ДЕМодулятор» деген сөздерден құралады. Модем телефонды линия арқылы мәліметтерді жіберуде қолданылады. Жалпы жоспарда *модем* - бұл қондырғы, цифрлық кодтыәртүрлі екі жиілік дыбыстардың ауысу мен кері қайту – дыбыс тербелісін цифрлық ақпаратқа ауыстырады. Екі модем қосылғаннан кейін модулятор тұрақты тербеліс периодының басты сигналымен генерацияланады және болып жатқанды өзгертіп екінші сигналды қосады. Сонымен, жүріп жатқанның бір параметрі жіберу сигналының өзгеруіне байланысты. Демодулятор сараптама жасайды, түсетін сигнал жүріп жатқан сигналдан қалай ерекшеленеді де басты сигналды қалыптастырады. Басқаша айтқанда, жіберуші модем болып жатқан жиілікті пайдалы сигналмен моделдейді және жоғары жиіліктегі сигналды жібереді, ал модем-адресат сигналды ASCII кодына (American Standard Code for Information Interchange – ақпараттармен алмасудың стандартты коды – машиналық көріністегі латын алфавитінің әріптері, сандар мен басқа символдардан тұратын 128 код символ топтамасы, олардың әрбіреуіне 7-биттік екілік сан. Сегізінші бит жіберу кодының дұрыс контроліне жұмыс жасайды) сәйкес кері цифрлы формаға демодулдейді.

Бірінші модемдер Bell Laboratories зертханалық центрінде өңделген және Bell 103 атауын алды. Бұл қондырғылар екі сыңарлы жиілікті

қолданады: сыңарлы жиілік бір модемге және сыңарлы жиілік екінші модемге. Жіберуші модем 1,07-1,27 кГц арасындағы жиіліктерге ауысу арқылы, ал жауап беруші модем 2,025-2,225 кГц арасындағы жиілікте мәліметтерді жібереді.

Bell 103 модемдері 300 бит/с жылдамдықпен жұмыс істейді, солай минутына 30-ға жуық символ жіберуге болады. Мұндай жылдамдық тек қана мәтіндік хаттамалармен алмасқан кезде қолайлы, себебі бұл жылдамдық адамның оқу жылдамдығымен салыстырғанда әлдеқайда артық. Мәліметтерді 300 бит/с жіберетін модемдер 1980 жылға дейін өмір сүрді. Сөйтсе де, адамдар суреттерді жібере бастағалы ондай жылдамдық аздау екені айқындалды.

Осы заманғы модемдер – бұл өте күрделі схемалы модуляциялардықолданып, мәліметтерді жіберер кезде қаптайтын күрделі құрылғылар болып келеді. Клиенттік компьютер коммуникациондық портқа ноль менбірдің реттелген түрге келтіріп, әртүрлі командалар мен мәліметтердіжібереді. Модем мәліметтерді қабылдап, оларды командалар және ақпаратқа ажыратады да, телефон линиясы арқылы жіберіп жүзеге асырады.Қолданушы модем арқылы телефондық желіге ортақ қолданумен қосылып, ал ISP, басқа модеммен, цифрлық сигнал арқылы Интернетпен қосылады.

Бүгінгі күнгі модемдер екі түрлі: ішкі және сыртқы. Ішкі модемдер кеңелмелі плата арқылы жұмыс істесе, сыртқылары бөлек құрылғы ретінде автономды блок питаниямен істейді.

Модем (ағылш. *modem, modulator-demodulator*) — сандық сигналдарды аналогтық сигналдарға айналдыратын және керісінше амалжасайтын электрондық құрылғы.

Деректерді әртүрлі арқаулар арқылы, мысалы телефон сымы арқылы немесе радиосигналдар түрінде беруге болады. Телефон сымы арқылы аналогтық сигналдар деп аталатын импульстарды беруге болады. Аналогтық сигналдар шуыл немесе электрлік-магнитті импульстар түріндегі кедергілердің әсеріне ұшырауы мүмкін.

Дыбыстық сигналдар дерек берудің едәуір жаңа әдісі болып табылады, онда дерек беру және қабылдау үшін екілікті пішім қолданылады(нөлдер мен бірліктер тізбегімен жұмыс істейтін кодтау/кодты ағыту жүйесі). Дерек беру мен қабылдау үшін компьютерлерде дәл осы пішім қолданылады. Алайда сандық сигналдарды телефон желісі арқылы беру мүмкін емес. Сондықтан телефон сымы арқылы дыбыстық сигналдарды берер алдында оларды аналогтық сигналдарға айналдыру керек. Және тиісінше деректі қабылдаушы жақ алынған деректі сандық сигналдарға айналдыруға тиіс. Модем осындай айналдыруды жүзеге асырады – сандық пішімнен аналогтық пішімге және керісінше.

Басқа компьютерге хабар жіберу кезінде модем шығару құрылғысы рөлін атқарады. Ал сіздің компьютеріңізге хабар келген кезде, модем енгізу құрылғысы ретінде жұмыс істейді. Модем сыртқы немесе жүйелік блокқа кірістірілген болуы мүмкін. *Сыртқы модемдер* – бұл сым арқылы жүйелік блокқа қосылатын модемдер.

Модемдер сондай-ақ компьютердің жүйелік блогына кіріктірілген болуы мүмкін. Мұндай модемдер *ішкі модемдер* деп аталады

Бақылау сұрақтары:

1) Интернет технологиясында соңғы миля нені білдіреді?
2) Модем функциясы қандай?
3) ASCII-кодировкасы?
4) Модем жылдамдықтары қандай?

ДӘРІС №4. ИНТЕРНЕТ ЖҮЙЕСІНЕ ҚАТЫНАСУ ТЕХНОЛОГИЯЛАРЫ. DSL,ISDN ТЕХНОЛОГИЯЛАРЫ

Дәрістің мақсаты: Интернет жүйесіне қатынас жасаутехнологияларымен танысу.
Дәрістің мазмұны: DSL, ISDN технологиялары, Асимметриялы DSL(ADSL).

DSL-технологиялары.

Қарапайым телефон тек төменгі жиіліктегі желілерді қолданады. Егер «телефонды» жіңішке жолының орнына кеңірек жолды қолданса, телефон сымы едәуір көп ақпарат жібере алады. *Өткізу жолы* немесе *өткізу жолының ені* - өткізу каналы жиіліктерінің диапазонындағы ең жоғары жиілік пен ең төменгі жиіліктің арасындағы айырымы. Телефон желісі арқылы сигналды жіберудің аналогты түрі 300 Гц-тен 3,4 кГц-ке дейінгі диапазонды алады. Цифрлы сигнал үлкендеу диапазонды талап етеді. Жіберу жылдамдығы өскен сайын, жиіліктер диапазоны да өсу керек. Сондықтан, «өткізу жолы» терминімен компьютер желісімен мәліметтерді жіберу жылдамдығының жоғары шекарасы белгіленеді. «*Кеңжолды қатынас»* термині арна ақпарат жіберу үшін кеңейтілген жиіліктер жолын береді дегенді білдіреді. Ақпаратты жіберудің жоғары жылдамдығы кең жолды пайдаланған кезде бірнеше сигнал бір арнаның бойымен, бірақ әртүрлі жиіліктерде параллельді жіберілу мүмкіндігіне байланысты. Өйткені, уақыт бірлігінде ақпараттың мол көлемі жіберіледі. Арнаны бірнеше бөлікке бөліп, сигналдарды жіберу *мультиплексорлау* деп аталады. «Таржолды қатынас» терминімен дауысты жіберуге жеткілікті арна түсіндіріледі. Кейде «кеңжолды емес қатынас» термині кездеседі. Ол арнамен мәліметтерді жіберу жылдамдығы 64 Кбит/с-тан кем екендігін білдіреді. Телефон желісін цифрлы мәліметтерді жоғары жылдамдықта жіберу үшін қолдануға мүмкіндік беретін технологиялардың бірі DSL. DSL аббревиатурасы - «Digital Subscriber Line». Бұл технология телефонмен
сөйлесуге кедергі жасамай, мәліметтерді жіберуге жоғары жиіліктерді қолдануға мүмкіндік жасайды.

DSL-технологиясы көмегімен бір уақытта Интернетпен жұмыс істеу және телефонмен пайдалануға болады. Қарапайым модемге қарағанда, DSL-қосылу жылдамдығы едәуір жоғары. DSL үшін жаңадан сымдар жүргізу керек емес, өйткені телефон желісінің құрылғылары пайдаланылады.

Асимметриялы DSL (ADSL).

Интернетпен жұмыс кезінде ақпараттың негізгі ағыны Желіден пайдаланушыға өтеді. Желіге ақпараттың едәуір аз көлемі жіберіледі. Мысалы, сіз Web-беттерді қарағанкезде, кішігірім сұраныс жібересіз, ал желіден мәтін ғана емес, сонымен қатар суреттерді де қабылдайсыз. Яғни, ақпарат алмасу ассиметриялы болып келеді. Бұндай трафикті жіберу үшін ассиметриялы арна керек. ADSL (Asymmetrical Digital Subscriber Line) немесе ассиметриялы DSL пайдаланушыға мәліметтерді жоғары жылдамдықпен жіберуге мүмкіндік береді, сондай-ақ пайдаланушыдан желіге келген сигнал желіден пайдаланушыға келген сигналға қарағанда төменгі жиіліктерде жіберіледі. Ең жаңа технологияларды пайдалануға байланысты ADSL бойынша ақпаратты жіберу жылдамдығы басқа қатынас түрлерімен салыстырғанда өте жоғары: 8 Мбит/с абонентке бағытталғанда, 1 Мбит/с абоненттен желігебағытталғанда. Жоғары жылдамдық Web-сайттармен, мультимедиа- ақпаратпен ыңғайлы жұмыс істеуге, үлкен файлдарды жылдам тасымалдауға және интерактивті қосымшаларды толық пайдалануға мүмкіндік жасайды.

ADSL артықшылығы болып орнатылуының жеңілдігі табылады: пайдаланушы мен телефон компаниясын бірімен-бірін қосатын бұрыннан бар телефон кабелінің сымдары қолданылады. ADSL Интернетке үнемі қатынасты қамтамасыз еткеннің өзінде ADSL пайдаланушылары өткізу жолын басқа абоненттермен бөліспейді. Бірақ ADSL кемшіліктері де бар, ең алдыңғысы – алыстықтың шектелуі. ADSL-технологиясын қолданған кезде кері бағытта ақпаратты жіберу жылдамдығы ара қашықтыққа байланысты болады. Егер абонентке бағытталған ақпаратты 3 км қашықтықта 8 Мбит/с жылдамдықпен алуға болса, 5 км қашықтықта – тек 1,5 Мбит/с.

Пайдаланушы ДК-і ADSL-модемге қосылады. ADSL-модемі жұмысының принципі 24 кГц-тен 1100 кГц-ке дейінгі жиіліктер диапазоны әрқайсысына виртуалды модем бекітілетін 4000-герцтік жолдарға бөлінетіндігіне негізделген. Соған байланысты виртуалды модемдердің әрқайсысы өз жиіліктер диапазонымен жұмыс істейді.

ADSL – модемі жиілікті айырымға қосылады. Жиілікті айырым (немесе сплиттер) кәдімгі телефон

байланысының төменгі жиілікті сигналы мен жоғарыжиілікті ADSL-сигналын айыратын төменгі жиіліктердің фильтрі болып табылады. Жиілікті айырым үш ұяшыққа ие блок түрінде жұмыс істейді: біреуі - ADSL-модемге қосылуы үшін, екіншісі – телефон
құрылғыларына қосылуы үшін, үшіншісі – ADSL желісіне қосылу үшін. Жиілікті айырым бір желіге телефонды да, компьютерді де қосуға мүмкінік береді. Телефон станциясындағы дәл сондай жиілікті айырым абоненттік сымның басқа жағында жоғары және төменгіжиілікті сигналдарды бөледі. Дыбысты сигнал телефонды желіге, ал цифрлы сигнал DSLAM (Digital Subscriber Line Access Multiplexer) қатынас мультиплексорына бағытталады. Бұдан провайдер желісі арқылы Интернетке өтеді.
Қатынас мультиплексоры – барлық DSL-абоненттерін біржоғарыжылдамдықты желіге қосатын, телефон компаниясының ғимаратында орнатылған механизм.
ADSL – экономды технология. Қарапайым жағдайда аналогты өткізу қабілеті бар бөлінген арнаға қарағанда, желі пайдаланушыға арзан түседі.
ISDN-*технологиясы.*
Бөлінген телефонды желі – екі абонентті үнемі қосатын телефонды байланыс желісі. Бөлінген желінің едәуір таралған технологиясы болып ISDN (Integrated Services Digital Network) табылады. *ISDN* – ақпаратты жіберудің цифрлы түрінің стандарты. *ISDN*-желісінің негізгі компоненті - 64 Кбит/с өткізу қабілеттілігі бар бір бағытталған bearer-арнасы немесе B- арна. Бұл арнамен цифрлы ақпарат жіберіле алады. Өткізу жолын кеңейту үшін B-арналар екі-екіден топтасады да, арналар тобына ақпарат алмасуды басқаратын D-арнасы (16 Кбит/с) қосылады. Ақпарат алмасу кәдімгі мыс сымның көмегімен атқарылады. Модемнің орнына ISDN-адаптерді орнатқан пайдаланушылар 128 Кбит/с дейін баратын жылдамдықтағы Интернетке қол жеткізе алады.
ISDN-адаптерлерін кейде ISDN-модемдері деп атайды. Бұл терминді дұрыс деуге болмайды, өйткені ISDN-адаптерлер модуляция және демодуляция функцияларын атқармайды, ISDN-желісі алдынан-ақ цифрлы болып табылған. ISDN-арнасын телефон компаниясы береді. ISDN- желісімен телефонмен сөйлесу және сол уақытта Интернетке ақпарат жіберуге болады.
Төменде әртүрлі технологиялар бойынша Интернетке қатынасудың максималды жылдамдықтары келтірілген.

Қатынасу технологиясы	Максималды жылдамдық
Коммутирленетін телефондық линия	56 Кбит/с
ISDN	128 Кбит/с
Кабельді телевидение	36 Мбит/с
ADSL 2	1 Мбит/с (запрос), 10 Мбит/с (жауап)
2,4 ГГц диапзонындағы радиоарна	До 54 Мбит/с
Серікті қатынас	Жүйеге байланысты бірнеше Мбит/с
T1	1,544 Мбит/с
T3	44,74 Мбит/с
GSM-data	9,6 Кбит/с
GPRS	160 Кбит/с (теоретикалық максимум)
UMTS	2,048 Мбит/с

Бақылау сұрақтары:
1) DSL – технологиясымен ақпаратты жіберудің ерекшелігі неде?
2) ISDN– технологиясымен ақпаратты жіберудің ерекшелігі неде?
3) ADSL–технологиясымен ақпаратты жіберудің ерекшелігі
4) ISDN және DSL технологияларын қолдану үшін қандай құрылғы қажет?

Дәріс №5. Интернеттегі web серверлер. Web – сервер жұмысының механизмі. Статистикалық және динамикалық беттер. CGI технологиясы. Скриптер

Дәрістің мақсаты: интернеттегі web-серверлер жұмысының механизмімен, функцияларымен, статистикалық және динамикалық web-беттермен, CGI технологиясымен танысу.
Дәрістің мазмұны: статистикалық және динамикалық беттер. CGI- технологиясы. Скрипт программа.

Web-сервердің функциялары.

Web-сервердің басты тапсырмасы қолданушылардан HTTP-сұраныс қабылдау, сұраныстарды өңдеу және оларды сандық компьютерлік кодқа айналдыру. Содан кейін, HTTP-жауапты миллиондаған нөлдер мен бірлерді кескінге, медиа-ағынға, әріптерге, HTML парақтарға айналдырады.

Кез келген веб сервер қолданушыға ыңғайлы болатындай веб браузеріболғаны дұрыс. Ол веб серверге интернет ресурстардың URL-адресқа өзгертілген сұраныстарды жібереді.

Стандартты функциялармен қатар, кейбір веб серверлердің қосымшалары болады. Мысалға алатын болсақ, сәйкес келетін программалық қамтамалар ресурсқа қолданушының қатынас жасауынбақылап отырып, оларды жеке журналға жазуы мүмкін. Көп жағдайда веб сервер мэйл сервермен бірге орнатылады. Бұл қолданушыларға сайттыңөзінен тікелей, бір ғана гиперсілтемені баса отырып поштаның бетіне тез арада өтуге мүмкіндік береді.

Веб сервер не үшін қажет?

Жоғарыда атап өткендей, веб сервер ақпарат алмасу үшін Интернет желісін қолданатын барлық қолданушылар үшін қажет. Дегенмен, көп мөлшерде оған қатынас жасайтын адам емес, программалар мен құрылғылар. Мысалы, антивирустық программалар серверден желіден іздеуді және өзінің мәліметтер базасын жаңартуды сұрайды. Веб серверлермен белсенді жұмыс жасайтын ұялы телефондар, теледидарлар, Интернетке қатынас жасай алатын басқа да құрылғылар. Программаларды автоматты жаңартуға баптауға болады. Бұл жағдайда қолданушыныңқажеттілігі болмайды. Кейбір жаңартылатын мәліметтер базасын жеке түрде қадағалап отырғаны дұрыс.

Ең танымал веб серверлер.

Атақты және кең таралған веб серверлар болып Microsoft IIS және Apache табылады. Осылардың арасында ең танымалы Apache. Оның программалық қамтамасы барлық бар болатындардың ішінен 60% қолданылады. Көп жағдайда бұл мүмкіндік Apache серверінің тегін болуына байланысты. Өндірушілер күнделікті өздерінің өніміндегі қателіктерін ретке келтіреді. Бұл программаның кемшілігі мамандандырылған программистер мен қолданушыларға арналған, сол себептен де болар қызмет көрсету және баптау біршама күрделі.

Microsoft компаниясы, керісінше, өзінің IIS веб серверін түсінуге қолайлы болатындай етіп, ортастатистикалық қолданушыларға дайындауға тырысып жатыр. Олардың баптауын мамандардың көмегінсіз жүргізуге болады.

Apache де, IIS де көптеген операциялық жүйелермен жақсы жұмыс жасайды. Мысалы, Mac OS, Linux, Microsoft Windows түрлі модификациясымен және т.б. көптеген түрлерімен. Бұл екі сервер жұмыс жасағанда ыңғайлы, жылдамдығы жақсы, оларға қосымша модульдерді қосуға болады.

Браузердің адрестік жолындағы қажетті URL-ді теріп болған соң, браузер пайдаланған протокол (HTTP) жөнінде мәлімет және сервер атын алады. Сервер атын IP- адреске ауыстыру үшін браузер DNS серверіне- жүгінеді. Алынған IP- адрес негізінде браузер ізделінді Web- сервермен байланыс орнатады және HTTP протоколын пайдалана отырып ізделінді ресурсты сұрайды. Сервер браузерге серверде сақталатын HTML бетін жібереді. Браузер HTML- тэгтерін оқу нәтижесінде сіздің компьютеріңіздің экранында бет ашалады, одан сіз өз сұранысыңыздың нәтижесін көре аласыз. Әдетте қарапайым Web беттерде тек текст қана емес графиктерден де тұрады, яғни әртүрлі типтегі бірнеше файлдан тұрады. Браузер ол файлдарды тану үшін, сервер қандай файлды (HTML форматындағы текст немесе JPG форматындағы графика және т.б) жіберу керек жөнінде мәлімет береді, содан соң файл мазмұны жіберіледі. Әдетте, Web беттер көптеген файлдардан тұрғанына қарамастан бір рет сұраныс уақытысынды Web сервер тек қана бір файлды жібереді. Яғни, html текст алып одан

графикалық элементке сілтеме тауып, браузер серверге жаңа сұраныс жібереді (html текст келген серверге ғана жіберу міндетті емес). Әрбір жаңафайлды көшіріп алу үшін браузер жаңа HTTP

сұранысын жіберу керек. Бұнда қазіргі серверлермен браузерлер көп ағынды режімде жұмыс істейтіндіктен бір мезгілде бірнеше сұраныс орындалуы мүмкін. Егер берілген адресте сұранып отырған ресурс жоқ болса онда Web сервер мынандай мәлімет береді*: 404/File not found (файл табылмады).*

Статистикалық және динамикалық беттер.

Статистикалық беттер Web сервер каталогында орналасқан файлдардың нақ копиясы болып табылады және өңдеуші өзі онда бір нәрсені ауыстырмайынша өзгермейді. Дегенмен беттер динамикалық түрде қалыптасады, яғни дискідегі дайын файлдан емес, белгілі бір программада сұраныстың өңделу уақытында. Осындай беттердің қалыптасуының бірнеше әдістері бар.

Тікелей Web серверде сұранысқа сәйкес қалыптастыру.

Web беттерді динамикалық қалыптастырудың мүмкіндігін жүзеге асыру үшін серверге мынадай бағыт берілуі керек, қандай файлдар «кәдімгі» болады, ал қандайы оның программалық өңделуіне нұсқаулары болады. Бұл жағдайда бетті сервердің өзі құрастырады (арнайы командалар көмегімен немесе ішкі бағдарлама). Беттердің динамикалық қалыптастыратын командасы болатын программа мәтіні *скрипт* деп аталады.

Тұтынушы компьютерінде қалыптастыру.

Бұл жағдайда программа мәтіні динамикалық web- парағын қалыптастыру үшін алдымен тұтынушының локальдық компьютеріне беріледі, бұнда браузер web- парағын алу және өңдеу үшін оған сәйкес заттар шақыру керек. Динамикалық беттерді қалыптастыру үшін бірнеше технологиялар бар.

CGI- технологиясы.

Біздің сұранысымызға сәйкес дайындалған динамикалық беттер «жылдам қалыптасты». Мысалы кез-келген пікірлер кітабы сізге белгілі бір форманы көрсетеді, мұнда сіз өз мәтініңізді қосасыз, келесіде осы бетті ашсаңыз онда жаңа хабарлама тұрады. Web-парағына динамикалық мазмұнды қосуға мүмкіндік жасайтын технологиялардың бірі CGI (CommanGateway Interface) болып табылады. Ол сол немесе басқа URL мен статистикалық документі емес программаны түсінуге мүмкіндік жасайды нәтижесінде нақты уақытта мәліметтер қалыптастырады. Мысалы егер сіз белгілі бір аймақтағы ауа райының сол кездегі мәліметін бергіңіз келсе, онда сіз әр бір ретте жаңа бетті құруыңыз керек. Бұл CGI технологиясының негізінде жүзеге асыруы мүмкін. Серверде жұмыс істеу бастағанда CGIпрограммасы қосылады, ол цифрлы өлшеуіш құралына айналып температура, қысым және т.б мәліметтер береді. Әр кезде осы адрестен мәлімет алу үшін байланысқаныңызда сіз сол уақыттағы мәліметті аласыз. Басқа мысал: егер сіз ізденіс жүйесінен белгілі мәлімет алғыңыз келсе онда CGI программасы жұмысының нәтижесін ізделінді адрестер жиынтығы түрінде аласыз. CGI программасын нақты уақыттағы Web сервердің бір бөлігі ретінде қарастыруға болады. Сервер тұтынушының сұрағын CGI программасына береді, ол оларды өңдеп жұмыс нәтижесін тұтынушы экранына қайтарып береді. Клиент үшін адресте URL статистикалық құжат па немесе CGI программа ма еш қандай айырмашылығы жоқ. CGI программалары жұмысының нәтижесі статистикалық құжат сияқты форматта болады. CGI терминін тек қана программа емес протокол ретін де түсінуге болады. Бұл жағдайда CGI Web сервер үшін стандартты тәсіл болып табылады– тұтынушы сұранысын бағдарламаға беру және одан мәлімет алу. Сервер мен оның қосымшасының арасындағы бір біріне мәлімет жіберу жөніндегі CGI протоколы HTTP протоколының бір бөлігі болып табылады. CGI программасының үлкен бөлігі CGI скрипталары болып табылады. Скрип дегеніміз интерпритацияланатын немесе басқа программаларымен жұмыс жасайтын ережелер жинағы. Perl, JavaScript тілдері тура осы скрипталық тілдер түрінде ойлап табылған. Олар сценарийлер жазу тілдері деп те аталады. Негізінен CGI программасы скрипттік тілде және де компилярлық тілде жазылуы мүмкін. C,C++, Delphi. CGI альтернативті технологиясы Microsoft компаниясының технологиясы болып табылады. Ол былай аталады Active Server Page (ASP) ол да сол принциппен құрылған: web-серверге қосылған скрипт, парақ тұтынушыға жіберуден бұрын серверде орындалады. Осы принциппен орындалатын басқа да бір қатар технологиялар бар. Динамикалық мазмұны сервер жағындағы қалыптасатын схемадан өзгеше динамикалық мазмұны тұтынушы жағында көрінеді. Соңғы жағдайда активті құжаттар webсерверде және локольдік компьютерде сақталады. Онда белгілі бір есептеулер орындалады және осы есептеулер нәтижесі экранда көрсетіледі. Активті құжаттарды дайындауда түрлі технологиялар пайдаланады: бұл мәліметтер JavaScript те Java апплеттерде жазылған қосымша болуы мүмкінжәне басқару элементі ActiveX болады.

ДӘРІС №6. WEB-РЕСУРСТАРДЫ АЛУ ТЕХНОЛОГИЯСЫ. WEB-ПАРАҚТАРДЫ КЭШТЕУ. ПРОКСИ-СЕРВЕР МЕХАНИЗМІНІҢ ЖҰМЫСЫ

Дәрістің мақсаты: web-ресурстарды алу технологиясымен танысу, web-парақтарды кэштеу.
Дәрістің мазмұны: web-парақтарды кэштеу. Прокси-сервер.Құжаттардың жарамдылық мерзімі. WEB-жылдамдатқыштары. Кэштеупроцесін оптимизациялау арқылы жылдамдату. Индексті құру. Домендік атты теру. Сілтемелердің тематикалық жинағы. Индекстелген каталогтер.

Web-парақтарды кэштеу.

Егер сервер үлкен жойылуда болса, Интернетке шығу жылдамдығы төмен болса немесе қайта қосылу режимінде жұмыс істесе, көбінесе ақпарат кешіктіріліп беріледі. Бұл кезде қосылуды жылдамдату сұрағы туындайды. Осы сұрақты шешудің бір жолы, мәліметтерді жойылған серверлерден бірнеше рет алуға болмауында. Web навигациясы кезінде көбінесе кітаптағындай «парақты артқа парақтау» сияқты, яғни параққа қайта қосылу қажеттілігі туындайды. Браузерде *Артқа* пернесін басып, алдында көріп өткен параққа қайта оралғанда, оны серверден қайта алудың мағынасы жоқ екені анық. Сондықтан көріп өткен құжатты компьютердің дискінің арнайы буферлік аймағында (кэште) сақтаса, қолданушының қайтаоралғысы келген жағдайда сілтемені тез ашып көруге мүмкіндігі болады. Қазіргі заманға сай браузерлер дәл осылай жұмыс жасайды. Мәселен, сіз парақтарды 1-2-3 кезегімен қарап, 3-ші парақ 4-ге ссылкасы бар деп ойлайық. Осы жағдайда 3-ші парақтан 4-ге өтумен (Жүйеден жүктеу) салыстырғанда 2-ге өту тез (кэштан жүктеу) орындалады.

Қазіргі заманға сай браузерлерде (Internet Explorer және Netscape Navigator) кэштелген құжаттарды сақтау үшін папканың көлемін өзгерту мүмкіндігі бар.

Прокси-сервер.

Кэштеу технологиясында өзінің шектеулері бар. Мысалы, сіз және сіздің әріптесіңіз Интернетке бір провайдер арқылы қосылған болсаңыз. Біркезде сіз, алдында сіздің әріптесіңіз алған құжатты сол провайдердің каналынан сұрайсыз. Жүйеден қайта-қайта бір құжатты сұрау дұрыс емес. Мәліметтерді сервер провайдер деңгейінде кэштау дұрыс.

Бұл мәселені шешу провайдерлер прокси-серверге (ағылшынның *proxy*-орынбасар деген сөзінен) жүктейді. *Прокси-сервер* – бұл қауіпсіздікті,әкімшілік бақылауды және кэштау функцияларын жоғарылатуға мүмкіндік беретін, қолданушының жұмыс станциясы мен Интернет арасындағы посредник ролін атқаратын сервер.

Прокси-сервер қолданушыдан қандай да бір интернет-сервисті орындауға сұраныс алады, мысалы, web-парақты көріп өтуге сұраныс. Егер прокси кэштау функциясын орындаса, ол сервердің локальді кэшінде сұрап отырған парақтың жоқтығын қарайды. Егер бұл парақ бар болса, ол қолданушыға сұраныс Интернетке берілмей қайтарылады. Ал, егер кэшта бұл парақ болмаса, прокси-сервер клиент ролінде қолданушының атынан, өзінің IP-адресін қолданып, Интернеттен керек параққа сұраныс жібереді.

Парақ қайта оралғанда, прокси-сервер оны қолданушыға жібереді. Прокси- сервердің кэштау функциясы Интернетпен жұмысты едәуір жылдамдатаалады жене жүйелік трафиктің көлемін азайта алады. Web-парақтарды кэштеу Жүйенің жұмысын жақсартудың бір жолы болып табылады, себебі ол web-түйіннің уақытты бұзылыстарын қалыпқа келтіреді.

Әдетте, прокси-сервер программасы жұмыс істейтін компьютерде едәуір дисктік аула бөлінеді. Үлкен провайдерлер құжаттарды кэштеугеондаған және жүздеген гигабайт жады бөле алады.

Осылайша, үнемі ізделіп отыратын құжат тек құжатты шығарушынынсерверінде ғана сақталып қоймай, сонымен қатар «қолданушыға жақынырақ», провайдер прокси-серверінде немесе қолданушының өз компьютерінде сақтала алатындықтан, іздеген құжаттарды тез көре алу мүмкіндігі бар.

Біз мыңдаған клиенттері бар провайдерлер жөнінде айтқана, жүйедегі құжаттардың белгілі бір бөлігін ғана сұрайтынын айтып өткен жөн. Мыңдаған клиенттермен жұмыс істейтін үлкен ISP-дің прокси-сервері, сұраныстардың 50%-ға жуығын өз дискінен орындайды. Әдетте ISP сервері регионның басқа интернет провайдерлердің серверлерімен байланысты. Сондықтан, егер клиентті қызықтырып отырған құжат провайдердің прокси-сервер кэшінде табылмаса, ол басқа прокси-серверлерде табылуы мүмкін. Осылайша, кэш-ауласындағы қолданушылардың көлемі көп есе көбейеді.

Құжаттардың жарамдылық мерзімі.
Ескірген кэштелген файлды алмау үшін, кэштелген құжаттарды қашан сұранысқа қоюға болады, ал қашан қоюға болмайтығын анықтайтын бірнеше ережелер бар. Бұл ережелердің бір бөлігі HTTP протоколында жазылып өткен, ал қалғаны браузер баптауында және прокси-сервер әкімшілігімен ұсынады. Прокси-серверде кэштеуге шешім қабылдаудың бірнеше критерилері бар. Мысалы, егер объект жасырын болса, ол кэштелмейді. Сол сияқты кэштелген құжатты клиенттің сұранысы бойыншажіберіле алуға шешім қабылданатын критерилер бар. Мысалы, егер құжаттың жарамдылық мерзімі өткен болса, онда сервер оны клиентке жібермей, сайтты құрастырушының серверінде жаңалау құжаттың бар, жоқтығын сұрайды. Жарамдылық мерзімі өтіп кеткен құжат жай жаңартылмаған, яғни ескірмеген болуы мүмкін және оны клиенттің сұранысына беруге болады.
Web-жылдамдатқыштары.
Стандартты браузерлер кэштеу технологиясын Web-парақтарды жүктеуді жылдамдату үшін қолданатынын айтып кеткен болатынбыз. Бірақ,жүктеу процесін Internet Explorer-ға қарағанда жақсырық орындайтын арнайы программалар бар. Жылдамдатуды жүйелік қосылулардың параметрлерін оптимизациялау арқылы, мәліметтерді кэштеу және «қажеті жоқ» мәліметтерді алып тастау арқылы жүзеге асыруға болады. Кейбір
программаларда жоғарыда айтып өткен механизмдердің бірден бірнешеуіжүзеге асырылған.
Кэштеу процесін оптимизациялау арқылы жылдамдату.
Мәліметтерді кэштеу кезінде Web-парақтарды жүктеуді жылдамдатуды айтқанда келесі бағыттарды белгілеп өткен жөн:

- IP-адрестерді кэштеу;
- Локальді компьютерде дискті кэшті оптимизациялау;
- Әрбағыттың неден құралғанын түсіндіріп өтейік.

Сіз URL браузерге сол немесе басқа ресурсты жібергенде, ол DNS- серверге доменді атты IP-мекенге жіберуге сұраныс жасайтыны белгілі. Қажетті мәліметтерді табу алдында қаншама DNS-серверлерді сұрап өтеді. Әдетте, IP-адрес қолданушы компьютерінде кэштелмейді және сіз бірнеше минут алдын қолданып отырған мекен қайта сұралып отырады, ол тағы да уақыт алады. Сондықтан, Web-парақтарды көрсетуді жылдамдатудың бір жолы DNS-серверлерді едәуір оптималды сұрау жүргізетін және *IP- мекендерді кештеуді* ұйымдастыратын программаны қолдану болып табылады.
IP-мекенді алып болғаннан кейін, шолушы (обозреватель) компьютердің кэшінде көрсетілген парақ көшірмесінің сақталғандығын тексеріп, егер кэштегі құжат жарамды болса, ол қолданушыға жіберіледі. Бұл процесте уақыт алады және оны жылдамдату – бөлек мәселе, сондықтан оны басқа программаға жүктеуге болады. Осы мәселемен айналысатын көптеген программалар көп орын алатын, бірақ жылдам істейтін өз кэшін құрады. Жылдамдатудың екінші бағыты осыны құрайды. Үшінші механизмді яғни, алдын-ала ескертілетін оқытуды(упреждающее чтение) түсіндіру үшін, кәдімгі браузердің жұмысына қайта оралайық. Құжат браузермен көрсетілгеннен кейін, әдетте үзіліс болады. Қолданушы құжатты оқығанша жүктеу процесі тоқтатылады, яғни, жүйе жұмысы тоқтап қалады. Бұл механизм үзіліс кезінде қосылыстарды қолдануға мүмкіндік береді, бұл кезде осы параққа сілтемесі бар құжаттар жүктеледі. Осы құжаттардың фондық режимде жүктелуі қолданушы сұраныс жібергенде тез көрсетуге мүмкіндік береді. Үшінші механизм осы *префетчингтан* (*prefetch*-ағылшынның алдын-ала алып келу деген сөзінен) құралады. Кейде осы механизмді алдын-ала ескертілетін жүктеме (ағылшынның *pre-loading* деген сөзінен) деп те атайды.

Бақылау сұрақтары:

1) Web-парағын кэштеу принципі неде?
2) Прокси-серверінің жұмыс механизмі неде?
3) Web-парағын кештеу оптимизациясының механизмі қандай?
4) Web-парағын кештеудің үш негізгі бағытын атаңыз?

ДӘРІС №7. ИНТЕРНЕТТІҢ ІЗДЕУ ТЕХНОЛОГИЯЛАРЫ

Дәрістің мақсаты: қажетті ақпаратты іздеу принциптерімен, іздеу машинасының жұмыс механизмімен танысу.

Дәрістің мазмұны: қажетті ақпаратты іздеу принциптері. Іздеу машинасының жұмыс механизмі. Индексті құру.

Web-тен іздеу.

Интернетте миллиондаған сайттар бар, соның ішінде өзекті ақпаратпен қоса көптеген ескі қорлар орналыстырылған. Интернет – белгілі бір басқарушысы жоқ демократиялық ақпарат көзі болып табылады. Кез келген адам желіге өзінің қорын орналастыра алады. Қорытындылап келгенде, интернетте ақпараттың қайталанбауына, оның стандартқа сай келуіне көп адамдар мән бере бермейді. Желіде барлығы бар екені белгілі, бірақ желіден қажетті ақпаратты алу қиын. Яғни, мәліметті табу үшін, оны жақсы іздей білу керек. Осы бөлімде интернет желісімен жұмыс істейтін іздеу аспаптары сипатталып, іздеу жүйесінің жұмыс механизмі түсіндірілген, іздеу оптимизациясына практикалық түсініктеме берілген. Интернетте ақпаратты іздеуге арналған мынадай әртүрлі аспаптар бар: іздеу машиналары (поисковиктер), индекстелген каталогтер (рубрикаторлар), рейтингілер, метаіздеуіш жүйелер және тематикалық сілтемелердің тізімі, онлайн энциклопедиялары мен анықтамалар. Осы кезде әртүрлі үлгідегі ақпаратты табуда іздеу аспаптарының түрлі категорияларын қолдану тиімді болып келеді. Әр категорияны жеке қарастырайық.

Индекстелген каталогтар.

Каталог дегеніміз тақырыптары бойынша топтастырылған иерархиялық құрылым түрінде берілетін мәліметтер. Иерархиялық құрылымның бірінші деңгейіндегі тематикалық бөлімі «спорт», «демалыс»,

«ғылым», «дүкендер» сияқты кең тараған тақырыптардан тұрады. Ал әр бөлімнің бөлімшелері болады. Осылайша, біртіндеп каталог бұтақтары арқылы саяхат жасап, іздеу облысын кішірейте отырып, сіз өзіңізге керекті облысты дәл анықтай аласыз. Мысалы оқу орындарын іздеу барысында мынадай тізбек пайда болуы мүмкін: *Білім-> Оқу орындары -> Жоғары оқу орындары ->Институттар*.

Қажетті ішкі катологты тапқаннан кейін, одан сілтемелер жинағын аласыз. Катологтарды программалар емес, адамдар құрастырғандықтан, катологтағы барлық сілтемелер профильді болып табылады. Егер сіз ортақ тақырыпта жалпы ақпарат іздесеңіз, онда каталогтарға қатынаған дұрыс. Алегер сізге нақты бір құжатты табу керек болса, онда каталог тиімсіз іздеу құралы болып табылады. Желіде ортақ қолданылатын каталогтардан басқа, ерекшеленгенкаталогтар да бар. Егер де бір каталогта өте көп қор орналасса, онда олардыкең таралуына байланысты бірнеше бөліктерге бөлуге (ражнирование) болады. Мысалы, Яндекс каталогында бөліктеу басқа сайттардың біздің сайттағы сілтемелерінің индексімен жүргізіледі. Желіде каталогтардан басқа рейтингтер де бар. Каталогтан рейтингтіңайырмашылығы, мұнда қорларды тікелей оның иесі суреттесе, ал каталогта - авторы, демек оның редакторлары суреттейді.

Сілтемелердің тематикалық жинағы.

Сілтемелердің тематикалық жинағы – бұл кәсіби топтармен немесе жеке жинақтаушылармен құрылған тізімдер. Шектелген кәсіби тақырыпты ірі каталогтің жұмыскерлер тобына қарағанда сол жұмыстың кәсіби маманыжақсы ашуы мүмкін.

Домендік атты теру. Каталог – бұл ыңғайлы іздеу жүйесі, бірақ егер сізге Intel немесе IBM компаниясының сервері керек болса, сіз каталогке қатынай алмайсыз.Сәйкес сайттардың атын табу қиын болмайды: *www.intel.com, www.ibm.com.*Сол сияқты, сізге егер ауа райына арналған сайт қажет болса, оны *www.weather.com* серверінен іздеген дұрыс болады. Көп жағдайларда кілттік сөз арқылы сайтты табу мәтінде көп кездесетін сөзден тұратын құжатты тапқаннан ыңғайлы. Іздеу кезінде танымалы емес компаниялардың адресінің атын ойдан тергенде, бірде-бір іздеу жүйесінде тіркелмеген сервермен байланыс орнатуы мүмкін болғандықтан, ол іздеудің басқа түрлерімен табысты бәсекелесе алады. Осыған ұқсас іздеулер тиімсіз, сондықтан ізделінетін сайттың атын таба алмайтын болсаң, іздеу машинасын қолдану керек.

Іздеу машиналары.

Сұранысқа жауап ретінде сіз әдетте құжаттардың ұзын тізімін аласыз, оның көбі сіздің сұрағыңызға жауап бермейді және сол тақырыпқаешқандай қатысы болмайды. Сондай құжаттар

релевантты емес (ағылшын сөзінен шыққан, *relevant*- лайықты, қатысты) деп аталады, ізденіс бойынша табылған құжаттар релевантты құжаттар деп аталады.
Табылған сілтемелердің тізіміндегі релевантты құжаттардың проценті сұраныстың дұрыс қойылуына байланысты болады. Іздеу машинасы тапқан барлық құжаттардың ішіндегі релевант құжаттардың бөлігін іздеу дәлдігі деп атайды. Релевантты емес құжаттарды шуы бар құжаттар деп атайды. Егер табылған құжаттардың барлығы релевантты болып келсе (шуы жоқ құжаттар), іздеу дәлдігі 100% құрайды. Егер барлық релевантты құжаттар табылса, онда іздеу толымдығы 100% тең. Сайып келгенде, іздеу сапасы екі өзара тәуелді параметрлермен анықталады: дәлдікпен және іздеу толықтығымен. Толықтықтың артуы дәлдікті төмендетеді және керісінше.
Іздеу машинасының жұмыс механизмі.
Іздеу жүйелерін анықтама қызметімен салыстыруға болады, онда агенттер кәсіпорындарды аралап, мәліметтерді деректер базасына жинайды. Клиент анықтама қызметіне жолыққанда ақпарат сол деректер базасынан алынады. Мәліметтер базада ескіріп отырады, сондықтан агенттер оларды оқтын-оқтын жаңартады. Кейбір кәсіпорындар мәліметтерді өздері жібереді, сондықтан агенттердің оларға барудың қажеті болмайды. Басқаша айтқанда, анықтама қызметінің екі функциясы болады: жасау және деректер базасын тұрақты жаңарту және клиент сұранысы бойынша базадан хабар іздеу. Сол сияқты, іздеу машинасы да екі бөлімнен тұрады: робот – ол берілген серверлерді аралап деректер базасын қалыптастырады және іздеу механизмі. «Робот» терминінің көптеген синонимдері бар, роботтан басқа оны желілі агент немесе торапта жүргеніне байланысты құрт немесе өрмекші дейді. Робот базасы негізі роботтың өзімен (робот өзі жаңа қорларға сілтемелер тауып алады) және аз дәрежеде өз сайттарын іздеу машиналарында тіркейтін қор иелерімен қалыптасады. Деректер базасын қалыптастыратын роботтан басқа табылған сілтемелердің рейтингісін анықтайтын программа бар. Іздеу машинасының жұмыс принципі пайдаланушы көрсеткен кілттік сөздер арқылы ішкі каталогтан (деректер базасы) релеванттігі бойынша сұрыпталған сілтемелер тізімін беру арқылы іске асырылады. Іздеу жүйесі тек ішкі каталогтармен операциялайтынын атап айтқан жөн. Іздеу машинасының мәліметтер базасы жүйедегі түйіндік адрестерді сұрау арқылы әрдайым жаңартылатынына қарамастан, іздеу машинасының ішкі қорларын және желі қорларын салыстыруға келмейді, сондықтан әрқашан машина ескірген адрес немесе қажетсіз ресурс табатыны өте ықтимал. Проблема тек қана ішкі қорлардың шектілігінде ғана емес, тағы роботтың жылдамдығының шектілігінде тұр. Іздеу машинасының ішкі қорларының көбеюі проблеманы шешпейді, себебі аралау жылдамдығы ақырғы болып табылады. Бірақ іздеу машинасының ішінде каталогтарға бөлінген Интернеттің кіріс қорларының белгілі бөлігінің көшірмесі болады деуге болмайды. Толық ақпарат (кіріс құжаттар) бәрі бірдей сақталмайды, көбіне жиі тек оның бөлігі – индекстенген тізім немесе индекс деп аталатын, құжат жолынан шағын бөлігі сақталады. Индекс құрау үшін кіріс мәліметтер қор көлемі минималды, ал іздеу тез әрі максималды пайдалы ақпарат беретіндей түрлендіріледі. Индекстелген тізімді түсіндіру үшін оның қағаз аналогы – конкорданс, яғнисөздікті келтіруге болады, онда белгілі жасушымен қолданылатын сөздер алфавиттік тәртіпте болады және де жазушы шығармасында келтірілгеніне сілтеме болады.
Айтпаса да түсінікті, конкорданс (сөздік) шығарма тексінен шағын және одан керекті сөзді іздеген кітапты түгел парақтағаннан көп жеңіл.
Индексті құру.
Желілік агенттер немесе робот-өрмекшілер Желі бойымен «өрмелейді», Web – беттерді талдайды және не әрі қай парақта табылғаны туралы ақпарат жинайды. Кезекті HTML-парақтарды табысымен көптеген іздеу машиналары (әр іздеу машиналарында әртүрлі) сөздерді, суреттерді, сілтемелерді және де басқа да элементтерді белгілейді. Сөздердің парақта

барлығы ғана емес, әрі оның орналасуы, яғни бұл сөздің қайда орналасқаны: тақырыпта (title), тақырыпшаларда (subtitles), метатэгте (meta tags) немесе басқа орындарда. Әдетте негізгі сөз ескеріледі де, шылау мен одағайлар: «ал», «бірақ» және «немесе» еленбейді. Метатегтер парақ иелерінің өзіне сол арқылы ізделінетін кілттік сөздер мен тақырыпты анықтауға мүмкіндік береді. Іздеуден осы сияқты сайттарды шығару – жақсы іздеу жүйесінің тағы бір тапсырмасы. Әрбір роботтың өз қараниетті жарнама үшін жазаланған қор тізімі бар. Тапсырма берілген Web-парақтарда ақпарат жиналғаннан кейін алынған мәліметтерді индекстеу жүреді. Робот-өрмекшілер Web-парақтардың ақпараттарын қарастырып, кілттік сөздер арқылы индекстенген іздеу базасын құрады, содан кейін пайдаланушы сұранысы арқылы жүйе дұрыстығына (релевантты) қарай сайттар тізімін береді. Айқын, егер сіз сайтты «гүл» деген кілттік сөзбен іздесеңіз, онда іздеу машинасы сол сөз бар парақтарды тауып қана қоймай, бұл сөздің қай жерде сайт тақырыбына қатыстылығын анықтай алуы керек. Сөздің Web- парақтың профиліне қатыстығын анықтау үшін оның парақта қаншалықты жиі ұшырасатынын, берілген сөз туралы сілтемелердің бар-жоқтығын бағалау керек. Қысқаша айтқанда, парақта табылған сөздерді маңыздылық дәрежесіне қарай рангілеу керек. Сөздерге салмақтылық коэффициенттері оның қанша және қайда кездесетініне қарай (парақ тақырыбында, беттің басы не аяғында, сілтемеде, метатегте және т.б) меншіктеледі. әрбір іздеу механизмі салмақ коэффициенттерін берудің өз алгоритмдері бар – бұл әртүрлі іздеу машиналарының бір кілттік сөз арқылы сұрауға әртүрлі қорлар тізімін берудің бір себебі. Парақтар әрдайым жаңартылып отыратындықтан, онда индекстеу үрдісі де жиі орындалып отырылуы керек. Робот-өрмекшілер сілтемелерді аралай жүріп, индекстен тұратын файлды құрады, ол үлкен болуы мүмкін. Оның көлемін азайту үшін ақпарат көлемін минимизациялау мен файлды сығуға жүгінеді. Өңделгеннен кейін мәліметтер үнемі жаңарып отыратын базада сақталады. Бірнеше роботтары бар іздеу машинасы секундына жүздеген парақтарды өңдей алады. Бүгінде мықты іздеу машиналары жүздеген миллион парақты сақтайды және күніне ондаған миллион сұранысты қабылдайды.

Индекс арқылы іздеу. Индекс арқылы іздеу мынадан құралады, яғни пайдаланушы сұраныс құрастырып оны іздеу машинасына береді. Бірнеше кілттік сөздерді қолдануда сұраныс тілін пайдаланған пайдалы, оның негізін буль операторлары құрайды.

Ең жиі қолданылатын буль операторлары:

- AND – бұл арқылы біріктірілген барлық терминдер ұсынылған құжатта қатысуы керек. Кейбір іздеу жүйелері «+» белгісін «AND» орнына қолданады;
- OR – кем дегенде бір кілттік сөз «OR »-мен қатысты, ізделінетінқұжатта болуы керек;
- NOT- «NOT»-тан кейінгі кілттік сөз ізделінетін құжатта кездеспеуі керек. Кейбір іздеу жүйелері «-» белгісін «NOT» орнына пайдаланады;
- FOLLOWED BY – кілттік сөздер бірінен кейін бірі кезектесіп келуі керек;
- NEAR – сөздердің бірі екінші сөзден белгілі санды сөздерден кейін келуі керек;
- тырнақшалар – тырнақша ішіндегі сөздер - бұл текст фрагменті құжат немесе файл ішінде кездесуі тиіс. Айта кетейік, сұраныс тілі семантикасы нақты бір іздеу машиналарында бір біріне ұқсамауы мүмкін, әдетте ол туралы іздеу машинасының нұсқауында мәлімет келтіріледі.

Шектерінде логикалық комбинация анықталатын мәтін *іздеу бірлігі* деп аталады. Бұл сөйлем, абзац не бүкіл құжат болуы мүмкін. Түрлі іздеу жүйелерінде әртүрлі іздеу бірліктері қолданылуы мүмкін. Сөйлем шегіндегііздеу тек индексінде толық мекенжай (адрес) бар жүйелерде ғана болуы мүмкін.

Пайдаланушы іздеу жүйесіне сұраныс жібергеннен кейін, ол сұраныс синтаксисін өңдейді, кілттік сөздерді индекстегі сөздермен салыстырады. Содан кейін сұранысқа жауап беретін сайттар тізімі релеванттылығына қарай рангіленіп, пайдаланушыға берілетіндей іздеу нәтижесі құрастырылады.

Бақылау сұрақтары:

1) Интернетте қажетті ақпаратты іздеудің жалпы принциптеріқандай?
2) Іздеу машинасының жұмыс механизмі қандай?
3) Интернетте ақпарат іздеу технологияларында индекс ұғымынені білдіреді?
4) Индексті құру принциптері қандай?

Дәріс №8. Интернет сервистері. Электрондық пошта. E-mail сервисінің жұмыс істеу механизмі. E-mail-дің екі протоколы. Электрондық поштаның адресі

Дәрістің мақсаты: интернет сервистерінің жұмыс жасау механизмімен танысу.
Дәрістің мазмұны: екі сервис – екі протокол. Пошталық клиенттің негізгі функциялары. Хатты жіберу және жеткізу. Арнаулы таңбаларды және толтырма хабарламаларды жөнелту.

Электрондық почта кез келген құжатты – ол мейлі қызмет жайлы жалпы мәлімет бола ма немесе жай хабар түріндегі мәтін бола ма – бәрі бір олардың ақпараттық негізін бірдей түрде нық ұстап, нақты әрі сенімді түрдежеткізе алады.
Электрондық поштаны пайдалану кезінде компьютерде әртүрлі арнаулы программалар қолданылады және олардың қажеттілігіне қарай бірінен біріне жеңіл ауысып отыруға болады. Мұндай программалар мынадай жұмыс атқарады:

- алынған мәліметтерді талдау және жаңа мәліметер дайындау;
- байланыс торабымен жылдам мәлімет алмасу.

Кәдімгі почта тәрізді электрондық почта да бір абонент пен екінші абонент арасында мәлімет алмасу үшін қолданылады. Жіберілетін мәліметте екі адрес көрсетілуі тиіс: хатты жіберуші және оны алушының аты-жөні мен оның компьютерінің желідегі адресі көрсетіледі. Хат иесінің адресін компьютер автоматты түрде өзі толтырады. Бұған қоса электрондықпочта адрестерінде мынадай мәліметтер жазуға болады:

- мәлімет тақырыбының қысқаша сипаттамасы;
- хаттың көшірмесі тағы қандай адрестерге жіберілуі тиіс.

Электрондық почта жұмысы барысында хаттың тақырыбына қоса оның жіберілген мерзімі, уақыты т.с.с. керекті мәліметтер тіркеліп қалады.
Көптеген желілер алғашқы ретте ғылыми, конструкторлық және оқу орындарын байланыстыратын жүйе ретінде жасалған болатын. Сондықтан ұжымдық пікір-талас жүргізуге арналған желі мүмкіндігі телеконференция деп аталып, ол белгілі бір салада ғана қолдануға негізделген еді. Бірақкейінен желілер практикада кең пайдаланылып, оның алғашқы ғылыми семинар, конференцияға арналған мүмкіндігі біршама шетте қала бастады.
Телеконференция ұйымдастыру кәдімгі кітапхана немесе архив жұмысына ұқсас жүргізіледі. Мұнда да көпсатылы тақырыптық каталогтар мен олардағы мәліметтерді сақтау ісі ұйымдастырылады.
Бірсыпыра байланыс тораптарының файлдық серверлері өз абоненттеріне өздерінде жинақталған ақпараттық материалдарды, яғни файлдар архивін пайдалануды ұйымдастырып жүр. Ол материалдар мәтіндік құжаттардан, кітаптардан, сызбалардан, компьютерлерге арналған программалардан т.б. тұрады. Бұл мәліметтер ұзақ уақыт сақталады және олардың көлемі де үлкен болады.

Интернетке кіру рұқсат етуді регистрациядан өткізу уақытында ISP қолданушыға пошталық жәшікке дискінің белгілі-бір көлемін, пошталық жәшіктің адресін (E-mail Account Address), қолданушының атын (E-mailAccount Login Name) және поштаға заңсыз кірудің рұқсатынан арылу үшін парольді (E-mail Account Password) бөліп береді. Электрондық пошта адресі: қолданушының_аты@домен_аты форматында болады, мысалы, *auesis@aipet.kz.* «@» таңбасының сол жағындағы бөлік – бұл сервердегі пошталық жәшіктің аты (Account Name), мұнан бұл адрестің иесі хаттардың алады (берілген мысалда Auesis). Ережедегідей, қолданушының аты пошталық жәшіктің атымен сәйкес келеді. «@» таңбасының оң жағындағы бөлік *домен* деп аталады және осы пошталық жәшіктің орналасқан жеріне көрсетеді. Электрондық поштаның адресін пайдаланушы жеткізудің ақырғыбекеті, яғни поштаны көріп жатқан компьютер болып саналмайтынын ескере кету жөн. Әр қолданушы бірнеше адрестерді әртүрлі пошталық серверлерде регистрациядан өткізе алады. Бүгінгі таңда электрондық поштаның қызметін ақысыз көрсететін жүздеген серверлер бар, сондықтан да кез келген Интернет қолданушы әлемнің әр бөліктерінде пошталық жәшіктерін емін-еркін регистрациялауы мүмкін. Сондықтан электрондық пошта адресін сіздің үйіңіздегі компьютердің адресі айқындамайды, ал Сіз пошта алатын сервердің адресі беріледі.

Белгілі бір қолданушыда екі адрес регистрацияланған деп ойлайық: sasha@ny-server.com және sasha@almaty-server.ru.
Ол әр жәшігінен өзінің компьютеріне хаттар алады, бірақ біреуін ол Нью-Йорктегі серверден, ал екіншісін – Алматыдағы серверден көшіреді. Осыған байланысты электрондық поштаның көрсеткіштік адресациясы жеткізу маршрутын адрес анықтайтыны туралы айтады.
Екі сервис – екі протокол.
Электрондық пошта клиент-серверлік архитектураның принципімен құрастырылған. Қолданушы клиенттік программамен араласады, ол программа өз қатарымен пошталық сервермен араласады. Поштаны жіберу және алу процедурасы әртүрлі дәрежеде жеке адамның анықталуын (идентификация личности) сұрайтыны айқын, осыдан мұнда кәдімгі поштадағыдай толық ұқсастық (аналогия) көрінеді: сізге өзіңіздің пошталық жәшіктің ішінен хатты алу үшін кілт керек. Бұл жағдайда кілт сіздің поштаны алу құқығы бар екендігін растайтын автоматизация құралы болып табылады. Хатты жіберу кезінде авторизация міндетті емес. Жіберу және алу әртүрлі дәрежеде жеке адамның анықталуын сұрайтынына байланысты екі түрлі – хатты жіберуге және алуға арналған протоколдар бар. Хатты жіберу кезінде авторизация міндетті еместігіне қарамастан, бүгінгі таңда провайдерлердің көпшілігі қауіпсіздік және спаммен күресу ойымен оны енгізу жоғары қойылады.
Хаттарды жөнелту үшін SMTP протоколы (Simple Mail Transfer Protocol – поштаға сілтеме қарапайым протоколы) және оған сәйкес SMTP- серверлері қолданылады. Көбінесе поштаны жіберу сіздің пошталық серверіңіздің интернет-провайдерінде болады, бірақта бұл онша міндетті емес. Әдетте SMTP-серверлер идентификацияны талап етпейді, сондықтан сіз хатты басқа да осындай серверден жіберсеңіз де болады. Пошталық хабарламаны алу үшін қазірі уақытта тек нақты қолданушының поштасын серверден алып тұратын және сондықтан жәшікке кіруге рұқсат ететін құқықтық авторизацияны сұрайтын POP3 протоколы (Post Office Protocol – пошталық офис протоколы) көп қолданылады.
Пошталық клиенттің негізгі функциялары.
Барлық пошталық клиенттердің негізгі функциялары – хабарламаларды алу, оларды автономды режимде көруді ұйымдастыру, хабарламаларды сорттау, жауап хабарламларын дайындауды автоматизациялау және адрестік кітапшаны қолдау.
Хатты жіберу және жеткізу.
Интернетке қосылғаннан кейін, клиенттік программа пошталық сервермен байланысып, оған жіберушінің пошталық адресін және хабарламаның мәтінінің өзін жібереді. Поштаны жіберу кезінде клиенттік программа шығыс поштаның серверімен, яғни SMTP-сервермен 25 нөмірлі портқа SMTP протоколы арқылы қосылып, ара қатынасқа түседі. Пошта жіберілетін SMTP-серверінің аты клиенттік пошталық программаға оны күйге келтіру кезінде алдын-ала енізілу керек. Осы процесті мұқият қарастырайық. Хат жіберушінің серверіне жеткізілгеннен кейін, жіберушінің SMTP-сервері алушының серверімен байланысу керек. Бұл үшін оған алушының SMTP-серверінің IP-адресін білу қажет. Осы адресті білу үшін, ол DNS-серверге алушының SMTP-серверінің IP-адресін білуге сұраныс тастайды.
Егер де қайсы-бір себеппен жіберушінің SMTP-сервері алушының SMTP-серверімен байланысқа түсе алмаса, онда жіберілім жеткізілу үшін кезекке тұрады. Әдетте әр 15 минута сайын кезектегі хатты қайта жөнелту әрекеті өндіріледі. Шамамен 4 сағаттан кейін жіберушіге проблема туралы хабарлама жіберіледі, ал 5 күн өткен соң серверлердің көпшілігі әрекеттерінтоқтатып, жіберушіге жіберілменген поштаны қайтарады, сондықтан хат із- түссіз жоғалып кете алмайды.
Егер пошталық клиент пен SMTP-сервердің қатынас протоколына қарасақ, оның жетерліктей қарапайым екендігіне көз жеткізуге болады. Мұнда интуитивті түсінікті командалар қолданылады: мысалы HELO (таныстырыламын), MAIL FROM (жіберушіні анықтаймын), RCPT TO (алушыны анықтаймын) және т.с.с.
Жіберушінің SMTP-сервері алушының SMTP-серверімен байланысқа түскен сәтте-ақ, ол жіберілімді жөнелтеді. Алушының сервері адресте көрсетілген атты қолданушының бар екендігін анықтайды және оның жәшігіне жіберілім жөнелтеді.
Пошта серверде орналасқаннан кейін, алушы оны сол жерден көшіріп алуға мүмкіндік алады. Пошталық жәшіктен жиналған поштаны алу кезінде, клиенттік программа POP3-сервермен POP3

протоколы арқылы байланысады. Интернетке кірген кезде өзіңіздің пароль мен логинді көрсеткендіктен кейін, әдетте поштаны алу кезінде осы мәліметтерді қайта енгізу керек емес. Сіздің пошталық клиентіңіз POP3-сервермен байланысып, локальді машинаға электронық пошта жіберілімдерінің копияларының жөнелтуін анықтайтын бұйрықтарды жібереді.
Әдетте, пошта алынғаннан кейін пошталық клиентте керісінше көрсетілмесе, сервердегі хабарламалар өшіріледі. SMTP-сервер секілді, POP3-сервер алушының пошталық клиентпен «сөйлесу» кезінде интуитивтітүсінікті командалар жиынтығын қолданады: USER (қолданушының ID енгізіңіз), PASS (пароль енгізіңіз), QUIT (POP3-серверден шығыңыз), DELE (хабарламаны өшіріңіз).
Егер SMTP-протокол қарапайым және тек қана серверге алушының адресі бар хаттардың мазмұнын жіберетін болса, онда POP3-сервермен жұмыс әлдеқайда күрделірек болатынын ескеру жөн, ол хаттарды бөлектеліп көшіруді іске асырады, үлкен хаттардан бас тартуға жол береді және т.б. Поштаны қабылдау үшін пошталық серверден хаттарды таңдамалы түрде компьютерге көшіріп алуға, папкалармен жұмыс істеуге және т.б. мүмкіндік беретін әлдеқайда жаңартылған IMAP-протокол қолданылуы мүмкін. Электрондық хат адресатқа жетпеген кезде, бірнеше себептермен болуы мүмкін. Мүмкін, алушының сервері берілген жіберушінен хаттар алмаған жағдайда. Мұнан басқа, егер сервер жұмысы аса көп болса және сұранысты қарастыралмаса, жіберілім кері қайтарылады. Егер хат алушы серверімен жоқталмаса да, ол қайта оралуы мүмкін, мысалы, алушы атын жазу кезінде қателессеңіз – жүйе мұндай атты алушының жоқтығы туралы хабарлайды. Және соңғысы, көрсетілген атты алушы бар болса да, оның пошталық жәшігі толып кеткен кездегі жағдай туындауы мүмкін.
Арнаулы таңбаларды және толтырма хабарламаларды жөнелту.
SMTP-протокол ASCII-дың барлық символдарын жіберуді рұқсат етпейді, ал олардың ішінен ағылшын алфавитіне сәйкес келетін және ASCII кодтаудың негізгі кестесімен айқындалады. ASCII кодтаудың екі кестесін анықтайтыны белгілі: базалық жетібиттік (0-127 нөмерлі символдар) және кеңейтілген сегізбиттік (128-255).
Кириллица символдары кодтаудың кеңейтілген кестесіне жатады, және SMTP-протоколы үшін олар арнаулы символдар болып келеді.Арнаулы символдарды жіберу кезінде әдетте MIME стандарты (Multipurpose Internet Mail Extensions – Интернеттің көпмақсаттың кеңейтілулерлі пошталық стандарты) қолданылады, ол арнаулы символдарды жіберу және мәтіндік емес ақпаратты электрондық пошта арқылы жіберу үшін кодтауды рұқсат етеді, мысалы хатқа қосымшада орналастырылған графикалық файлдар.
Бастапқыда электрондық пошта файлдарды жіберу үшін арналмаған еді, сондықтан қайсыбір файлды жіберу үшін оның мәтіндік хабарламаға сәйкес қайта кодировка жасау керек. MIME стандарты *файлдың тақырыбы арқылы оның қандай типті мәлімет* (мәтін, графика, әуен және т.б.) *қамтитынын анықтай алады* және қосарланған файлдарды кодтаудың біртұтас әдісін ұсынады. MIME стандарттары қолданған кезде, кириллицаны кодтау проблемалары туындамайды.

Бақылау сұрақтары:
1) E-mail клиентін ISP регистрациясы қалай өткізіледі?
2) Электрондық поштаның жіберу және жеткізу механизмікандай?
3) E-mail сервисінде қандай протоколдар қолданылады?
4) Арнаулы таңбаларды және толтырма хабарламалардыжөнелтудің ерекшеліктері?

Дәріс №9. Интернет сервисі. Файлдармен алмасу. FTPсервисінің жұмыс механизмі. FTP – клиенттер

Дәрістің мақсаты: FTP сервисінің жұмыс механизмі мен Интернет сервисімен танысу.
Дәрістің мазмұны: FTP файлдарын жіберу протоколы (FILE TRANSFER PROTOCOL). PTP-ресурсының адрес форматы.

FTP файлдарын жіберу протоколы(FILE TRANSFER PROTOCOL).

FTP (File Transfer Protocol–файлдарды жібер протоколы) 1971 жылы ұйымдастырылды және қазіргі уақытқа дейін жақсы қолданылуда. FTP TCP/IP базалық деңгейіндегі протолдарымен базаланады және өз алдына Интернетте компьютерлер арасындағы файлдаралмасуын көрсетеді.
FTP негізгі тағайындалуы – Интернеттегі файлдарды жойылған компьютерден локальдіге (Download) және локальді компьютерденжойылғанға (Upload) қайта жіберу (көшіру, қайта беру). FTP протоколының көмегімен сондай-ақ өз файлдарыңды жойылған компьютерден жоюға және атын өзгертуге болады. Айта кететін жайт, ақпаратты тек оқуға арналған WWW-серверлеріне қарағанда, FTP серверлері ақпаратты жүктеуге және де ақпаратты серверге енгізуді қамтамасыз етеді. Жекешеде, Web-беттерден конструкцияланатын файлдарды жіберу үшін, көбінесе FTP-протоколдары қолданылады. Әрине, серверде ақпаратты өзгерту қамтамасы, тек осы ақпарттың иесіне берілу қажет. FTP-серверіне қолданушының жолығу
кезінде рұқсаты логинмен және парольмен жарияланады, бірақ ол үлкен дәрістерге арналған желіде сервер жоқ дегенді білдірмейді. Көптеген серверлерге уақытша атпен кіруге болады (мысалы, anonymous немесе guest) және бұл кезде сізге, тек оқуға арналған файлдардың шектеулі жиынтығы беріле алады.

PTP-ресурсының адрес форматы.

Жалпы жағдайда PTP-ресурсының URL форматы мынадай түрге ие: ftp://<user>:<password>@<host>:<port>/<url-path>,
мұнда:

- user – қолданушы аты (логин);
- password - пароль;
- host – домендік ат немесе сервердің IP-адресі;
- url-path – файлға жол.

Әдетте портты көрсетпейді, бірақ бұл жағдайда 21-стандарты қолданылады.
Жалпы рұқсаты бар серверге жолығу кезінде, қолданушының аты ретінде anonymous қолданып, ал паролі ретінде - өзінің почталық адресін қолданамыз. URL-дағы анонимдік FTP үшін қысқартылған синтаксис қабылданған: ftp: //<host>/<url-path>, яғни аты жоқ болған кезде, автоматты түрде anonymous аты қолданылады.

FTP-клиенттері.

FTP қызметі клиент-сервер архитектурасына негізделген. Клиенттік компьютерде программалық-клиент жіберіледі, ол сервермен байланысады және жібереді немесе файлдарды қабылдайды. FTP-серверге рұқсат алу FTP-клиенттерінің әртүрлі класстарының көмегімен жүзеге асады, атап айтқанда:

- консольдік клиент – бұл *ftp.exe* программасы, ол Windows операциялық жүйесімен бірге беріледі,
- FTP-клиент, браузерге енгізілген (мысалы, Internet Explorerнемесе Netscape Navigator);
- FTP-клиент, файлдық менеджерге енгізілген (FAR немесеWindows Commander);
- FTP-клиент, HTML – редакторға енгізілген (HomeSite,Dreamweaver немесе FrontPage);
- Негізделген программалар (мысалы, CuteFTP немесе WS FTP).

FTP.exe көмегімен рұқсат беру.

ftp.exe – бұл консольдік клиент, оның графикалық интерфейсі жоқ және өте ыңғайлы. Біз мұны «бұл қалай жұмыс істейді» деген сұраққа жауап беру үшін мысалға алдық.
ftp.exe жүктеуі интерактивті сеансты шақырады: программа сізге командалық жолды қамтамасыз етеді және сауалдарды күтіп отырады. Диалог жүргізу үшін оншақты командалар қолданылады,

олардың кейбіреуітөменде көрсетілген:

- cd [жойылған_директория] - каталогты ауыстыру;
- mkdir [директории_аты] - каталогты жойылған машинада құру;
- get [жойылған_файл] [локальдік_файл] – жойылған файлдың көшірмесін сіздің компьютерге беру. Егер локальдік файл аты берілмесе, олжойылған файлдың атымен сәйкес келеді;
- reget [жойылған_файл] [локальдік_файл] – жойылған файлды ары қарай енгізу, егер оның жарты бөлігі локальді машинада болса. Бұл командакөбінесе біріктірудің мүмкін болған бөлінулерінде үлкен файлдарды алу үшін пайдалы;
- put [локальдік_файл] [өшірілген_файл] – файлды өшірілген жүйеге жіберу. Егер өшірілген файлдың аты көрсетілмесе, онда ол локальдік жүйедегі атаумен сәйкес келеді;
- delete [жойылған_файл] - жойылған_файл атты файлды өшіру;
- close – жойылған сервермен FTP-сеансын бөлу және командалық интерпретаторға қайтару;
- bye — FTP-серверімен жұмысты аяқтау (интерпретатордан шығуды қамтамасыз етеді).

Командалардың толық тізімін және қосымша мәліметтерді сәйкестендірілген RFC-тен (RFC аббревиатурасы Request for Comments (түсіндіру ұсынысы) – Internet Engineering Task Force (IETF) организациясымен басылатын документтер сериясы және көптеген Интернетті және оның технологиясын қамтитын сұрақтарды қарастырады) алуға болады.

Ескерте кететін жайт, FTP-протоколы ұзақ уақытты интерактивті біріктіруді, диалог жүргізуді және байланыс сеансын тоқтату командасы берілмегенше, біріктіруді қамтамасыз етеді. FTP-протоколы екі типті файлды түсінеді – текстік және екілік, мұнда барлық текстік емес файлдар екілік түрінде қабылданады. FTP-біріктірілу арқылы жүретін жұмыс текстік хабарламалардың ауысымы (қызметтік командалар) сияқты жүзеге асады, және де екі компьютер арасында ауысымда болатын файлдармен де жүзеге асады. Жұмыстың консольдік формасы қолданушылар үшін өте қолайлы емес, бірақ ол компьютер қай команданы орындап жатқанын түсінуге қолайлы жағдай береді.

Бақылау сұрақтары:

1) FTP-сервисінің Интернетте тағайындалуы қандай?
2) FTP-ресурсының Интернеттегі формат адресі қандай?
3) Қандай FTP-клиенттері Интернетте қолданылады?
4) Консольді FTP-клиенттерімен жұмыс кезінде қандайкомандалар қолданылады?

Дәріс №10. Интернеттегі мәліметтерді қорғау. Цифрлыққолтаңба. Криптография және Интернет. Симметриялық және асимметриялық кілттер. Цифрлық қолтаңбаның механизмі. Қауіпсіздік проблемалары

Дәрістің мақсаты: интернеттегі мәліметтерді қорғау мен қауіпсіздік проблемаларын шешу жолдарымен таныстыру, криптография мен интернеттің байланысын түсіндіру.

Дәрістің мазмұны: симметриялық және асимметриялық кілттер. Цифрлық қолтаңбаның механизмі. Интернет желісіндегі мәліметтерді қорғау. Қауіпсіздік проблемаларын шешу.

Нақты мысал қарастырайық. Фирмасының директоры В фирмасының директорына өте қажетті бір құжатты электрондық почта арқылы жіберді делік. Қарап отырсақ мұнда қандай коммуникация қауіпсіздігі пайда болады. В фирмасының директоры бұл хатты алғаннан соң келесідей сұрақтар пайда болады. Шыныменде осы хатты А фирмасының директоры жіберді ма? (жіберуші идентификациясы). Бұл сұрақтың туындау себебі, бұл құжатты А фирмасының директоры ретінде басқа біреудің де жіберуі мүмкін. Бұл құжатты жіберу кезінде жолдан ұстап өзгертулер енгізген жоқ па? (жіберу аутентификациясы). Бұл құжатты қабылдаушыдан басқа адамдар оқыған жоқ па? (құпияның сақталуы). Бәрімізге белгілі құпияны сақтау үшін мәліметтерді шифрлау арқылы жетуге болады. (жіберу конфиденциялы). Берілген 3 мақсаттарды шифрлау базасы арқылы шешуге болады, онымен криптография айналысады.

Криптография – (грекшеден аударылғанда Cryptos – құпия деген мағынаны береді) – бұл ғылым және шифрлау технологиясы мәліметтерді өзгертуге және авторлық құқықты сақтайды. Криптография тек қана тексттерді шифрланған формаға аударып қана қоймайды, сонымен қатар жүйеде жұмыс жасап отырған кезде қолданушының аутентификациясынжәне идентификациясының берілуін қадағалап отырады. Криптография ең басты қауіпсіз коммуникация болып табылады. Біз жүйеде жұмыс жасаған кезде тек қана адамдармен сөйлесіп қана қоймаймыз, сонымен қатар басқа да қызметтер барысында араласатынымыз бізге мәлім. Мысалға, біз қандай да бір серверден программа көшіретін болсақ, біз үшін осы сервер шығарушы – фирманың сервері ма, әлде біздің компьютерімізге вирус болып түсетін пираттық фирманың сервері емес па деген ойлар келеді. Жіберу кезінде құжатты жеке және публикалық кілттер арқылы шифрлауға болады, ал оның шифрын екі пардағы кілттер арқылы ашуға болады. Құжатты шифрлап, жеке кілт арқылы жіберсе, оның шифрын публикалық кілт арқылы ашуға болады, және де керісінше. Жеке кілт тек қана иесіне белгілі оны басқа біреуге беруге болмайды, осы уақытта публикалық кілт барлық корреспонденттерге ашық жария етіледі.

Қос кілттер – жеке және публикалық – оларды аутентификация және құпияны сақтау үішн қолдануға болады.

Екі адамның мәлімет алмасуы үшін екі қос кілт болуы қажет.

Шифрлау кезінде қос кілттің арқасында сізге публикалық кілтті корреспонденттерге жіберіп отыру қажет емес. Бұл кілтті жүйеге ашық жазып қою сізге ыңғайлы болады. Сол кезде барлық адамдар осы кілтті өздеріне жазып алып, сіздерге құпиялы мәліметтерді жібере алады.

Симметриялық және асимметриялық кілттер арқылы шифрлеу.

Әрбір уақытта ассиметриялық шифрлау симметриялық шифрлаудан тиімділігі жағынан жеңіліп қалады, сондықтан да көптеген адамдар шифрлау жүйесінде ассиметиялық және дәстүрлі симметриялық шифрлау жүйесін қолданады. Ашық кілтпен шифрлау симметриялық кілт қолданылады, бұл жіберілген мәліметті ширлау үшін қолданылады.

Цифрлық қолтаңба.

Цифрлық қолтаңбаның механизмін түсіндіру үшін, бір жақты хэшфункияны енгізу қажет. Біржақты хэш функция бұл функция, шығарылған мәліметтің ұзындығын белгілі ұзындыққа айналдырады, оны жіберу дайджесті деп атайды. 16 байттық хэш функцяны шығару кезінде сіз 16 байттық мәлімдеме аламыз.

Хэштеу – бұл біржақты, т.б, шифрлауға қарағанда қайтарымсыз. Дайджест бойынша шығарылған мәлімдемені қайтаруға болмайды, бірақ оған идентификация жасауға болады. Нақты бір мысал қарастырайық. Мысалы А хэш функция арқылы дайджестті қандайда бір код ретінде алынды делік (код 1). Одан соң ол А өзінің жеке кілтін пайдаланып, сол документтің аналогының қолтаңбасы болатын дайджестті шифрлайды. Содан соң А ашық текстті және цифрлық қолтаңбалы В жібереді.

В абоненті оны публикалық кілт арқылы шифрлап ашады да, бұл хаттың А келгеніне сенімді болады. Осылай аутентиикация орындалады. Содан соң бұл хаттың жолда өзгертілмегені жөнінде тексеру қажет. Соңында В шифрлау арқылы дайджестті алады – код 1. Содан кейін А сияқты В –да хэш – функцияны пайдаланып, дайджестті қандайда бір код ретінде алады –код 2. Егер код1 және код 2 бір біріне сай келсе, сонда ғана В бұл тексттің жолда өзгертілмегеніне көзін жетізеді.
Осындай жағдайда, цифрлық қолтаңба және электрондық қолтаңба – бұл әдіс жіберушінің аутентификациясы немесе авторлық қолтаңба, бұлмәліметтің мазмұнының өзгертілмеуіне себеп. Цифрлық қолтаңба шифрлау арқылы қойылуы да мүмкін, сонымен қатар ашық жіберуде орындалуы мүмкін.
Электрондық ақпараттық ресурстар мен ақпаратық жүйелерді қорғаудың мақсаттары:
Электрондық ақпараттық ресурстар мен ақпараттық жүйелерді қорғау:
- электрондық ақпараттық ресурстардың тұтастығын және сақталуын қамтамассыз ету, оларды рұқсатсыз өзгертуге немесе жоюға жол бермеу;
- қол жетімділігі шектелген электрондық ақпараттық ресурстардың құпиялығын сақтау;
- электрондық ақпараттық ресурстарға қол жеткізу құқығын іске асыру;
- электрондық ақпараттық ресурстарды өңдеу және оларды беру құралдарына рұқсатсыз ықпал етуге жол бермеу мақсатында құқықтық ұйымдастырушылық және техникалық (бағдарламалық техникалық) шараларды қабылдауды білдіреді;
Электрондық ақпараттық ресурстар мен ақпараттық жүйелердіқорғау:
- электрондық ақпараттық ресурстарды құрсаулау, яғни ақпараттық жүйеге және ол ұсынатын электрондық ақпараттық ресурстарға қолжетімділікті шектеуге немесе жабуға әкелетін іс-қимылдар жасау;
- электрондық ақпараттық ресурстарды түрлендіру, яғни бағдарламаға, деректер қорына, материалдық жеткізушідегі мәтіндік ақпараттарға өзгерістер енгізу;
- электрондық ақпараттық ресурсты көшіру, яғни ақпаратты басқа материалдық жеткізушіге ауыстыру;
- құқық иесінің рұқсатынсыз бағдарламалық өнімдерді пайдалану; *Электрондық ақпараттық ресурстарды қорғауды ұйымдастыру.*Электрондық ақпараттық ресурстарды қорғауды:
- жалпыға бірдей қол жетімді электрондық ақпараттық ресурстарға қатысты - электрондық ақпараттық ресурстарды тарататын тұлға;
-қолжетімділігі заңмен шектелген электрондық ақпараттық ресурстары бар ақпараттық жүйенің меншік иесі, иеленушісі немесе ұлттықоператоры;
- қолжетімділігін олардың меншік иесі немесе иеленушісі шектелген электрондық ақпараттық ресурстарға қатысты - электрондық ақпараттық ресурстардың меншік иесі немесе еиеленушісі ұйымдастырады.
Криптография екі бөліктен тұрады: криптография және криптоанализ.
Криптография – бұл заңсыз қолданушылардан қорғау үшінақпараттарды өңдейтін ғылым.
Криптоанализ – бұл дешифлеуді орындау және тәжірибеде оны қолдану әдістері туралы ғылым.
Криптографиалық терминдерде шығыс жолдауы ашық мәтінменаталады. Шифрлеу дегеніміз шығыс мәтіннің мазмұнын басқалардан жасыру үшін өзгертуді айтады. Шифромәтін деп шифрленген хабарды айтады. Шифрмәтіннен ашық мәтінді шығару процесін шифрден алып шығу дешифрлеу дейміз. Көбінесе шифрлеу және дешифрлеу процесінде кілт қолданылады, осы кілтті білген адам ғана шифрден мәтінді алып шығарады.
Шифрлеу алгоритм негіздері.
Криптоалгоритмге қарасты бірнеше классификация схемалары пайдаланылады. Барлық криптоалгоритмдердің классификациялар схемасының негіздері мыналар:
- құпиятізім. Жіберуші және қабылдаушы хабарламаны өздері ғана өңдей алады. Басқа адамдарға шифрлеу алгоритмі белгісіз болады;
- криптография кілтімен. Барлық адамдарға алгоритм белгілі, бірақ ол
«кілтке» байланысты, ол кілт тек жіберуші мен қабылдаушыда ғана бар;

- симметриялық криптоалгоритмдер. Хабарламаны шифрлеу және дешифрлеу үшін кілт қолданылады;
- асимметриялық криптоалгоритмдер. Хабарламаны шифрлеу үшін «ашық» кілтті қолданады, ол барлығына белгілі, ал дешифрлеу үшін «жабық» кілтті қолданады, ол тек қабылдаушыға ғана белгілі. Кез келген криптоалгоритм кілтімен біз құпиятізімгеайналдыруымызға болады. Ол үшін бағдарламадағы қолданып жүрген кодты өзгерту керек. Қайта ашу мүмкін емес.

Алгоритмдер деректерге байланысты бөлінеді:

- орын алмастыру. Ақпараттар жиыны, байттар,биттер және т.б. өзгермейді, бірақ олардың қатар тәртібі өзгереді, сондықтан басқаларға ақпаратты қолдану мүмкін емес;
- ауыстыруды тексеру. Криптоалгоритм заңы бойынша ақпараттар жиыны өзгереді. Көбінесе бастапқы алгоритмдер осы топқа жатады.

Криптоалгоритмдер ақпарат жиынының көлеміне байланысты бөлінеді:

- ағындық шифрлер. Кодтау өлшем бірлігі бір бит болып табылады. Кодтау шешімі алғаш үздіксіз кіріске байланысты емес. Бұл сұлба ақпараттарды жіберу жүйесінде қолданады, яғни кейбір жағдайда ақпартты жіберу кезінде ықтиярлы уақытта басталуы және аяқталуы мүмкін, не болмаса әлдеқалай тоқтап қалуы мүмкін. Көп қолданатын ағындық шифрлер – скремблерлер;
- блоктық шифрлер. Кодтау өлшем бірлігі бірнеше байттар тізбегі (қазіргі кезде 4-32) жатады. Кодтау шешімі осы блоктың барлық негізгі байттар түрінен тұрады. Бұл сұлба файлдарды кодтау және ақпараттар пакетін жіберу кезінде қолданады.

Кілт қолданатын алгоритмдер екі топқа бөлінеді:

- симметриялық алгоритмдер құпия кілтімен;
- асимметриялық алгоритмдер ашық кілтімен.

Айырмашылығы: симметриялық алгоритм шифрлеу және дешифрлеу үшін бір кілтті қолданады, ал асиметриялық алгоритм әртүрлі кілттердіқолданады. Дешифрлеу кілті мен шифрлеу кілті сәйкес келмейді.

Цифрлық қол қою. Кейбір асимметриялық алгоритмдерді цифрлық қол қою генерациясында қолданады. Цифрлық қол қою деп кейбір құпия кілтті қолданатын генерлирленгенмәліметтер тізімін айтады. Цифрлық қол қоюды хабардың нақты жіберуші адамнан келгенін анықтау үшін пайдаланады. Құжатта цифрлық қол қою былай құрылады: құжаттан дайджесті (message digest) генерациялайды және оған қоса ақпарат қосылады, құжатқа кім қол қойғаны, штамп уақыты, және т.б көрсетіледі. Шыққан қатар құпия кілтпен шифрленеді. Осы шифрленіп алынған бит цифрлық қол болып саналады.

Криптографиялық хэш-функциялар.

Криптографиялық хэш-функцияларды көбінесе дайджест хабарлау генерациясында цифрлық қол құру кезінде қолданылады. Хэш-функциялар хабарларды жазып алынған көлемді бірнеше хэш-мәндерге (hush value) бірдей етіп бөліп тастайды.

Кездейсоқ сандардың криптографиялық генераторлары. Кездейсоқ сандардың криптографиялық генераторлары кездейсоқ сандардан құрайды және олар криптографиялық қосымшаларда қолданылады, мысалы – кілттерді генерациялау кезінде.

Шифрленген алгоритм. Шешу жұмысы қиын болу үшін жақсы криптографиялық жүйелерді құрады. Оны іске асыру үшін көп күш жұмсау қажет емес, тек ұқыптылық пен базалық білім болу керек. Теориялық негізде алатын болсақ, барлық кілт мәндерін таңдап шығу әдісімен шифрленген алгоритмді кілтті қолдану көмегімен ашады. Егер кілт белгілі бір әдіспен тандалса, онда компьютердің керек қуаты экспоненциалды өседі, сәйкесінше кілттің ұзындығы да өседі.

Бақылау сұрақтары:

1) Интернетте мәліметті жіберу кезінде қандай мәліметті қорғаумеханизмі қолданылады?
2) Жіберуші идентификациясы, жіберу аутентификациясы, құпияны сақтау мәндері қандай?
3) Симметриялық және ассиметриялық шифрлеудің мәні қандай?
4) Цифрлық қолтаңба механизмінің алгоритмін түсіндір.

ДӘРІС №11. ИНТЕРНЕТТЕ ПАЙДАЛАНУШЫЛАРДЫ ИДЕНТИФИКАЦИЯЛАУ. COOKIES МЕХАНИЗМІ

Дәрістің мақсаты

Интернет ортасында пайдаланушыларды идентификациялау тәсілдерін, Cookies механизмінің жұмыс істеу принциптерін, Cookies файлдарын Web-қосымшаларда қолдану ерекшеліктерін және Cookies негізіндегі технологиялардың артықшылықтары мен кемшіліктерін түсіндіру.

Дәрістің міндеттері

Дәріс нәтижесінде білім алушы:

- идентификация және аутентификация ұғымдарын ажырата алады;
- HTTP хаттамасының күйсіздік (stateless) ерекшелігін түсінеді;
- Cookies түсінігін және оның құрылымын біледі;
- Cookies-тің браузер мен сервер арасында қалай алмасатынын түсіндіреді;
- Session және Persistent Cookies түрлерін ажыратады;
- Secure, HttpOnly, SameSite атрибуттарының қызметін түсінеді;
- Cookies-тің Web-қосымшаларда қолданылуын талдайды;
- Cookies технологиясының қауіпсіздік және құпиялық мәселелерін анықтайды.

1. Интернетте пайдаланушыны идентификациялау

Web-қосымшалармен жұмыс істеген кезде серверге көптеген пайдаланушылар бір уақытта қосылады. Сондықтан ақпараттық жүйе белгілі бір сұраныстың қай пайдаланушыға тиесілі екенін анықтай алуы қажет.

Идентификация – ақпараттық жүйеде пайдаланушының кім екенін анықтау процесі.

Мысалы, пайдаланушы сайтқа:

- логин;
- e-mail;
- телефон нөмірі;
- пайдаланушы идентификаторы

арқылы өзін таныстыра алады.

Интернет-дүкен пайдаланушының себетіне қандай тауар салғанын, әлеуметтік желі кімнің профилін көрсету керектігін, ал білім беру порталы қай студенттің нәтижелерін көрсету керектігін білуі тиіс.

2. Идентификация, аутентификация және авторизация

Бұл үш ұғым өзара байланысты болғанымен, мағынасы әртүрлі.

Идентификация

Пайдаланушының жүйеге өзін таныстыруы.

Мысалы:

Логин: student01

Аутентификация

Пайдаланушының шынымен сол тұлға екенін тексеру процесі.

Мысалы:

Логин: student01

Пароль: ********

Жүйе енгізілген парольді тексереді.

Авторизация

Аутентификациядан өткен пайдаланушыға қандай ресурстар мен операцияларға рұқсат берілетінін анықтау.

Мысалы:

Студент → өз бағаларын көреді

Оқытушы → бағаларды енгізеді

Әкімші → пайдаланушыларды басқарады

Демек:

Идентификация

↓
«Сіз кімсіз?»
↓
Аутентификация
↓
«Бұл шынымен сіз бе?»
↓
Авторизация
↓
«Сізге не істеуге рұқсат?»

3. HTTP хаттамасының күйсіздігі

Web-браузер мен Web-сервер арасында ақпарат алмасу негізінен HTTP/HTTPS арқылы орындалады. HTTP дәстүрлі түрде **stateless**, яғни әр сұраныс жеке қарастырылатын хаттама болып табылады.
Мысалы:
1-сұраныс:
Browser → Server
GET /index.html

2-сұраныс:
Browser → Server
GET /profile.html
Сервер қосымша механизм болмаса, осы екі сұраныстың бір пайдаланушыға тиесілі екенін HTTP сұраныстарының өзінен автоматты түрде анықтай алмайды.
Осы мәселені шешуге сессия идентификаторлары және Cookies сияқты механизмдер көмектеседі.

4. Cookies дегеніміз не?

Cookie – Web-сайттың браузерге сақтауға беретін шағын мәтіндік дерегі.
Cookie ішінде, мысалы:

- сессия идентификаторы;
- интерфейс тілі;
- пайдаланушы баптаулары;
- таңдалған тақырып;
- кейбір аналитикалық идентификаторлар

сақталуы мүмкін.
Cookie серверге кейінгі HTTP сұраныстарымен бірге автоматты түрде жіберілуі мүмкін.
Мысалы:
language=kk
theme=dark
Алайда пароль немесе басқа құпия ақпаратты Cookie ішінде ашық түрде сақтау дұрыс емес.

5. Cookies механизмінің жұмыс істеу принципі

Cookies механизмі браузер мен сервердің өзара әрекеттесуіне негізделеді.

1-кезең. Пайдаланушы сайтты ашады

Пайдаланушы
↓
Браузер
↓
HTTP Request
↓
Web-сервер

2-кезең. Сервер Cookie орнатады

Сервер жауап тақырыбында:
Set-Cookie: session_id=abc123
сияқты ақпарат жібере алады.

3-кезең. Браузер Cookie-ді сақтайды

session_id = abc123

4-кезең. Келесі сұраныс

Сәйкес шарттар орындалған жағдайда браузер Cookie-ді серверге жібереді:
Cookie: session_id=abc123
Сервер session_id бойынша пайдаланушының сессиясын анықтай алады.
Жалпы механизм:
Пайдаланушы
↓
Браузер
↓
HTTP Request
↓
Web-сервер
↓
Set-Cookie
↓
Браузер
↓
Cookie сақтау
↓
Келесі HTTP Request + Cookie
↓
Web-сервер
↓
Пайдаланушы сессиясын анықтау

6. Cookie құрылымы

Cookie бірнеше параметрден тұруы мүмкін.
Мысалы:
Set-Cookie: session_id=abc123; Path=/; Secure; HttpOnly; SameSite=Lax
Мұндағы:
session_id=abc123 – Cookie атауы мен мәні.
Path – Cookie қай URL жолдарына қолданылатынын көрсетеді.
Path=/
Cookie бүкіл сайтқа қатысты болуы мүмкін.
Domain – Cookie қай доменге қатысты екенін анықтайды.
Expires – Cookie мерзімінің аяқталатын уақытын көрсетеді.
Max-Age – Cookie қанша секунд сақталуы керектігін анықтай алады.

7. Cookies түрлері

Cookies бірнеше белгі бойынша жіктеледі.

7.1 Session Cookie

Session Cookie әдетте ағымдағы браузерлік сессияға арналған.
Мысалы:
session_id=abc123
Ол пайдаланушының жүйедегі сессиясын байланыстыру үшін қолданылуы мүмкін.

7.2 Persistent Cookie

Persistent Cookie-ге сақтау мерзімі беріледі.

Мысалы:
Set-Cookie: language=kk; Max-Age=2592000
Мұндай Cookie браузер жабылғаннан кейін де белгіленген мерзімге дейін сақталуы мүмкін.

7.3 First-party Cookie

Пайдаланушы кірген сайттың өз домені контекстінде орнатылатын Cookie.
Мысалы, пайдаланушы:
example.kz
сайтына кірсе, сол сайт пайдаланушының тіл параметрін сақтауы мүмкін.

7.4 Third-party Cookie

Пайдаланушы ашқан негізгі сайттан басқа домен контекстімен байланысты Cookie. Тарихи тұрғыдан мұндай Cookies жарнама және сайттар арасындағы бақылау технологияларында кең қолданылды. Қазіргі браузерлер мұндай механизмдерге елеулі шектеулер енгізеді.

8. Cookies-ті қолдану

Cookies Web-қосымшаларда әртүрлі міндеттерді шешеді.

Пайдаланушы сессиясы

Пайдаланушы жүйеге кіргеннен кейін сервер сессия идентификаторын Cookie арқылы байланыстыра алады.
Login
↓
Server
↓
Session құру
↓
Session ID
↓
Cookie
↓
Browser

Тілді сақтау

Пайдаланушы қазақ тілін таңдады:
language=kk
Келесі кіргенде сайт қазақ тілінде ашылуы мүмкін.

Интерфейс параметрлері

Мысалы:
theme=dark
сайттың қараңғы режимін сақтау үшін қолданылуы мүмкін.

Электрондық сауда

Cookie немесе серверлік сессия пайдаланушының сауда себетін байланыстыруға көмектеседі.

9. JavaScript арқылы Cookies-пен жұмыс

JavaScript-те JavaScript қолжетімді Cookies-пен document.cookie арқылы жұмыс істеуге болады.
Cookie орнату:

```
document.cookie = "language=kk; path=/";
```

Cookie оқу:

```
console.log(document.cookie);
```

Мерзім беру мысалы:

```
document.cookie =
  "theme=dark; max-age=86400; path=/";
```

Мұндағы 86400 – секундпен есептелген бір тәулік.
Бірақ қауіпсіздік тұрғысынан сессия сияқты маңызды Cookies-ті JavaScript арқылы басқару әрдайым дұрыс емес. Сервер Cookie-ге HttpOnly атрибутын бере алады.

10. Cookies және пайдаланушы сессиясы

Cookie мен Session ұғымдарын шатастырмау қажет.
Cookie – браузер жағындағы механизм.
Session – көбінесе сервер жағында пайдаланушының күйін сақтауға арналған механизм.
Мысалы:
БРАУЗЕР

Cookie:
session_id = X82ABC

↓

WEB-СЕРВЕР

Session X82ABC:
user_id = 152
role = student
language = kk
Яғни браузерде пайдаланушы туралы барлық ақпаратты сақтаудың орнына кездейсоқ және болжануы қиын сессия идентификаторы сақталып, негізгі деректер сервер жағында орналасуы мүмкін.

11. Cookies қауіпсіздігі

Cookies дұрыс конфигурацияланбаса, қауіпсіздік тәуекелдерін арттыруы мүмкін.

Secure

Secure
атрибуты Cookie-дің қауіпсіз HTTPS байланысы арқылы жіберілуін талап етеді.

HttpOnly

HttpOnly
Cookie-ге клиенттік JavaScript арқылы қол жеткізуді шектейді. Бұл белгілі бір Cookie мәнін XSS арқылы оқып алуға қарсы қосымша қорғаныс береді.

SameSite

SameSite Cookie-дің сайттар арасындағы сұраныстарда жіберілу тәртібін басқаруға көмектеседі.
Негізгі мәндері:
Strict
Lax
None
SameSite дұрыс конфигурациялануы CSRF сияқты шабуылдардың тәуекелін төмендетуге көмектеседі.

12. Cookies негізіндегі технологиялардың кемшіліктері

Cookies пайдалы болғанымен, бірқатар кемшіліктері бар.

1. Сақтау көлемінің шектеулі болуы

Cookie үлкен көлемдегі ақпаратты сақтауға арналмаған.

2. HTTP трафигіне әсері

Сәйкес Cookie әрбір тиісті HTTP сұранысына қосылып жіберілуі мүмкін. Cookies шамадан тыс көп болса, артық трафик пайда болады.

3. Қауіпсіздік мәселелері

Қате конфигурацияланған Cookie:

- сессияны ұрлау;
- XSS салдары;

- CSRF;
- пайдаланушы сессиясын заңсыз пайдалану

сияқты қауіптермен байланысты болуы мүмкін.

4. Құпиялық мәселелері

Cookies және оларға ұқсас идентификаторлар пайдаланушы әрекетін талдау немесе бақылау үшін пайдаланылуы мүмкін. Сондықтан жеке деректер мен келісімге қатысты талаптарды ескеру қажет.

5. Пайдаланушы Cookie-ді өшіре алады

Браузер баптаулары арқылы Cookies жойылуы немесе белгілі Cookies түрлеріне тыйым салынуы мүмкін.

6. Cookies ескіруі мүмкін

Cookie мәндері өзгерген жүйелік күйге сәйкес келмей қалуы мүмкін. Сондықтан сервер Cookie мәніне соқыр сенбеуі тиіс.

13. Cookie, LocalStorage және SessionStorage салыстыру

Сипаттама	Cookie	LocalStorage	SessionStorage
Клиентте сақтау	Иә	Иә	Иә
HTTP сұранысымен автоматты жіберілуі	Иә, шарттары сәйкес болса	Жоқ	Жоқ
JavaScript арқылы қолжетімділік	HttpOnly болмаса	Иә	Иә
Сақталу сипаты	Атрибуттарына байланысты	Ұзақ мерзімді	Қойынды сессиясына тәуелді
Негізгі қолданылуы	Сессия, баптаулар	Клиенттік деректер	Уақытша клиенттік деректер

Сондықтан барлық ақпаратты Cookies ішінде сақтау міндетті емес. Қандай сақтау механизмі қолданылатыны Web-қосымшаның мақсаты мен қауіпсіздік талаптарына байланысты таңдалады.

14. Практикалық мысал

Қарапайым HTML парағында пайдаланушының таңдаған тілін Cookie-де сақтауға болады:

```
<!DOCTYPE html>
<html lang="kk">
<head>
   <meta charset="UTF-8">
   <title>Cookies мысалы</title>
</head>

<body>

<h2>Тілді таңдаңыз</h2>

<button onclick="setLanguage('kk')">
   Қазақша
</button>

<button onclick="setLanguage('ru')">
   Русский
</button>

<p id="result"></p>

<script>
```

```
function setLanguage(lang) {
   document.cookie =
      "language=" + lang +
      "; max-age=2592000; path=/";

   document.getElementById("result").innerText =
      "Таңдалған тіл: " + lang;
}
</script>

</body>
</html>
```

Бұл мысалда пайдаланушы батырманы басқан кезде language атты Cookie жасалады.

15. Cookies-ті браузер арқылы зерттеу

Студент Cookies механизмін браузердің Developer Tools құралымен көре алады.
Chrome немесе Edge браузерінде:
F12 → Application → Storage → Cookies
бөліміне өтуге болады.
Онда Cookie-дің:

- Name;
- Value;
- Domain;
- Path;
- Expires/Max-Age;
- Size;
- HttpOnly;
- Secure;
- SameSite

сияқты параметрлерін көруге болады.

16. Cookies механизмінің артықшылықтары мен кемшіліктері

Артықшылықтары	Кемшіліктері
Пайдаланушы сессиясын байланыстыруға мүмкіндік береді	Сақтау көлемі шектеулі
Пайдаланушы баптауларын есте сақтайды	Құпиялық мәселелері болуы мүмкін
Сервермен автоматты алмасуы мүмкін	Сұраныс көлемін арттыруы мүмкін
Web-сайттың дербестендірілуіне көмектеседі	Дұрыс қорғалмаса қауіпсіздік тәуекелі бар
Қолданылуы салыстырмалы қарапайым	Пайдаланушы өшіре немесе шектей алады

Қорытынды

Cookies – Web-технологиялардағы пайдаланушының сессиясын және белгілі бір күйді байланыстырудың маңызды механизмдерінің бірі. HTTP сұраныстары өздігінен алдыңғы сұраныстың контекстін сақтамайтындықтан, Cookies серверге белгілі бір браузер сессиясын тануға көмектеседі.
Cookies авторизация сессияларын қолдау, тіл мен интерфейс параметрлерін есте сақтау, пайдаланушы тәжірибесін дербестендіру сияқты міндеттерде қолданылады. Сонымен бірге Cookies-ті қолданғанда қауіпсіздік пен құпиялық мәселелерін міндетті түрде ескеру керек. Маңызды Cookies үшін Secure, HttpOnly және тиісті SameSite параметрлерін қолдану Web-қосымшаның қорғанысын күшейтеді.

Бақылау сұрақтары

1. Интернетте пайдаланушыны идентификациялау дегеніміз не?
2. Идентификация мен аутентификацияның айырмашылығы қандай?
3. Авторизация дегеніміз не?
4. HTTP неге stateless хаттама деп аталады?
5. Cookie дегеніміз не?
6. Cookies қандай мақсаттарда қолданылады?
7. Cookie браузерге қалай беріледі?
8. Set-Cookie не үшін қолданылады?
9. Session Cookie дегеніміз не?
10. Persistent Cookie дегеніміз не?
11. First-party және third-party Cookies айырмашылығы қандай?
12. Secure атрибутының қызметі қандай?
13. HttpOnly не үшін қолданылады?
14. SameSite атрибутының қызметі қандай?
15. Cookie мен Session арасындағы айырмашылық қандай?
16. Cookies-тің қандай кемшіліктері бар?
17. Cookies қандай қауіпсіздік мәселелерін тудыруы мүмкін?
18. Cookie, LocalStorage және SessionStorage арасында қандай айырмашылық бар?
19. Cookies браузерден қалай көруге болады?
20. Неліктен парольді Cookie ішінде ашық түрде сақтауға болмайды?

ДӘРІС №12. HTML ТІЛІНІҢ МУЛЬТИМЕДИЯЛЫҚ МҮМКІНДІКТЕРІ. ФРЕЙМДЕР ЖӘНЕ HTML ФОРМАЛАРЫ

Дәрістің мақсаты

HTML тілінде бір Web-параққа бірнеше құжатты орналастыру, мультимедиалық ақпараттарды енгізу және пайдаланушымен өзара әрекеттесуге арналған HTML формаларын құру принциптерін меңгеру.

Дәрістің міндеттері

- iframe элементінің қызметін түсіндіру;
- Web-параққа сурет, аудио және видео орналастыру;
- HTML формаларының құрылымын меңгеру;
- input, textarea, select, button элементтерімен жұмыс істеу;
- GET және POST әдістерінің ерекшеліктерін түсіндіру.

1. HTML тіліндегі мультимедиа

Қазіргі Web-сайттар тек мәтіндік ақпараттан тұрмайды. Оларда:

- графикалық кескіндер;
- дыбыс;
- бейне;
- анимация;
- карталар;
- сыртқы Web-құжаттар

кеңінен қолданылады.

HTML5 мультимедиалық файлдарды браузерде қосымша плагиндерсіз көрсетуге мүмкіндік береді.

2. Фреймдер туралы түсінік

Фрейм – Web-парақ ішінде басқа HTML құжатын немесе сыртқы Web-ресурсты көрсету тәсілі.

Ескі HTML нұсқаларында frameset және frame элементтері пайдаланылған. Қазіргі HTML5 стандартында олар ескірген. Оның орнына iframe қолданылады.

iframe — inline frame, яғни кірістірілген фрейм.

3. iframe элементі

Мысалы:

```
<iframe
   src="page2.html"
   width="600"
   height="400">
</iframe>
```

Мұндағы:

- src – көрсетілетін құжат;
- width – ені;
- height – биіктігі.

Сыртқы Web-ресурсты да көрсетуге болады, бірақ сайттың қауіпсіздік саясаты мұндай кірістіруге рұқсат беруі керек:

```
<iframe
   src="https://example.com"
   width="800"
   height="500">
</iframe>
```

iframe карталар, бейнелер және кейбір сыртқы сервистерді енгізу үшін де қолданылады.

4. Web-параққа сурет орналастыру

HTML-де сурет орналастыру үшін <img> элементі пайдаланылады:

```
<img
   src="images/photo.jpg"
```

```
   alt="Университет ғимараты"
   width="500">
```
Негізгі атрибуттар:

Атрибут	Қызметі
src	суреттің мекенжайы
alt	суреттің мәтіндік сипаттамасы
width	ені
height	биіктігі

alt атрибуты қолжетімділік үшін маңызды.

5. HTML5 және аудио

Web-параққа дыбыс орналастыру үшін:

```
<audio controls>
   <source src="audio/music.mp3"
         type="audio/mpeg">
</audio>
```

controls браузерде басқару элементтерін көрсетеді.
Қосымша атрибуттар:

- autoplay;
- loop;
- muted;
- preload.

6. HTML5 және видео

Видео орналастыру:

```
<video width="640" controls>
   <source src="video/lecture.mp4"
         type="video/mp4">
</video>
```

Бейненің бастапқы суретін көрсетуге болады:

```
<video
   controls
   poster="images/poster.jpg">
   <source src="video.mp4"
         type="video/mp4">
</video>
```

7. HTML формалары

HTML форма – пайдаланушыдан ақпарат алуға арналған Web-интерфейс.
Формалар:

- тіркелу;
- жүйеге кіру;
- іздеу;
- тапсырыс беру;
- сауалнама;
- кері байланыс

сияқты қызметтерде қолданылады.
Форманың негізгі құрылымы:

```
<form action="result.php" method="post">

   <label>Аты:</label>
```

```
    <input type="text" name="name">

    <button type="submit">
        Жіберу
    </button>

</form>
```

8. input элементі

input – HTML формаларындағы негізгі элементтердің бірі.

Мәтін енгізу

```
<input type="text" name="username">
```

Пароль

```
<input type="password" name="password">
```

Электрондық пошта

```
<input type="email" name="email">
```

Сан

```
<input type="number" name="age">
```

Күн

```
<input type="date" name="birthday">
```

Radio

```
<input type="radio" name="course" value="1">
1 курс

<input type="radio" name="course" value="2">
2 курс
```

Checkbox

```
<input type="checkbox" name="html">
HTML

<input type="checkbox" name="css">
CSS
```

9. textarea және select

Үлкен мәтін:

```
<textarea
    name="message"
    rows="5"
    cols="40">
</textarea>
```

Таңдау тізімі:

```
<select name="language">
    <option value="kk">Қазақша</option>
    <option value="ru">Орысша</option>
    <option value="en">English</option>
</select>
```

10. GET және POST

HTML формасы ақпаратты серверге көбінесе GET немесе POST әдісімен жібереді.

GET	**POST**
Параметрлер URL-ге қосылуы мүмкін	Деректер сұраныс денесінде жіберіледі

GET	POST
Іздеу сияқты сұраныстарға ыңғайлы	Формаларды өңдеуге жиі қолданылады
URL bookmark жасауға қолайлы	Деректер URL-де көрінбейді
Көлемдік/семантикалық шектеулері бар	Үлкенірек сұраныстарға қолайлы

POST өздігінен шифрлау емес. Құпия ақпарат тасымалданғанда HTTPS қажет.

Қорытынды

HTML5 мультимедиа элементтері Web-парақтарға сурет, аудио және бейне енгізуге мүмкіндік береді. HTML формалары пайдаланушыдан ақпарат қабылдаудың негізгі құралы болып табылады.

Бақылау сұрақтары

1. Фрейм дегеніміз не?
2. iframe қандай қызмет атқарады?
3. img элементіндегі alt не үшін қажет?
4. HTML-де аудио қалай орналастырылады?
5. video элементінің қызметі қандай?
6. HTML форма дегеніміз не?
7. action атрибуты не үшін қажет?
8. method не көрсетеді?
9. input қандай қызмет атқарады?
10. GET және POST айырмашылығы қандай?

ДӘРІС №13. СТИЛЬДЕРДІҢ САТЫЛЫ КЕСТЕЛЕРІ (CSS)

Дәрістің мақсаты

CSS технологиясының Web-парақтарды безендірудегі қызметін меңгеру, стильдерді HTML құжатына қосудың тәсілдерін, CSS селекторларын, Box Model және элементтерді позициялауды үйрену.

Дәріс жоспары

1. CSS түсінігі.
2. CSS синтаксисі.
3. CSS селекторлары.
4. Inline CSS.
5. Internal CSS.
6. External CSS.
7. Мәтінді форматтау.
8. Фон және шекара.
9. CSS Box Model.
10. Элементтердің өлшемдері.
11. position.
12. z-index.

1. CSS дегеніміз не?

CSS (Cascading Style Sheets) – HTML элементтерінің сыртқы көрінісін анықтайтын стильдер тілі.

HTML:

```
<h1>Web бағдарламалау</h1>
```

CSS:

```
h1 {
   color: blue;
   font-size: 36px;
}
```

HTML мазмұн мен құрылымды, ал CSS оның визуалды көрсетілуін сипаттайды.

2. CSS синтаксисі

Жалпы құрылымы:

```
selector {
   property: value;
}
```

Мысалы:

```
p {
   color: black;
   font-size: 18px;
}
```

Мұнда:

- p – селектор;
- color – қасиет;
- black – мән.

3. CSS селекторлары

Тег селекторы

```
p {
   color: black;
}
```

Class селекторы

HTML:
```
<p class="info">Ақпарат</p>
```
CSS:
```
.info {
   font-size: 20px;
}
```
ID селекторы
```
<h1 id="title">Web</h1>
#title {
   text-align: center;
}
```

4. Inline CSS

Стиль тікелей тегке жазылады:
```
<p style="color: red;">
   Web бағдарламалау
</p>
```
Бұл тәсіл шағын ерекше жағдайларға жарағанымен, үлкен жобада қолдау қиын.

5. Internal CSS

HTML құжатының <head> бөлімінде:
```
<style>
   body {
      font-family: Arial, sans-serif;
   }

   h1 {
      text-align: center;
   }
</style>
```
Бұл стиль бір HTML құжатына қолданылады.

6. External CSS

Жеке файл:
style.css
HTML-ге қосу:
```
<link rel="stylesheet" href="style.css">
```
style.css:
```
body {
   margin: 0;
   font-family: Arial, sans-serif;
}
```
Бір CSS файлын көптеген HTML парақтарына қосуға болады.

7. Мәтінді форматтау

Негізгі қасиеттер:
```
p {
   color: #333;
   font-family: Arial, sans-serif;
   font-size: 18px;
   font-weight: normal;
   text-align: justify;
```

```
  line-height: 1.6;
}
```

8. Фон және шекара

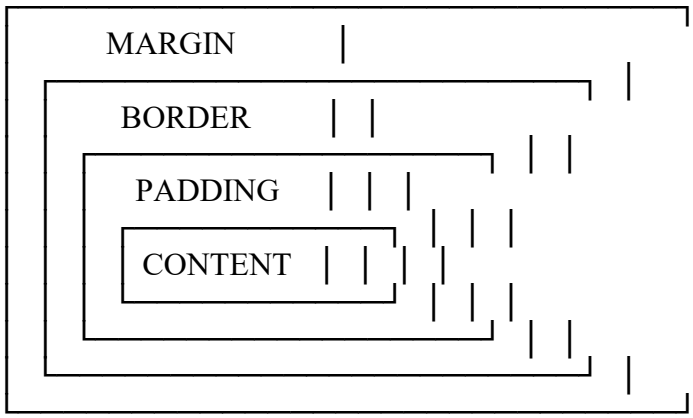
```
.card {
  background-color: white;
  border: 1px solid #ccc;
  border-radius: 10px;
}
```

Фондық сурет:

```
body {
  background-image: url("background.jpg");
  background-size: cover;
}
```

9. CSS Box Model

Әр HTML элементін шартты түрде қорап ретінде қарастыруға болады:



Content – мазмұн.
Padding – мазмұн мен шекара аралығы.
Border – шекара.
Margin – элементтің сыртқы аралығы.

Мысалы:

```
.card {
  width: 300px;
  padding: 20px;
  border: 1px solid black;
  margin: 30px;
}
```

10. Элементтерді позициялау

CSS position қасиетінің негізгі мәндері:

```
position: static;
position: relative;
position: absolute;
position: fixed;
position: sticky;
```

relative

```
.box {
  position: relative;
  left: 20px;
```

```
  top: 10px;
}
```

absolute

```
.box {
  position: absolute;
  top: 50px;
  left: 100px;
}
```

fixed

```
.menu {
  position: fixed;
  top: 0;
}
```

Элемент экранға қатысты бекітіледі.

11. z-index

z-index қабаттасқан элементтердің қайсысы үстінде орналасатынын басқаруға көмектеседі.

```
.menu {
  position: fixed;
  z-index: 1000;
}
```

Жалпы жағдайда үлкен z-index мәні жоғары қабатқа басымдық береді, бірақ нақты нәтиже stacking context ережелеріне де тәуелді.

Қорытынды

CSS HTML құжатының сыртқы көрінісін басқаруға мүмкіндік береді. External CSS үлкен Web-жобаларда стильдерді орталықтандырылған түрде басқаруға ыңғайлы.

Бақылау сұрақтары

1. CSS дегеніміз не?
2. CSS селекторы дегеніміз не?
3. Class пен ID айырмашылығы қандай?
4. Inline CSS дегеніміз не?
5. Internal CSS қалай қосылады?
6. External CSS артықшылығы қандай?
7. Box Model дегеніміз не?
8. margin және padding айырмашылығы қандай?
9. position қандай мәндер қабылдайды?
10. z-index не үшін қолданылады?

ДӘРІС №14. JAVASCRIPT ТІЛІ

Дәрістің мақсаты

JavaScript тілінің негізгі синтаксисін, айнымалыларды, операторларды, шарттарды, циклдерді, функцияларды, объектілерді және браузер оқиғаларын меңгеру.

Дәріс жоспары

1. JavaScript туралы жалпы мәлімет.
2. HTML құжатына JavaScript қосу.
3. Литералдар.
4. Айнымалылар.
5. Деректер типтері.
6. Операторлар және өрнектер.
7. Шартты операторлар.
8. Циклдер.
9. Функциялар.
10. Массивтер.
11. Объектілер.
12. Кластар және мұралау.
13. DOM және браузер объектілері.
14. Оқиғалар.

1. JavaScript дегеніміз не?

JavaScript – Web-парақтарға интерактивтілік қосуға арналған бағдарламалау тілі.
Оның көмегімен:

- батырмаларды өңдеуге;
- формаларды тексеруге;
- HTML мазмұнын өзгертуге;
- серверден деректер алуға;
- анимациялар жасауға;
- интерактивті Web-қосымшалар құруға

болады.

2. HTML құжатына JavaScript қосу

Ішкі сценарий:

```
<script>
   console.log("Hello JavaScript!");
</script>
```

Сыртқы файл:

```
<script src="script.js"></script>
```

3. Айнымалылар

Қазіргі JavaScript-те негізінен let және const қолданылады.

```
let studentName = "Aruzhan";
let age = 20;

const university = "University";
```

let – мәні кейін өзгеруі мүмкін айнымалы.
const – қайта тағайындалмайтын байланыс үшін қолданылады.

4. Деректер типтері

Мысалдар:

```
let name = "Aruzhan";     // string
```

```
let age = 20;             // number
let student = true;       // boolean
let result = null;        // null
let value;                // undefined
```
JavaScript-те объектілер де негізгі рөл атқарады.

5. Операторлар

Арифметикалық:
```
+ - * / % **
```
Салыстыру:
```
=== !== > < >= <=
```
Логикалық:
```
&&
||
!
```
Мысалы:
```
let age = 20;

if (age >= 18) {
   console.log("Кәмелетке толған");
}
```

6. Шартты операторлар

```
let score = 85;

if (score >= 90) {
   console.log("Өте жақсы");
} else if (score >= 75) {
   console.log("Жақсы");
} else if (score >= 50) {
   console.log("Қанағаттанарлық");
} else {
   console.log("Қанағаттанарлықсыз");
}
```

7. Циклдер

for

```
for (let i = 1; i <= 5; i++) {
   console.log(i);
}
```

while

```
let i = 1;

while (i <= 5) {
   console.log(i);
   i++;
}
```

8. Функциялар

Функция – белгілі бір міндетті орындайтын қайта пайдаланылатын код блогы.
```
function sum(a, b) {
   return a + b;
```

```
}

console.log(sum(10, 20));
```
Arrow function:
```
const sum = (a, b) => a + b;
```

9. Массивтер

```
let students = [
   "Aruzhan",
   "Dana",
   "Aibek"
];

console.log(students[0]);
```
Циклмен шығару:
```
for (let student of students) {
   console.log(student);
}
```

10. Объектілер

```
const student = {
   name: "Aruzhan",
   age: 20,
   course: 2
};

console.log(student.name);
```
Әдіс:
```
const student = {
   name: "Aruzhan",

   showName() {
      console.log(this.name);
   }
};

student.showName();
```

11. Кластар және мұралау

```
class Person {
   constructor(name) {
      this.name = name;
   }

   showName() {
      console.log(this.name);
   }
}

class Student extends Person {
   constructor(name, course) {
      super(name);
```

```
        this.course = course;
    }
}
```
extends арқылы бір класс екінші кластың функционалдығын мұралай алады.

12. DOM

DOM (Document Object Model) HTML құжатын бағдарламалық объектілер ағашы ретінде ұсынады.
HTML:
```
<h1 id="title">
    Web бағдарламалау
</h1>
```
JavaScript:
```
const title =
    document.getElementById("title");

title.textContent =
    "JavaScript сабағы";
```

13. Браузер объектілері

JavaScript браузер ұсынатын көптеген API және объектілермен жұмыс істейді.
Мысалы:
```
console.log(window.location.href);
console.log(navigator.userAgent);
```
Кең таралған объектілер:

- window;
- document;
- location;
- history;
- navigator.

14. Оқиғалар

Web-парақтағы пайдаланушы әрекеті **оқиға (event)** деп аталады.
Мысалы:

- click;
- change;
- input;
- submit;
- keydown;
- mouseover.

```
<button id="btn">
    Батырманы басыңыз
</button>

<p id="result"></p>

<script>
const btn = document.getElementById("btn");

btn.addEventListener("click", () => {
    document.getElementById("result")
        .textContent = "Батырма басылды!";
});
```

</script>

Қорытынды

JavaScript HTML және CSS-пен бірге Web-бағдарламалаудың негізгі технологияларының бірі. HTML құрылымды, CSS дизайнды, ал JavaScript Web-парақтың интерактивті әрекетін қамтамасыз етеді.

Бақылау сұрақтары

1. JavaScript дегеніміз не?
2. JavaScript HTML-ге қалай қосылады?
3. let және const айырмашылығы қандай?
4. Литерал дегеніміз не?
5. JavaScript деректер типтерін атаңыз.
6. if операторы не үшін қолданылады?
7. Цикл дегеніміз не?
8. Функцияның қызметі қандай?
9. Массив дегеніміз не?
10. Объект дегеніміз не?
11. Мұралау дегеніміз не?
12. DOM дегеніміз не?
13. Браузер объектілерін атаңыз.
14. Оқиға дегеніміз не?
15. addEventListener() не үшін қолданылады?

ДӘРІС №15. WEB-ҚОСЫМШАЛАР ИНТЕРФЕЙСІ. HTTP ХАТТАМАСЫ. PERL ЖӘНЕ PHP ТІЛДЕРІНІҢ НЕГІЗДЕРІ

Дәрістің мақсаты

Web-қосымшалардың клиент-серверлік архитектурасын, HTTP хаттамасының жұмыс механизмін, серверлік бағдарламалау түсінігін және PHP тілінің негізгі синтаксисін меңгеру.

Дәріс жоспары

1. Web-қосымша түсінігі.
2. Клиент-сервер архитектурасы.
3. HTTP хаттамасы.
4. HTTP Request.
5. HTTP Response.
6. HTTP әдістері.
7. HTTP күй кодтары.
8. Серверлік бағдарламалау.
9. Perl тіліне кіріспе.
10. PHP тілі.
11. PHP синтаксисі.
12. Айнымалылар.
13. Операторлар.
14. Шарттар.
15. Циклдер.
16. Функциялар.
17. HTML формаларын PHP арқылы өңдеу.

1. Web-қосымша

Web-қосымша – браузер арқылы пайдаланылатын бағдарламалық жүйе.

Мысалдар:

- электрондық дүкен;
- білім беру порталы;
- интернет-банкинг;
- электрондық кітапхана;
- әлеуметтік желі;
- ақпараттық жүйе.

2. Клиент-сервер архитектурасы

Web-қосымшаның жалпы схемасы:

Пайдаланушы
↓
Web-браузер
↓
HTTP/HTTPS Request
↓
Web-сервер
↓
Серверлік бағдарлама
↓
Деректер қоры
↓
HTTP/HTTPS Response
↓
Web-браузер

Клиент – серверге сұраныс жіберетін бағдарлама.
Сервер – сұранысты қабылдап, өңдеп, жауап қайтаратын жүйе.

3. HTTP хаттамасы

HTTP (Hypertext Transfer Protocol) – Web ортасында клиент пен сервер арасындағы ақпарат алмасудың қолданбалы деңгейдегі хаттамасы.
HTTPS – HTTP байланысын TLS арқылы қорғалған арнада жүзеге асырады.

4. HTTP Request

Мысал:
GET /index.html HTTP/1.1
Host: example.com
HTTP сұранысында:

- әдіс;
- ресурс;
- HTTP нұсқасы;
- тақырыптар;
- қажет болған жағдайда сұраныс денесі болады.

5. HTTP Response

Мысал:
HTTP/1.1 200 OK
Content-Type: text/html
Одан кейін жауаптың денесі беріледі.

6. HTTP әдістері

GET

Ресурсты алу үшін қолданылады.
GET /students

POST

Серверге деректер жіберу немесе өңдеу операциясын бастау үшін жиі қолданылады.
POST /students
Басқа әдістер:

- PUT;
- PATCH;
- DELETE;
- HEAD;
- OPTIONS.

7. HTTP күй кодтары

Код	Мағынасы
200	OK
201	Created
301	Moved Permanently
302	Found
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

Код Мағынасы

503 Service Unavailable

Күй кодтары сервердің сұранысты өңдеу нәтижесін клиентке хабарлайды.

8. Серверлік бағдарламалау

Клиенттік код браузерде орындалса, серверлік код Web-сервер жағында орындалады.
Серверлік бағдарламалау:

- формаларды өңдеу;
- пайдаланушыны аутентификациялау;
- деректер қорымен жұмыс;
- есептеулер;
- API құру;
- динамикалық HTML немесе JSON жауаптарын қалыптастыру

үшін пайдаланылады.

9. Perl тіліне кіріспе

Perl – жалпы мақсаттағы бағдарламалау тілі. Web дамуының ертерек кезеңдерінде CGI сценарийлерін құруда кең қолданылды.
Қарапайым мысал:
print "Hello, Web!";
CGI арқылы сервер сұранысты сценарийге беріп, сценарий жауап қалыптастыра алады.
Қазіргі Web-жобаларда басқа серверлік платформалар жиі қолданылғанымен, Perl және CGI Web бағдарламалаудың даму тарихын түсіну үшін маңызды.

10. PHP тілі

PHP – сервер жағында орындалатын Web-бағдарламалау тілі.
PHP коды:

```
<?php
echo "Сәлем, Web!";
?>
```

Сервер PHP кодын орындайды және клиентке оның нәтижесін жібереді.

11. PHP синтаксисі

PHP блогы:

```
<?php

// PHP коды

?>
```

Экранға ақпарат шығару:

```
<?php
echo "Web бағдарламалау";
?>
```

12. PHP айнымалылары

PHP айнымалысы $ таңбасынан басталады:

```
$name = "Aruzhan";
$age = 20;
$score = 95;
```

Шығару:

```
echo $name;
```

13. PHP операторлары

Арифметикалық:
+ - * / %
Салыстыру:
== === != > < >= <=
Логикалық:
&&
||
!

14. Шартты оператор

```
$score = 85;

if ($score >= 90) {
   echo "Өте жақсы";
} elseif ($score >= 75) {
   echo "Жақсы";
} elseif ($score >= 50) {
   echo "Қанағаттанарлық";
} else {
   echo "Қанағаттанарлықсыз";
}
```

15. Циклдер

for

```
for ($i = 1; $i <= 5; $i++) {
   echo $i . "<br>";
}
```

while

```
$i = 1;

while ($i <= 5) {
   echo $i . "<br>";
   $i++;
}
```

foreach

```
$students = ["Aruzhan", "Dana", "Aibek"];

foreach ($students as $student) {
   echo $student . "<br>";
}
```

16. PHP функциялары

```
function sum($a, $b) {
   return $a + $b;
}

echo sum(10, 20);
```

17. HTML формасы мен PHP байланысы

index.html:

```
<form action="result.php" method="post">

   <label>Аты-жөні:</label>

   <input
      type="text"
      name="student"
      required>

   <button type="submit">
      Жіберу
   </button>

</form>
```
result.php:
```
<?php

$student = $_POST["student"] ?? "";

if ($student !== "") {

   $safeStudent =
      htmlspecialchars(
         $student,
         ENT_QUOTES,
         "UTF-8"
      );

   echo "Қош келдіңіз, " . $safeStudent;

} else {

   echo "Аты-жөніңіз енгізілмеген.";

}

?>
```
Мұнда браузердегі формадағы ақпарат POST сұранысы арқылы серверге беріледі.

18. PHP және деректер қоры

Нақты Web-қосымшаларда PHP серверлік деректер қорымен байланыса алады.
Жалпы архитектура:
HTML форма
↓
HTTP POST
↓
PHP
↓
Деректерді тексеру
↓
Деректер қоры
↓

PHP
↓
HTTP Response
↓
Пайдаланушы
Мысалы, студенттерді тіркеу жүйесінде:
Student
---
id
name
email
speciality
course
сияқты кесте қолданылуы мүмкін.

19. Клиенттік және серверлік бағдарламалау айырмашылығы

Клиенттік бөлік	**Серверлік бөлік**
Браузерде орындалады	Серверде орындалады
HTML	PHP
CSS	Perl және басқа серверлік тілдер
JavaScript	Серверлік бағдарламалық логика
Интерфейс	Деректерді өңдеу
DOM-пен жұмыс	Деректер қорымен жұмыс
Пайдаланушымен әрекеттесу	Аутентификация, бизнес-логика

Нақты Web-қосымшада JavaScript те серверлік ортада орындала алады, бірақ бұл дәрісте классикалық браузерлік JavaScript қарастырылып отыр.

Осылайша, HTML, CSS, JavaScript және PHP жеке-жеке емес, бір Web-қосымшаның әртүрлі деңгейлерін құрайды.

Бақылау сұрақтары

1. Web-қосымша дегеніміз не?
2. Клиент-сервер архитектурасы қалай жұмыс істейді?
3. HTTP дегеніміз не?
4. HTTP және HTTPS айырмашылығы қандай?
5. HTTP Request дегеніміз не?
6. HTTP Response дегеніміз не?
7. GET әдісі не үшін қолданылады?
8. POST әдісінің қызметі қандай?
9. 200 коды нені білдіреді?
10. 404 қатесі нені білдіреді?
11. 500 коды қандай қатені көрсетеді?
12. Серверлік бағдарламалау дегеніміз не?
13. CGI дегеніміз не?
14. Perl тілінің Web тарихындағы рөлі қандай?
15. PHP дегеніміз не?
16. PHP айнымалысы қалай жарияланады?
17. PHP-де шартты оператор қалай жазылады?
18. foreach қандай жағдайда қолданылады?
19. HTML формасындағы деректер PHP-ге қалай жіберіледі?

· ҚАЗАҚСТАН РЕСПУБЛИКАСЫ АУЫЛ ШАРУАШЫЛЫҒЫ МИНИСТРЛІГІ
«С.СЕЙФУЛЛИН атындағы ҚАЗАҚ АГРОТЕХНИКАЛЫҚ ЗЕРТТЕУ УНИВЕРСИТЕТІ» КеАҚ

«WEB БАҒДАРЛАМАЛАУ»

ЗЕРТХАНАЛЫҚ ЖҰМЫСТАР ЖИНАҒЫ

Астана, 2026ж

Зертханалық жұмысқа әдістемелік нұсқау

Сіздің **зертханалық есебіңіз** кемінде келесі тақырыптарды қамтуы тиіс:

A. **Зертханалық жұмыс нөмірі**
B. **Мақсаттары**
C. **Тапсырманың қойылымы**
D. **Кодтау (бастапқы код)**
E. **Нәтиже (компиляция, жөндеу және тестілеу)**
F. **Талқылау және қорытынды**

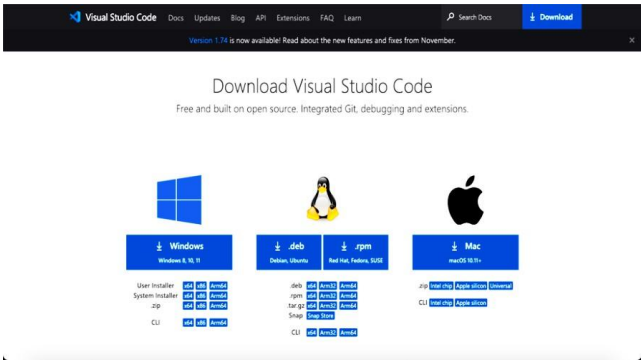
1. Тізімде көрсетілген барлық **зертханалық жұмыстарды** орындау керек.
2. Әрбір бағдарламада кодтағы әрбір функцияның, соның ішінде негізгі функцияның үстіне түсініктемелер қосылуы тиіс.
3. Негізгі кодтың үстіндегі түсініктеме блогында бағдарламаның мақсаты сипатталуы тиіс. Кодта қажет болған жерде және қажет уақытта тиісті түсініктемелер берілуі керек.
4. Код интерактивті, жалпыға қолданыстағы және дұрыс құжатталған болуы, сондай-ақ **нәтиже (Output)** деректерін қамтуы тиіс.

ЗЕРТХАНАЛЫҚ ЖҰМЫС №1.
ВЕБ-БАҒДАРЛАМАЛАУҒА КІРІСПЕ

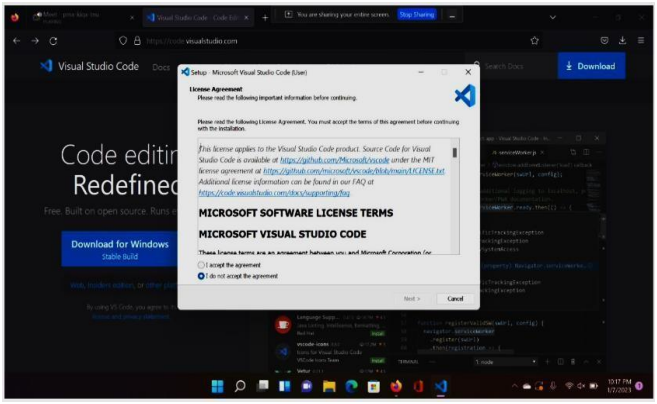
Жұмыстың атауы: Visual Studio Code бағдарламасын орнату.
Теориялық бөлім: Visual Studio Code (VS Code) - Microsoft компаниясы әзірлеген бастапқы код редакторы. Ол **Windows, Linux және macOS** операциялық жүйелерінде жұмыс істейді.
VS Code бағдарламасы бағдарламалық код жазуға және өңдеуге арналған. Оның негізгі мүмкіндіктері:

- кодты синтаксистік ерекшеліктеріне қарай бояу;
- кодты автоматты түрде толықтыру;
- кодты жөндеу (**Debugging**);
- кодты қайта құрылымдау (**Refactoring**);
- дайын код үзінділерін пайдалану (**Snippets**);
- **Git** жүйесімен жұмыс істеу.

VS Code - веб-бағдарламалау кезінде **HTML, CSS, JavaScript, PHP** және басқа да бағдарламалау тілдерінде код жазуға ыңғайлы құрал.
Жұмысты орындау тәртібі
1-қадам. Visual Studio Code бағдарламасын жүктеу
Алдымен Visual Studio Code бағдарламасының ресми сайтына өтіңіз: Visual Studio Code ресми сайты
Сайттан **«Windows Download»** батырмасын басыңыз. Бағдарламаның орнату файлы **.exe** форматында жүктеледі. Файлдың көлемі шамамен 90 МБ.
1-сурет. Visual Studio Code бағдарламасын жүктеу



2-қадам. Лицензиялық келісімді қабылдау
Жүктелген орнату файлын ашыңыз. Орнату терезесі пайда болады. Содан кейін **«Далее» (Келесі)** батырмасын басыңыз.

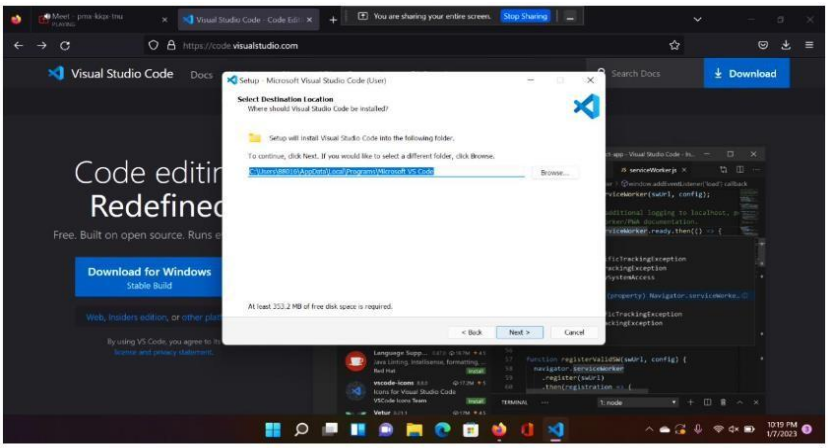


2-сурет. Лицензиялық келісімді қабылдау

3-қадам. Орнату орнын таңдау

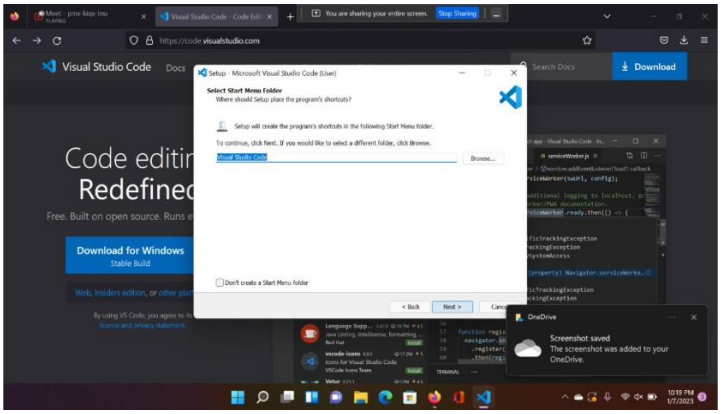
Келесі терезеде Visual Studio Code бағдарламасы орнатылатын бума көрсетіледі.Егер бағдарламаны әдепкі ұсынылған орынға орнатқыңыз келсе, ештеңені өзгертпей: **«Далее» (Келесі)** батырмасын басыңыз.

3-сурет. Бағдарламаны орнату орнын таңдау



4-қадам. «Пуск» мәзіріне қосу

Келесі терезеде бағдарламаның **«Пуск» (Бастау)** мәзірінде орналасатын атауы

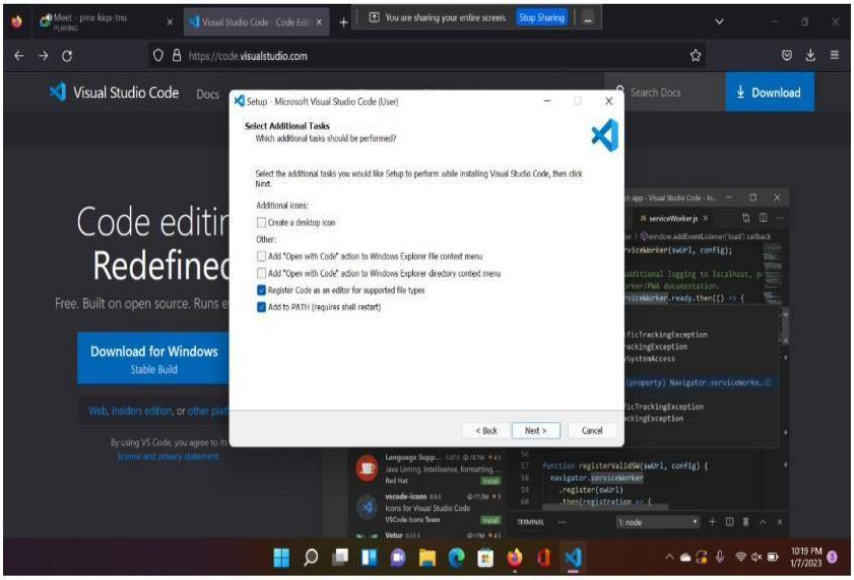


көрсетіледі.

4-сурет. «Пуск» мәзіріне бағдарлама таңбашасын қосу

5-қадам. Қосымша параметрлерді таңдау

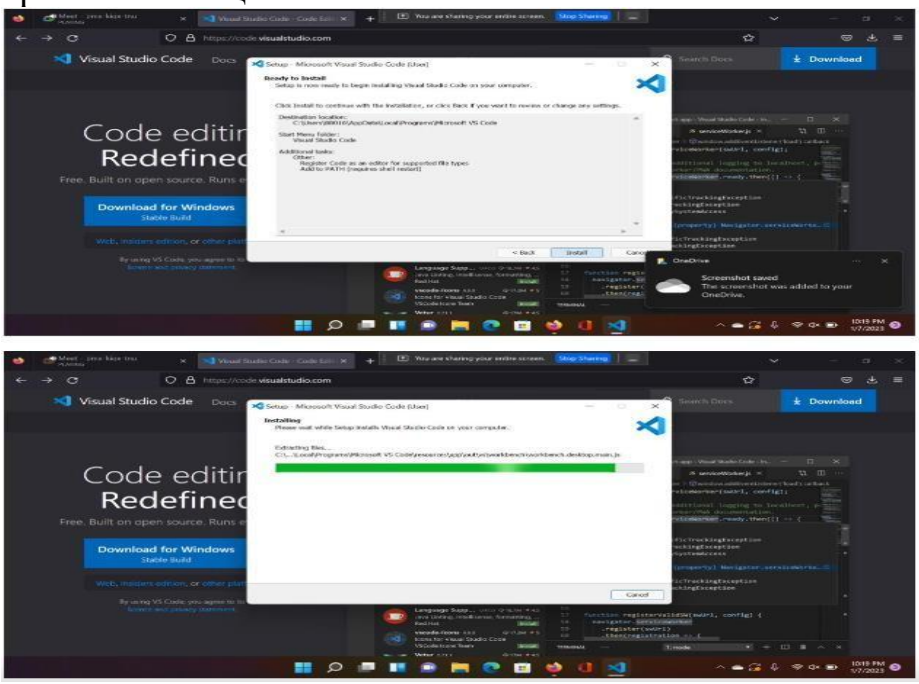
Бұл терезеде Visual Studio Code үшін қосымша параметрлер ұсынылады. Егер қосымша өзгерістер енгізу қажет болмаса, параметрлерді өзгертпей:**«Далее» (Келесі)** батырмасын басыңыз.



5-сурет. Қосымша параметрлер

6-қадам. Бағдарламаны орнату

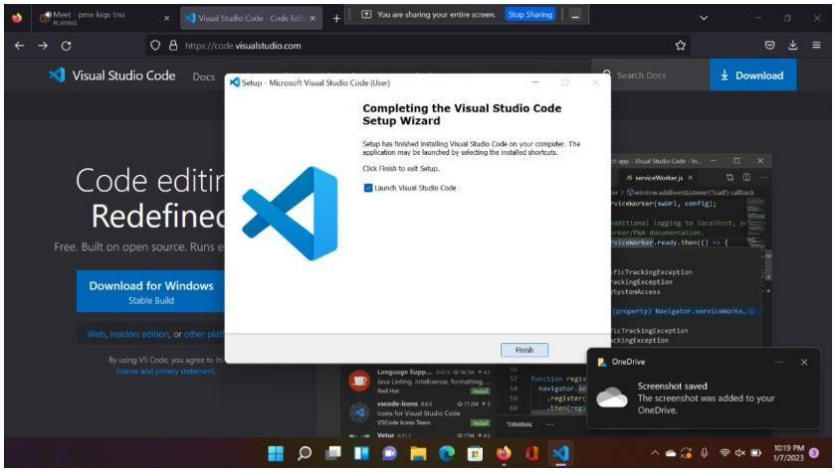
Келесі терезеде орнатуға дайындық туралы ақпарат көрсетіледі. **«Установить» (Орнату)** батырмасын басыңыз.



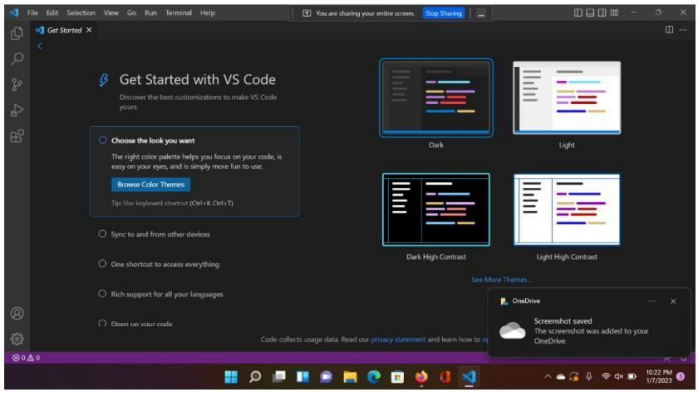
6-сурет. Visual Studio Code бағдарламасын орнату

7-қадам. Орнатуды аяқтау

Орнату процесін аяқтау үшін:«**Закончить» (Аяқтау)** батырмасын басыңыз. Осыдан кейін Visual Studio Code бағдарламасын іске қосуға болады.
7-сурет. Орнатуды аяқтау



8-қадам. Visual Studio Code интерфейсін зерттеу
8-сурет. Visual Studio Code бағдарламасының интерфейсі



Зертханалық жұмыс барысында **Visual Studio Code** бағдарламасын компьютерге орнату және оның негізгі интерфейсімен танысу орындалды.VS Code - веб-бағдарламалау үшін қолданылатын ыңғайлы код редакторы. Ол арқылы **HTML, CSS, JavaScript, PHP** және басқа да тілдерде бағдарламалық код жазуға болады.

Зертханалық жұмыстың нәтижесінде студент:

1. Visual Studio Code бағдарламасын жүктей алады;
2. бағдарламаны компьютерге орната алады;
3. VS Code интерфейсінің негізгі элементтерін ажырата алады;
4. веб-бағдарламалау үшін жұмыс ортасын дайындай алады.

ЗЕРТХАНАЛЫҚ ЖҰМЫС №2
HTML ҚҰРЫЛЫМДЫҚ БЛОКТАРЫ ЖӘНЕ НЕГІЗГІ ТЕГТЕР

Жұмыстың атауы: HTML және оның негізгі элементтеріне кіріспе
Теория: HTML (Hypertext Markup Language) - веб-беттердің құрылымы мен мазмұнын жасау үшін қолданылатын стандартты белгілеу тілі. Ол тақырыптар, абзацтар, тізімдер, сілтемелер, суреттер және тағы басқа элементтерден тұрады.
HTML кодын жазу үшін:

А. Мәтіндік редакторды (мысалы, Visual Studio Code, Notepad) ашыңыз.
Б. Жаңа HTML файлын жасаңыз.
В. Файлды ".html" кеңейтімімен сақтаңыз (мысалы, index.html, contact.html, biography.html).
Г. Нәтижені көру үшін HTML файлын веб-браузерде ашыңыз.

HTML элементтерін түсіну:
Мысалы:

```
<! DOCTYPE html>
<html>
  < head>
    < title> Менің алғашқы HTML парақшам</заголовок>
  </ head>
  < body>
    <h1> Сәлем, Әлем</h1>
    <p> Бұл менің алғашқы HTML парақшам. </p>
  </ body>
</html>
```

HTML құрылымын элементтерге бөліңіз:

- <!DOCTYPE html> → Құжат типінің декларациясы
- <html> → Түбірлік элемент
- <head> → HTML құжаты туралы метаақпаратты қамтиды
- <title> → Құжаттың атауын орнатады (браузер қойындысында көрсетіледі)
- <body> → HTML құжатының мазмұнын қамтиды
- <h1> → 1-деңгейлі тақырып (тақырып тегі)
- <p> → Абзац

***Мәтін және тақырыптар*:** Әртүрлі тақырып тегтерімен (<h1>, <h2>, <h3>) және абзацтармен (<p>) тәжірибе жасаңыз. Мысал:

```
<!DOCTYPE html>
<html>
<head>
<title>Мәтін және тақырыптар</title>
</head>
<body>
<h1>Негізгі курс</h1>
```

<h2>1-тақырыпша</h2>
Бұл мәтін абзацы.
<h3>2-тақырыпша</h3>
Тағы бір абзац.
</body>
</html>

***Гиперсілтеме*:** Гиперсілтемелер жасау үшін <a> (якорь) тегін енгізіңіз. Мысал:

Қосымша ақпарат алу үшін менің веб-сайтыма кіріңіз.

Блоктық деңгейдегі және жолдық элементтер:

- Блоктық деңгейдегі элементтер жаңа жолдан басталады және бүкіл енді алады (мысалы, , <ul>, <li>, <hr>, <h1>).
- Жолдық элементтер жаңа жолдан басталмайды және тек қажетті енді алады (мысалы, , <a>, <img>,
,).

Мәтінді форматтау тегтері: Мәтінді форматтау тегтерін зерттеңіз:

- → Қалың қаріп
- → Мәтінді ерекшелеу
- <u> → Асты сызылған мәтін

Мысал:

Бұл қалың мәтін.
Бұл ерекшеленген мәтін.
Бұл <u>асты сызылған</u> мәтін.

Семантикалық тегтер: Жақсы құрылым үшін семантикалық тегтерді енгізу:

- <header>, <nav>, <section>, <article>, <aside>, <footer>

Мысал:

<!DOCTYPE html>
<html>
<head>
<title>HTML семантикалық тегтері</title>
</head>
<body>
<header>
<h1>Менің жеке сайтым</h1>
</header>
<nav>

Басты бет
Мен туралы

</nav>

```
<section>
<article>
<h2>Мақала тақырыбы</h2>
<p>Мақаланың мазмұнын осы жерден табуға болады.</p>
</article>
<aside>
<h3>Қатысты сілтемелер</h3>
<ul>
<li><a href="#">Блогтар</a></li>
<li><a href="#">Байланыс</a></li>
</ul>
</aside>
</section>
<footer>
<p>&copy; 2024 Менің жеке сайтым</p>
</footer>
</body>
</html>
```

Зертханалық тапсырма:

- Веб-бет жасаңыз. Мазмұнды құрылымдау үшін тақырыптар, абзацтар және тізімдер сияқты блоктық деңгейдегі элементтерді пайдаланыңыз.
- Мазмұнның белгілі бір бөліктерін ерекшелеу үшін мәтінді форматтау тегтерін (мысалы, <i>, <u>, <b>,) қолданыңыз.
- <nav> көмегімен навигациялық панель қосыңыз және навигациялық панельдің ішінде парақшаңыздың әрбір бөлімі үшін <a> гиперсілтемелерін жасаңыз.
- Әр бетте сәйкес семантикалық тегтерді (<header>, <section>, <article>, , <footer>) пайдаланыңыз.

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 3
HTML: КЕСТЕЛЕР

Эксперимент атауы: Кестелерге арналған HTML бағдарламасын жазу.
Теория: Кесте - жолдар мен бағандардың орналасуынан тұрады. HTML негіздерін біле отырып, кез келген адам кесте жасай алады. HTML тілінде кесте <table> тегі арқылы анықталады.
HTML кестесін құру қадамдары:

- Кестені <table> тегімен бастаңыз және соңында </table> тегін пайдаланыңыз.
- Содан кейін <caption></caption> тегі арқылы осы кестеге атау беріңіз.
- Бастапқы және соңғы тегтердің ішінде кестенің тақырыбы мен кесте денесін <thead></thead> және <tbody></tbody> тегтері арқылы анықтаңыз.
- <thead> және <tbody> тегтерінің ішінде кесте жолдары <tr> тегтері арқылы анықталады.
- <thead> ішінде әрбір жолдағы (<tr>) бағандарды <th> тегтері арқылы анықтаңыз.
- <tbody> ішінде әрбір жолдағы (<tr>) бағандарды <td> тегтері арқылы анықтаңыз.

Бастапқы код:

```
<table border="2" cellspacing="3" cellpadding="3">
    <thead>
        <tr>
            <th colspan="2">Қызметкер</th>
            <th colspan="2">Жалақы</th>
        </tr>
        <tr>
            <th>ID</th>
            <th>Аты-жөні</th>
            <th>Негізгі жалақы</th>
            <th>Үй жалдау ақысы</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>EG789</td>
            <td>Асқар Серіков</td>
            <td>30000</td>
            <td>5000</td>
        </tr>
        <tr>
            <td>EG421</td>
            <td>Айгерім Жұмабаева</td>
            <td>35000</td>
            <td>6000</td>
        </tr>
        <tr>
            <td colspan="2">Жалпы жалақы</td>
            <td>75000</td>
            <td>11000</td>
        </tr>
    </tbody>
</table>
```

[***Ескерту***: *cellspacing әрбір ұяшықтың сыртындағы арақашықтықты анықтайды. cellpadding әрбір ұяшықтың ішіндегі арақашықтықты анықтайды. border әрбір*

ұяшықтың шекарасын анықтайды. rowspan ұяшықтың бірнеше жолды қамтуына мүмкіндік береді, ал colspan - бірнеше бағанды қамтуына мүмкіндік береді.]

Нәтижелер:

Қызметкер		Жалақы	
ID	**Аты-жөні**	**Негізгі жалақы**	**Үй жалдау**
EG789	Асқар Серіков	30000	5000
EG421	Айгерім Жұмабаева	35000	6000
Жалпы жалақы		**75000**	**11000**

Сурет: HTML-кестесі

Зертханалық тапсырма - 1: HTML кесте тегтерін пайдаланып, сабақ кестесін жасаңыз.

Зертханалық тапсырма - 2: Төменде көрсетілген кестеге ұқсас кесте жасаңыз:

Тақырып	**Студенттер**	**Мәліметтер**		
Студенттер тізімі	**ID**	**Аты-жөні**	**Мамандығы**	**Тіркеу нөмірі**
	1	Бауыржан	Компьютерлік ғылымдар	12345
	2	Нұрлан	Электроника	23456
	3	Серік	Электротехника	34567
	4	Дәурен	Құрылыс	45678

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 4
HTML: ТІЗІМДЕР

Теория: HTML тізімдері ақпарат тізімдерін көрсету үшін қолданылады. Барлық тізімдер бір немесе бірнеше тізім элементтерінен тұруы мүмкін. HTML тізімдерінің үш негізгі түрі бар:

1. Реттелген тізім (ol)
2. Реттелмеген немесе маркерленген тізім (ul)
3. Сипаттамалар тізімі немесе анықтамалар тізімі (dl).

A. HTML реттелген тізімі (ol):

· HTML тілінде реттелген тізімдегі барлық тізім элементтері әдепкі бойынша маркерлердің орнына сандармен белгіленеді.

- HTML реттелген тізімі <ol> тегінен басталып, тегімен аяқталады.
-   тегтерінің ішінде тізім элементтерін тегтері арқылы анықтау қажет.
- тегтерінің ішінде тізімнің мазмұнын анықтау қажет.

Бастапқы код:

```
<body>
<h2>Жемістер тізімі</h2>
    <ol>
        <li>Алма</li>
        <li>Манго</li>
        <li>Банан</li>
        <li>Жүзім</li>
        <li>Апельсин</li>
    </ol>
</body>
```

Нәтижесі:

Жемістер тізімі

1. Алма
2. Манго
3. Банан
4. Жүзім
5. Апельсин

Сурет: Реттелген тізім

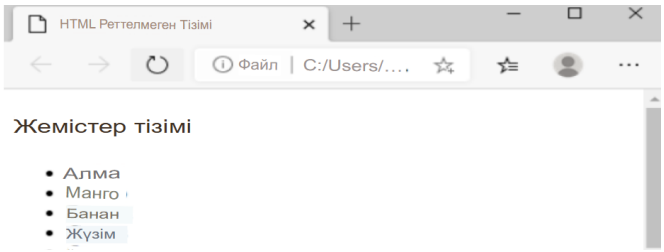
Реттелмеген тізім:

Бастапқы код:

<h2>Жемістер тізімі</h2>


```
    <li>Алма</li>
    <li>Манго</li>
    <li>Банан</li>
    <li>Жүзім</li>
    <li>Апельсин</li>
</ul>
```

Нәтижесі:



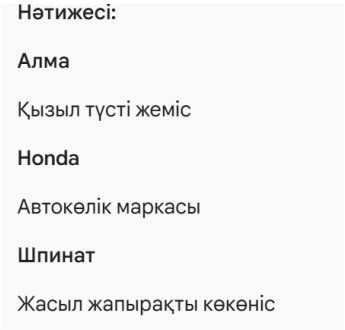
HTML сипаттамалар / анықтамалар тізімі:

- <dl> (Анықтамалар тізімі) тегі – анықтамалар тізімінің бастапқы тегі
- <dt> (Definition Term) тегі - терминді (атауды) анықтайды
- <dd> (Анықтаманың сипаттамасы) тегі – терминнің анықтамасын анықтайды
- </dl> (Анықтамалар тізімі) тегі – анықтамалар тізімінің жабу тегі

Бастапқы код:

```
<dl>
    <dt><b>Алма</b></dt>
    <dd>Қызыл түсті жеміс</dd>
    <dt><b>Honda</b></dt>
    <dd>Автокөлік маркасы</dd>
    <dt><b>Шпинат</b></dt>
    <dd>Жасыл жапырақты көкөніс</dd>
</dl>
```

Нәтижесі:



Сурет: Сипаттамалар тізімі

Зертханалық тапсырмалар

1: Келесі тізімдерге ұқсас HTML тізімдерін жасаңыз:

A. Себеттегі тапсырыстар реті:

a) А. Кітап
b) Б. Қалам
c) В. Футболка
d) Г. Джинсы

Б. Көлік құралдарының тізімі:

a) А. Велосипед
b) Б. Автомобиль
c) В. Автобус
d) Г. Такси

В. Бағдарламалау тілдерінің тізімі:

a) ● C
b) ● C++
c) ● Java
d) ● JavaScript
e) ● Rust

Г. Деректер қорының тізімі:

a) ● MySQL
b) ● MongoDB
c) ● PostgreSQL
d) ● GraphQL

[***Кеңестер:*** тізімдердің стиль түрлерін іздеңіз]

Зертханалық тапсырма - 2: HTML анықтамалар тізімін пайдаланып, төменде көрсетілген адамның лауазымы, ақпараты және қасиеттерін сипаттаңыз:

- Айтимова Улзада Жолдасбековна
- Бельдеубаева Жанар Толеубаевна
- Жаксыбаев Дархан Оракбаевич
- Жауханова Ләззат Абзаловна
- Исмаилова Айсулу Абжаппаровна
- Мустафаева Нагима Таировна
- Найзағараева Ақгүл Аманжолқызы
- Омарханова Динара Жоламановна

[***Кеңестер:*** Олар туралы ақпаратты келесі сілтемеден алуға болады: https://kazatu.edu.kz/ru/facultet/kafedra-informacionnyh-sistem]

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 5
HTML: ФОРМАЛАР, МУЛЬТИМЕДИА, ФРЕЙМДЕР ЖӘНЕ IFRAME

Теория: HTML-форма - веб-беттің әртүрлі элементтерді, мысалы, енгізу өрістерін, батырмаларды, жалаушаларды, радио батырмаларды және тағы басқаларды қамтитын, пайдаланушыларға өңдеу үшін деректерді веб-серверге жіберуге мүмкіндік беретін бөлімі.
Енгізу: <input> тегтері пайдаланушыларға деректерді енгізуге мүмкіндік беретін әртүрлі форма басқару элементтерін жасау үшін қолданылады. <input> тегінің type атрибуты көрсетілетін енгізу элементінің түрін анықтайды, мысалы, мәтін енгізу, жалауша, радио батырма және басқалары.
Негізгі енгізу түрлері:

- *Мәтін енгізу:* <input type="text" />
- *Электрондық пошта енгізу***:** <input type="email" />
- *Құпиясөз енгізу***:** <input type="password" />
- *Мәтіндік аймақты енгізу:* <textarea name="details" />
- *Радио батырма***:**
  - **a.** <input type="radio" name="gender" value="female" />
  - **b.** <input type="radio" name="gender" value="male" />
- *Жалаушаны (checkbox) енгізу:* <input type="checkbox" name="term-and-condition"/>
- *Опцияларды таңдау және енгізу:*

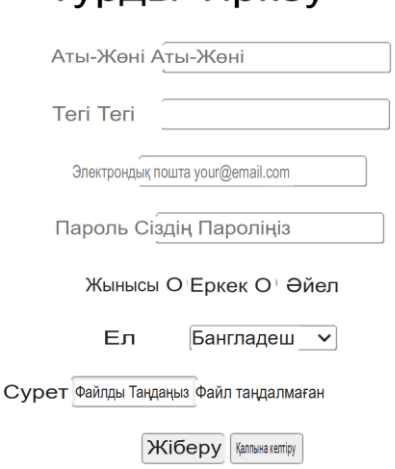
<select>
<option value="fall-2023">Күз 2023</option>
<option value="spring-2024">Көктем 2024</option>
<option value="fall-2024">Күз 2024</option>
</select>

- *Форманы жіберу батырмасы:* <input type="submit" value="Кіру" />

Бастапқы код:

```
<form action="/collect-tour-info" align="center">
<h1>Турға тіркелу</h1>
<label>Аты</label>
<input type="text" placeholder="Атыңыз"> <br><br>
<label>Тегі</label>
<input type="text" placeholder="Тегіңіз"><br><br>
<label>Электрондық пошта</label>
<input type="email" placeholder="sizdin@email.com"> <br><br>
<label>Құпия сөз</label>
<input type="password" placeholder="Құпия сөзіңіз"><br><br>
<label>Жынысы</label>
<input type="radio" value="Ер"/> Ер
<input type="radio" value="Әйел" /> Әйел<br><br>
<label>Ел</label>
<select>
<option>Бангладеш</option>
<option>АҚШ</option>
<option>Үндістан</option>
</select>
<br><br>
<label>Сурет</label>
<input type="file"><br><br>
<input type="submit" value="Жіберу">
<input type="reset" value="Тазалау">
</form>
```

Нәтижесі:



Сурет: HTML негізгі формасы

Зертханалық тапсырма – 1:
Ауруханаға пациенттердің қабылдануы туралы ақпарат жинау үшін **HTML-форма** құрыңыз.

Зертханалық тапсырма – 2:
Өзіңізді университет оқытушысы ретінде елестетіңіз. Студент сіздің университетіңізге оқуға түскісі келеді. Оқуға түсу үшін қажетті ақпаратты жинау мақсатында **HTML тілін пайдаланып форма құрыңыз.**

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 6
HTML МУЛЬТИМЕДИЯЛЫҚ ТЕГТЕРІНЕ КІРІСПЕ

A. <img> тегінің көмегімен кескіндерді көрсетуге арналған бөлім құрыңыз.
B. <video> тегінің көмегімен бейнефайлдарды ойнатуға арналған бөлім құрыңыз.
C. <audio> тегінің көмегімен аудиофайлдарды ойнатуға арналған бөлім құрыңыз.
D. <iframe> тегінің көмегімен басқа веб-беттің контентін / YouTube контентін көрсетуге арналған бөлім құрыңыз.

Теория: HTML мультимедиялық тегтері веб-беттерге кескіндер, аудио және бейне сияқты әртүрлі мультимедиялық контент түрлерін кірістіру және басқару үшін қолданылады. Мұндай тегтерге кескіндерге арналған <img>, дыбысқа арналған <audio> және бейнені ойнатуға арналған <video> тегтері жатады. Олар мультимедиялық элементтерді веб-бетке біркелкі кіріктіруге мүмкіндік береді.

Басқару, автоматты түрде ойнату және дереккөзді таңдау сияқты атрибуттардың көмегімен бұл тегтер әзірлеушілерге пайдаланушы тәжірибесін реттеуге мүмкіндік береді. Олар интерактивті, динамикалық және тартымды веб-сайттарды құруда маңызды рөл атқарады.

A. <img> тегінің көмегімен кескінді кірістіру

Бастапқы код:

```
<img
   src="https://i.ibb.co/Jpmpd9P/chris-stenger-fv-Jwch-RL6xw-unsplash-1.jpg"
   width="750"
   height="450"
   alt="Таулары бар көлдің әсем көрінісі"
/>
```

- **Ені мен биіктігі:** Кескіннің ені мен биіктігін орнатыңыз.
- **src:** Кескін файлының орналасқан жерін (дереккөзін) көрсетеді.
- **alt="...":** Кескіннің сипаттамасын білдіреді (міндетті емес).

Нәтижесі:



Сурет: Кірістірілген кескін

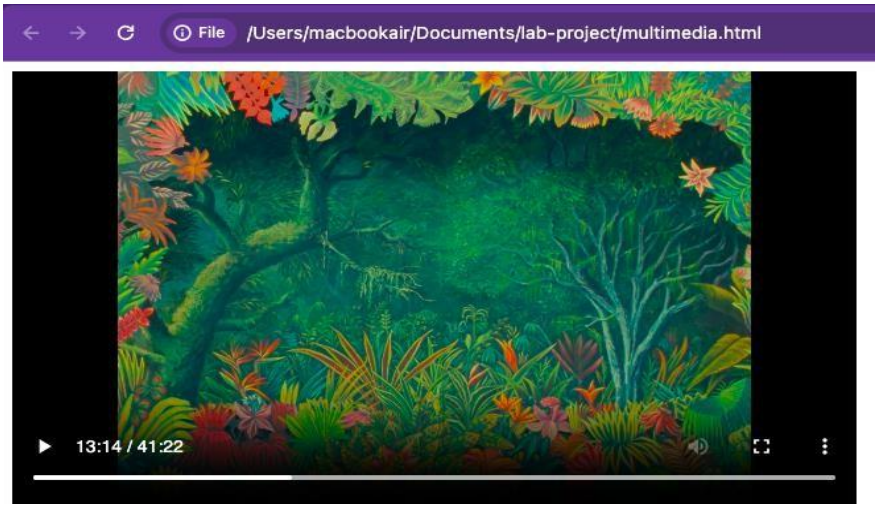
<video> тегінің көмегімен бейнені кірістіру. <video> тегі веб-бетке бейнеконтентті кірістіру үшін қолданылады.

Бастапқы код:

```
<video width="640" height="360" controls>
   <source src="videos/presentation.mp4" type="video/mp4"/>
</video>
```

- **Ені мен биіктігі:** Бейне ойнатқыштың ені мен биіктігін орнатыңыз.
- **Басқару элементтері (controls):** Ойнату, кідірту, дыбыс деңгейін реттеу және т.б. басқару элементтерін қосады.
- **<source>:** Бейнефайлды және оның түрін көрсетеді

Нәтижесі:



Сурет: Кірістірілген аудио

<audio> тегінің көмегімен аудионы кірістіру

<audio> тегі веб-бетке аудиоконтентті кірістіру үшін қолданылады.

Бастапқы код:

```
<audio controls>
   <source src="звук/playback.mp3" type="audio/mpeg"/>
</audio>
```

- **Басқару элементтері (controls):** Аудионы ойнату, кідірту, дыбыс деңгейін реттеу және т.б. басқару элементтерін қосады.
- **<source>:** Аудиофайлды және оның түрін көрсетеді.
  Нәтижесі:



Сурет: Кірістірілген аудио

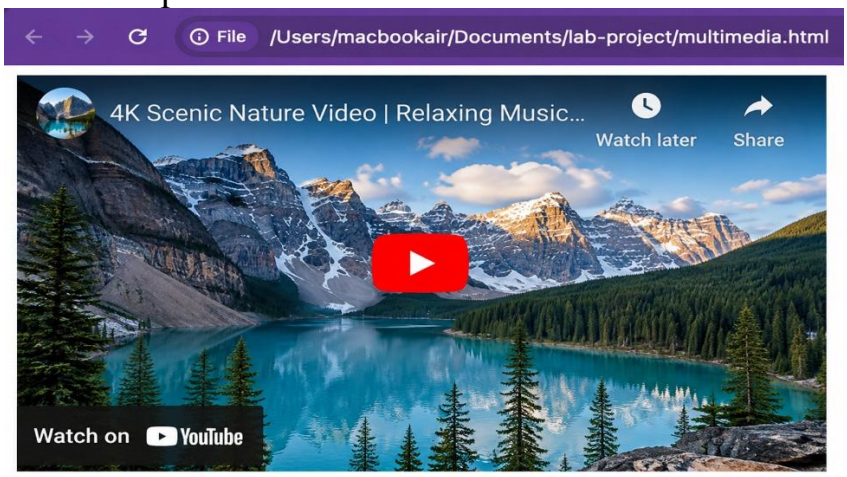
<iframe> тегінің көмегімен сыртқы құжатты кірістіру

<iframe> тегі (кірістірілген фрейм) ағымдағы HTML-құжатқа сыртқы құжатты немесе веб-бетті кірістіру үшін қолданылады.

Бастапқы код:

```
<iframe
    width="560"
    height="315"
    src="https://www.youtube.com/embed/LLuolLkKT5E?si=uVyjvJSTNafATRJE"
    title="YouTube бейне ойнатқышы"
    frameborder="0"
    allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;">
</iframe>
```

Нәтижелер:



Сурет: `<iframe>` көмегімен кірістірілген сыртқы құжат

Зертханалық тапсырма: Әрқайсысы жеке HTML-файлды пайдаланатын 4 беттен тұратын қарапайым веб-сайт құрыңыз: **image.html**, **video.html**, **audio.html** және **iframe.html**.

Әр бетке тақырып және беттер арасында оңай ауысуға мүмкіндік беретін **сілтемелерді** қосыңыз. Пайдаланушы сілтемелерді басу арқылы бір беттен екінші бетке өте алуы керек.

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 7
CSS-КЕ КІРІСПЕ

Теория: CSS (Cascading Style Sheets - каскадты стильдер кестесі) - веб-беттерді безендіру және олардың орналасуын реттеу үшін қолданылатын тіл. Ол HTML элементтерінің сыртқы көрінісін, мысалы, түстерді, қаріптерді, аралықтарды және жалпы орналасуды басқарады. Контентті (HTML) дизайннан (CSS) бөлу арқылы әзірлеушілер сыртқы көрінісі тартымды, қолдауға және жаңартуға ыңғайлы веб-сайттар жасай алады. CSS сонымен қатар адаптивті дизайн жасауға мүмкіндік береді, яғни веб-беттер әртүрлі экран өлшемдері мен құрылғыларда әдемі көрінеді.

CSS-ті HTML-мен бірге қолданудың 3 тәсілі бар:

1. Кірістірілген CSS (Inline CSS)
2. Ішкі CSS (Internal CSS)
3. Сыртқы CSS (External CSS)

А. Кірістірілген CSS (Inline CSS)

Кірістірілген CSS style атрибутының көмегімен тікелей HTML элементіне қолданылады.

Мысал:

```
<p style="color: blue; font-size: 16px;">
   Бұл кірістірілген CSS қолданылған абзац.
</p>
```

Б. Ішкі CSS (Internal CSS)

Ішкі CSS HTML құжатының <head> бөліміндегі <style> тегінің ішінде анықталады.

Мысал:

```
<html lang="kk">
<head>
   <style>
      h1 {
         color: green;
         font-size: 24px;
      }
   </style>
</head>

<body>
   <h1>Бұл ішкі CSS қолданылған тақырып.</h1>
</body>
</html>
```

В. Сыртқы CSS (External CSS)

Сыртқы CSS жеке CSS файлында анықталады және <link> тегі арқылы HTML құжатымен байланыстырылады.

Мысал (index.html файлының ішінде):

```
<!DOCTYPE html>
<html lang="kk">
<head>
   <link rel="stylesheet" href="styles.css" />
</head>

<body>
   <h2>Бұл сыртқы CSS қолданылған тақырып.</h2>
</body>
</html>
```

styles.css файлының ішінде:

```
h2 {
   color: red;
   font-size: 20px;
}
```

CSS селекторлары

CSS селекторлары - HTML элементтерін таңдау және оларға стиль қолдану үшін пайдаланылатын үлгілер. Селекторлардың әртүрлі түрлері бар: элемент селекторы, класс селекторы және ID селекторы.

А. Элемент селекторы (Element Selector)

Белгілі бір HTML элементінің барлық даналарын таңдайды.

Мысал:

```
p {
   color: lightblue;
}
```

Б. Класс селекторы (Class Selector)

Белгілі бір class атрибуты бар элементтерді таңдайды.

Мысал:

```
.container {
   background-color: pink;
}
```

В. ID селекторы (ID Selector)

Белгілі бір id атрибуты бар бір элементті таңдайды.

Мысал:

```
#navbar {
   background-color: cyan;
}
```

CSS типографиясы

- **color** → қаріптің түсін орнатады.
- **font-size** → қаріптің өлшемін орнатады.
- **font-weight** → қаріптің қалыңдығын орнатады.

- **font-style** → қаріптің стилін орнатады.

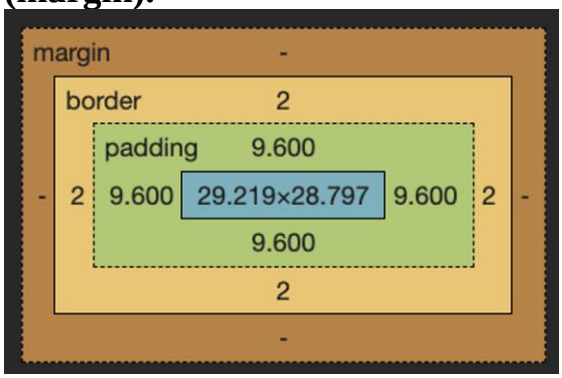
Мысал:

```
p {
   color: lightblue;
   font-size: 18px;
   font-weight: 700;
   font-style: italic;
}
```

CSS Box Model моделі

CSS Box Model - элементтердің орналасуын және олардың арасындағы арақашықтықты сипаттайтын модель. Ол веб-беттегі әрбір элементтің төрт негізгі құрамдас бөліктен тұратын тікбұрышты блок ретінде көрсетілуін сипаттайды:

мазмұн (content), ішкі аралық (padding), жиек (border) және сыртқы аралық (margin).



Сурет: DevTools құралындағы Box Model

А. Мазмұн (Content): Мазмұн - элементтің негізгі контенті. Ол контенттің ені мен биіктігін қамтиды.

Б. Жиек (Border):Жиек - мазмұнды қоршап тұратын сызық. Оның белгілі бір ені, стилі және түсі болуы мүмкін.

В. Ішкі аралық (Padding): Ішкі аралық - мазмұн мен жиектің арасындағы бос кеңістік. Ол элементтің ішкі арақашықтығын реттеуге көмектеседі.

Г. Сыртқы аралық (Margin): Сыртқы аралық - ең сыртқы қабат. Ол элементтің жиегі мен айналасындағы басқа элементтердің арасында бос кеңістік жасайды. Элементтер арасындағы сыртқы арақашықтықты реттеуге көмектеседі.

Мысал:

```
.box {
   width: 200px;
   height: 150px;
   padding: 20px;
   border: 2px solid gray;
   margin: 10px;
}
```

Зертханалық тапсырма

Кемінде **үш бөлімнен** тұратын қарапайым HTML-бет жасаңыз. Әр бөлімде тақырып, абзац, элементтер тізімі және т.б. болуы керек.

CSS көмегімен келесі стильдерді қолданыңыз:

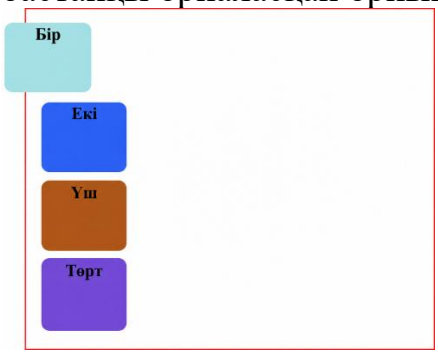
- Әр бөлімді әртүрлі безендіру үшін **класс селекторын (Class Selector)** және **ID селекторын (ID Selector)** пайдаланыңыз.
- Таңдалған элементтер үшін **Box Model** қасиеттерін (margin, padding және т.б.) реттеңіз.
- Тақырыптар мен абзацтарға әртүрлі **қаріп өлшемдерін (font-size)** орнатыңыз.
- Белгілі бір мәтіннің сыртқы көрінісін өзгерту үшін **қаріп қалыңдығын (font-weight)** және **қаріп стилін (font-style)** қолданыңыз.
- Бір немесе бірнеше элементке **фон түсін (background-color)** қолданыңыз.

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 8.
CSS ПОЗИЦИЯЛАУ

Теория: CSS позициялау веб-беттегі элементтердің орналасуын басқару үшін қолданылады. Ол әзірлеушілерге **статикалық (static)**, **салыстырмалы (relative)**, **абсолютті (absolute)**, **бекітілген (fixed)** және **жабысқақ (sticky)** сияқты қасиеттерді пайдаланып, элементтерді қажетті орынға орналастыруға мүмкіндік береді.
Позициялаудың әрбір түрі әртүрлі жұмыс істейді, бұл контентті орналастыруда икемділік береді. Мысалы, **салыстырмалы (relative)** позициялау элементті өзінің қалыпты орналасқан орнына қатысты жылжытады, ал **абсолютті (absolute)** позициялау элементті ең жақын позицияланған ата-аналық элементке қатысты орналастырады. Бұл динамикалық және бейімделгіш веб-бет макеттерін жасауға көмектеседі.
А. Статикалық позициялау (Static Positioning). Бұл - элементтердің әдепкі орналасу тәртібі. Егер белгілі бір элементтің стилінде position қасиеті көрсетілмесе, элемент стандартты **статикалық (static)** позицияда орналасады.
Б. Салыстырмалы позициялау (Relative Positioning). Салыстырмалы позициялау элементтің өзінің қалыпты/бастапқы орналасқан орнына қатысты жылжытылатынын білдіреді. Элементтің орналасуын өзгерту үшін top, right, bottom және left қасиеттерін пайдалануға болады.
Мысал:

```
.one {
   background-color: powderblue;
   position: relative;
   right: 50px;
}
```

Нәтижесі:
.one класы берілген элементтің фон түсі **ашық көк (powderblue)** болады және ол өзінің бастапқы орналасқан орнынан **50 пиксель солға қарай жылжиды**.



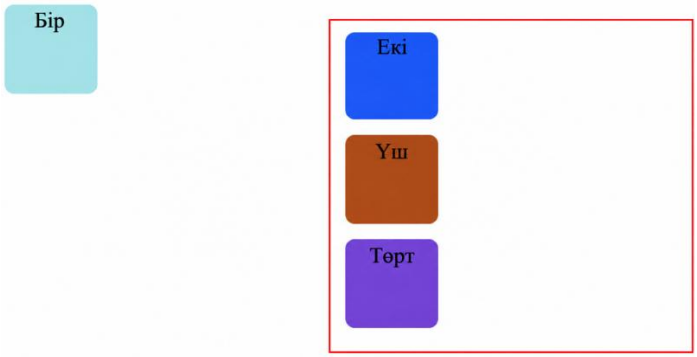
Сурет: Салыстырмалы позициялау

А. Абсолютті позициялау (Absolute Positioning). Абсолютті позициялау элементтің ең жақын **позицияланған ата-аналық элементке** қатысты жылжытылатынын білдіреді. Егер ондай ата-аналық элемент болмаса, онда элемент бастапқы қамтушы блокқа (әдетте <html> элементіне) қатысты орналастырылады.
Мысал:

```
.one {
    background-color: powderblue;
    position: absolute;
    top: 50px;
    left: 0;
}
```

Нәтижесі:

.one класы берілген элемент **абсолютті позицияланады**, жоғарыдан 50px, ал сол жақтан 0 қашықтықта орналасады.



Сурет: Абсолютті позициялау

А. Бекітілген позициялау (Fixed Positioning)

Бекітілген позициялау элементтің браузер терезесіне қатысты орналасатынын білдіреді. Бетті төмен немесе жоғары айналдырған кезде де элемент өз орнында бекітіліп қалады.

Мысал:

```
.navbar {
    background-color: lightblue;
    position: fixed;
    top: 0;
    width: 100%;
    text-align: center;
}
```

Нәтижесі:

.navbar элементі браузер терезесінің **жоғарғы жағында бекітіліп** орналасады. Оның ені 100% болады және ішіндегі мәтін ортаға туралanады. Бетті айналдырған кезде навигация жолағы орнын өзгертпейді.



Сурет: Абсолютті позициялау

Жабысқақ позициялау кезінде элемент бетті айналдыру барысында белгілі бір нүктеге жеткенге дейін салыстырмалы (relative) позицияланған элемент сияқты әрекет

етеді. Белгіленген нүктеге жеткеннен кейін ол **бекітілген (fixed)** элемент сияқты орнында қалып қояды.

Мысал:

```
.sticky-navbar {
    position: sticky;
    top: 0;
}
```

CSS z-index

z-index қасиеті элементтердің бір-бірінің үстіне орналасу ретін анықтайды. z-index мәні жоғары элемент мәні төмен элементтің **алдында көрсетіледі**.

Мысал:

```
#first {
    position: absolute;
    z-index: -1;
    background-color: #192733;
    top: 30px;
    left: 30px;
}

#second {
    position: absolute;
    background-color: #ff5c35;
    top: 10px;
    left: 10px;
}

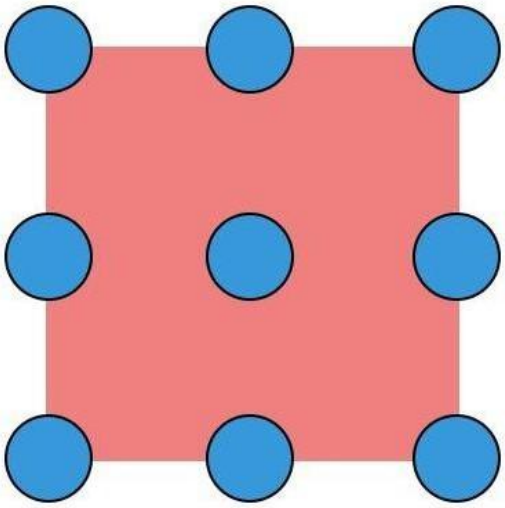
#third {
    position: absolute;
    z-index: -2;
    background-color: #4fb06d;
    top: 50px;
    left: 50px;
}
```

Нәтижесі:



Сурет: CSS **z-index**.

Зертханалық тапсырма: CSS қасиеттерін пайдаланып, осы суретке ұқсас дизайн жасаңыз.



[Кеңес: блоктарды дөңгелек ету үшін әр блокқа border-radius: 50%; қасиетін қосыңыз..]

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 9.
CSS FLEXBOX ЖӘНЕ GRID

Теория: Flexbox: Flexbox - элементтерді негізінен **жол (row)** немесе **баған (column)** түрінде орналастыруға арналған бір өлшемді макет моделі. Ол қолжетімді бос кеңістікке байланысты контейнер ішіндегі элементтердің өлшемі мен орналасуын динамикалық түрде өзгертуге мүмкіндік береді және күрделі макеттерді құруды жеңілдетеді.

display: flex - икемді контейнерді анықтайды. Берілген мәнге байланысты ол кірістірілген немесе блоктық контейнер болуы мүмкін. Бұл қасиет контейнердің барлық тікелей еншілес элементтері үшін **flex-контекст** құруға мүмкіндік береді.

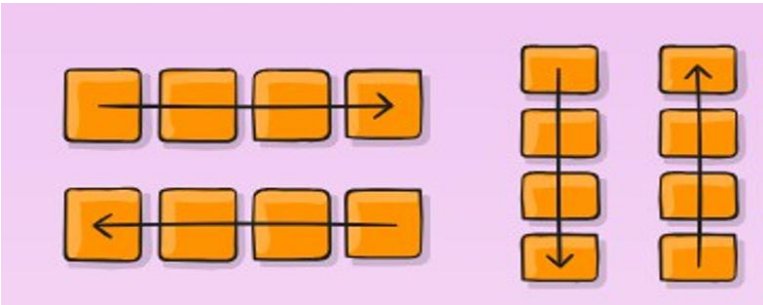
Мысал:

```
.container {
   display: flex;
}
```

flex-direction - негізгі осьті анықтайды және flex-контейнер ішіндегі элементтердің орналасу бағытын белгілейді.

Мысал:

```
.container {
   display: flex;
   flex-direction: row | row-reverse | column | column-reverse;
}
```



Сурет: Flex бағыты

justify-content - элементтердің негізгі ось бойымен туралануын анықтайды. Бұл қасиет flex-контейнерде қалған қосымша бос кеңістікті элементтер арасында дұрыс бөлуге көмектеседі.

Мысал:

```
.container {
   display: flex;
   justify-content: flex-start | flex-end | center | space-between | space-around | space-evenly;
}
```



Сурет: Flex элементтерін justify-content арқылы туралау

align-items - ағымдағы жолдағы flex элементтерінің **көлденең (қосалқы) ось бойымен** орналасуының әдепкі тәртібін анықтайды.

Мысал:

```
.container {
   display: flex;
   align-items: stretch | flex-start | flex-end | center | baseline;
}
```



Сурет: Flex элементтерін туралау

Gap қасиеті flex элементтерінің арасындағы бос кеңістікті нақты басқарады.

Мысал:

```
.container {
   display: flex;
   gap: 10px;

   gap: 10px 20px; /* жолдар мен бағандар арасындағы аралық */
   row-gap: 10px;
```

```
   column-gap: 20px;
}
```
Бастапқы код
index.html файлында:
```
<!DOCTYPE html>
<html lang="kk">
<head>
   <title>Құжат</title>
   <link rel="stylesheet" href="style.css" />
</head>

<body>

   <div class="parent">

      <div class="child">
         Flexbox - негізінен элементтерді жол немесе баған түрінде
         орналастыруға арналған бір өлшемді макет моделі.
      </div>

      <div class="child">
         Flexbox - негізінен элементтерді жол немесе баған түрінде
         орналастыруға арналған бір өлшемді макет моделі.
      </div>

      <div class="child">
         Flexbox - негізінен элементтерді жол немесе баған түрінде
         орналастыруға арналған бір өлшемді макет моделі.
      </div>

      <div class="child">
         Flexbox - негізінен элементтерді жол немесе баған түрінде
         орналастыруға арналған бір өлшемді макет моделі.
      </div>

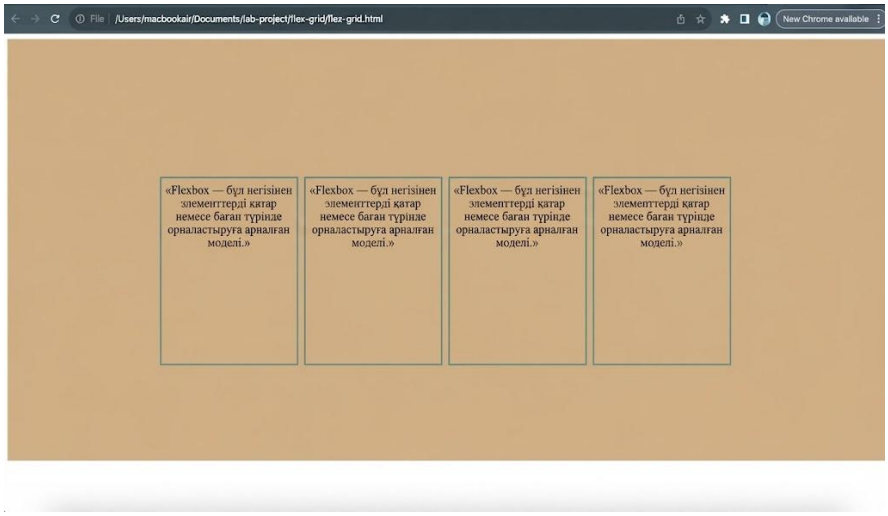
   </div>

</body>
</html>
```
style.css файлында:
```
.parent {
   display: flex;
```

```
    flex-direction: row;
    gap: 10px;
    padding: 10px;
    height: 600px;
    background-color: beige;
    align-items: center;
    justify-content: center;
}

.child {
    border: 2px solid darkcyan;
    padding: 10px;
    width: 200px;
}
```

Нәтижесі:



Сурет: CSS FlexBox

CSS Grid - жолдар мен бағандардан тұратын тор құрылымдарын жасауға мүмкіндік беретін екі өлшемді макет моделі. Ол журналдарда, газеттерде және адаптивті веб-қосымшаларда кездесетін күрделі торлы макеттерді жасауға өте қолайлы.

Мысал:

```
.container {
    display: grid;
}
```

Grid Columns & Rows - мәндер тізімінің көмегімен тордың бағандары мен жолдарын анықтайды.

Мысал:

```
.container {
    display: grid;

    grid-template-columns: 300px 400px 500px;
```

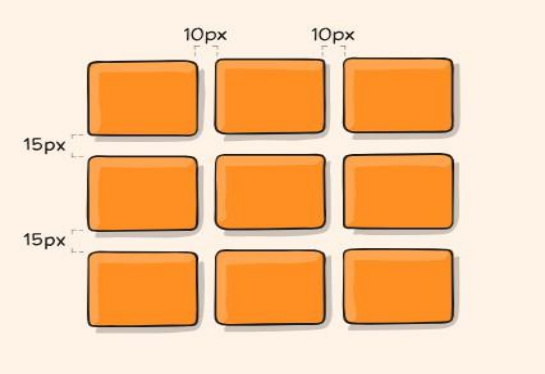
```
    /*
    grid-template-columns: 1fr 1fr 1fr;
    Әрбір элемент контейнер енінің ішінде бірдей өлшемге ие болады.
    */

    /*
    grid-template-columns: repeat(3, 1fr);
    Әрбір элемент контейнер енінің ішінде бірдей өлшемге ие болады.
    */

    grid-template-rows: 100px 1fr max-content;

    /*
    grid-template-rows: min-content 1fr min-content;
    */
}
```

Нәтижесі:



Сурет: CSS Grid

Қайталау (Repeat). Егер анықтамаңызда қайталанатын бөліктер болса, кодты ықшамдау үшін **repeat()** функциясын пайдалануға болады.

Мысал:

```
.container {
    display: grid;
    grid-template-columns: repeat(3, 100px);
}
```

Бастапқы код

index.html файлында:

```
<!DOCTYPE html>
<html lang="kk">
<head>
    <title>Құжат</title>
    <link rel="stylesheet" href="style.css" />
```

```
</head>
<body>
   <div class="parent">
      <div class="child">
         Flexbox - негізінен элементтерді жол немесе баған түрінде
         орналастыруға арналған бір өлшемді макет моделі.
      </div>
      <div class="child">
         Flexbox - негізінен элементтерді жол немесе баған түрінде
         орналастыруға арналған бір өлшемді макет моделі.
      </div>

      <div class="child">
         Flexbox - негізінен элементтерді жол немесе баған түрінде
         орналастыруға арналған бір өлшемді макет моделі.
      </div>

      <div class="child">
         Flexbox - негізінен элементтерді жол немесе баған түрінде
         орналастыруға арналған бір өлшемді макет моделі.
      </div>

      <div class="child">
         Flexbox - негізінен элементтерді жол немесе баған түрінде
         орналастыруға арналған бір өлшемді макет моделі.
      </div>

      <div class="child">
         Flexbox - негізінен элементтерді жол немесе баған түрінде
         орналастыруға арналған бір өлшемді макет моделі.
      </div>

      <div class="child">
         Flexbox - негізінен элементтерді жол немесе баған түрінде
         орналастыруға арналған бір өлшемді макет моделі.
      </div>

   </div>

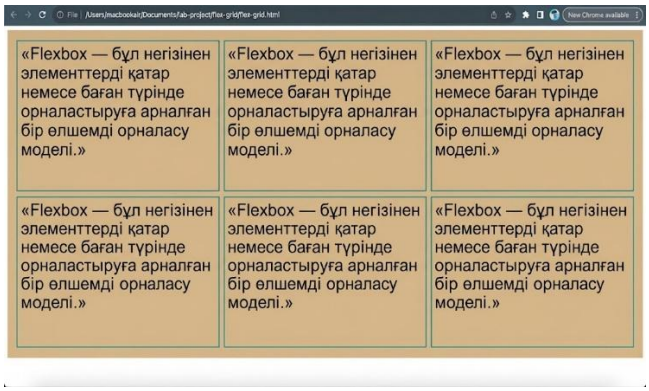
</body>
</html>
```

style.css файлында:

```
.parent {
   display: grid;
   grid-template-columns: 1fr 1fr 1fr;
   grid-template-rows: 1fr 1fr;
   gap: 10px;
```

```
    padding: 20px;
    height: 600px;
    background-color: beige;
}
.child {
    border: 2px solid darkcyan;
    padding: 10px;
    font-size: 20px;
    width: auto;
}
```

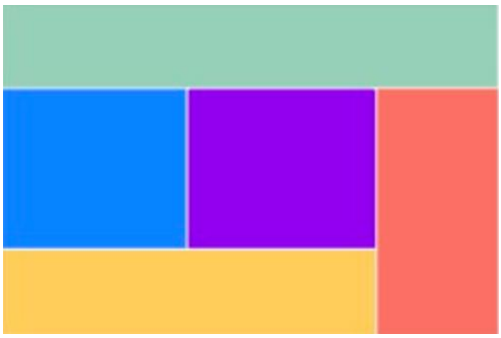
Нәтижесі:



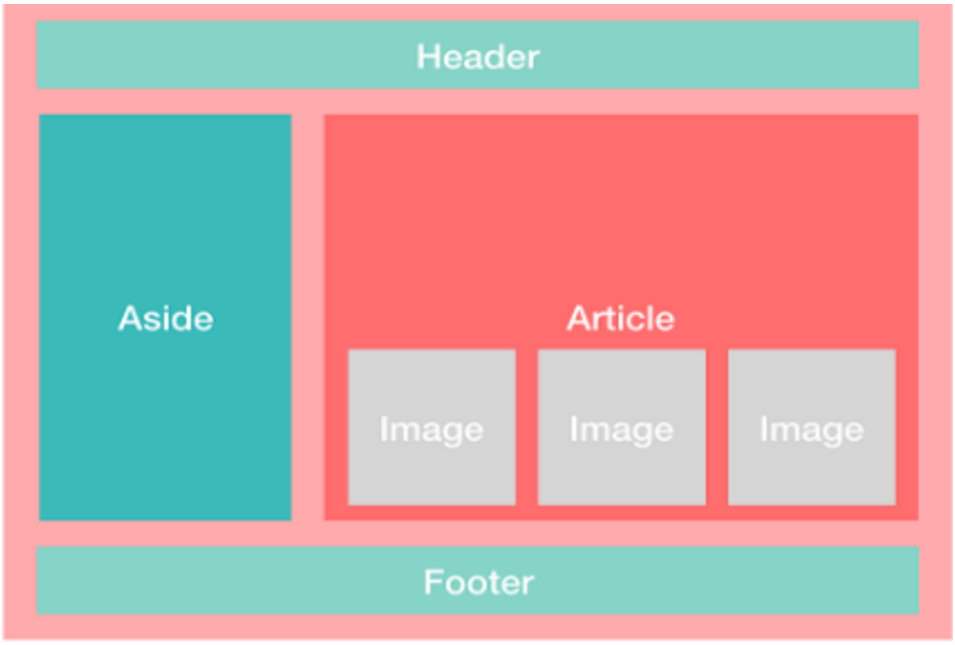
Сурет: CSS Grid

Зертханалық тапсырма - 1: CSS **Flexbox** және **Grid** технологияларын пайдаланып, төменде берілген дизайндарға ұқсас веб-беттерді құрастырыңыз.

1. 

2. 

Зертханалық тапсырма - 2: Төменде көрсетілген веб-бетке ұқсас веб-бет құрастырыңыз:


Header
Aside
Article
Image
Image
Image
Footer

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 10.
JAVASCRIPT-КЕ КІРІСПЕ

Теория: JavaScript - веб-беттерді интерактивті және динамикалық ететін танымал бағдарламалау тілі. Ол HTML элементтерін басқара алады, пайдаланушының батырманы басу сияқты әрекеттеріне жауап береді және бетті қайта жүктемей-ақ контентті жаңарта алады. JavaScript формаларды тексеру, анимациялар жасау және күрделі веб-қосымшаларды әзірлеу сияқты тапсырмалар үшін қажет. Ол тікелей браузерде жұмыс істейді, сондықтан заманауи веб-әзірлеудің маңызды құралы болып табылады.

Айнымалылар (Variables): Айнымалылар деректерді сақтау және өңдеу үшін қолданылады. JavaScript тілінде айнымалыларды атау кезінде **CamelCase** атау үлгісі қолданылады.

Мысал:

```
var myVariable = "Hello World";
```

Консоль (Console): Консоль кодтың орындалу нәтижесін көру үшін қолданылады.

Мысал:

```
var myVariable = "Hello World";
console.log(myVariable);
```

Деректер типтері (Data Types): JavaScript бірнеше деректер типін қамтиды, соның ішінде жолдық, сандық, логикалық, массивтік, объектілік және басқа да типтер бар.

Мысал:

```
var firstName = "John"; // Жолдық тип
var age = 25;           // Сандық тип
var isStudent = true;   // Логикалық тип

var fruits = ["apple", "banana", "orange"]; // Массив

var person = {
   firstName: "Jane",
   lastName: "Doe"
}; // Объект
```

Математикалық операторлар: Негізгі математикалық амалдарды +, -, *, / операторларының көмегімен орындауға болады.

Мысал:

```
var sum = 5 + 3;        // Қосу
var difference = 8 - 4; // Азайту
var product = 6 * 2;    // Көбейту
var quotient = 10 / 2;  // Бөлу
```

Математикалық әдістер: JavaScript тілінде кірістірілген математикалық әдістер бар.

Мысал:

```
var roundedNumber = Math.round(4.7);
```

```
// Санды бөлшексіз ең жақын бүтін санға дейін дөңгелектеу

var ceilingNumber = Math.ceil(4.3);
// Санды жоғары қарай ең жақын бүтін санға дейін дөңгелектеу

var flooringNumber = Math.floor(4.3);
// Санды төмен қарай ең жақын бүтін санға дейін дөңгелектеу

var randomValue = Math.random();
// 0 мен 1 аралығындағы кездейсоқ мәнді генерациялау

var maxValue = Math.max(4, 7, 12, 3);
// Берілген мәндердің ішінен ең үлкен мәнді алу

var minValue = Math.min(4, 7, 12, 3);
// Берілген мәндердің ішінен ең кіші мәнді алу
```

JavaScript тілінде **массив** - бір айнымалының ішінде бірнеше мәнді бір уақытта сақтай алатын объект.

Мысал:

```
var colors = ["red", "green", "blue"];

console.log(colors.length);
// "colors" массивінің ұзындығын алу

console.log(colors[1]);
// Индекс бойынша элементке қол жеткізу (нәтиже: "green")

console.log(colors.indexOf("green"));
// Элементтің индексін алу

colors.push("yellow");
// Массивтің соңына элемент қосу

var removedColor = colors.pop();
// Соңғы элементті жою және қайтару

var slicedColors = colors.slice(1, 3);
// Бастапқы массивтің бөлігін алу арқылы жаңа массив құру
```

Ескерту: JavaScript массивтерінің индексі 0-ден басталады.

Жол (String): Жолдар мәтінді білдіреді. Жолдарды әртүрлі тәсілдермен өңдеуге болады.

Мысал:

```
var message = "Hello";
var person = "John";
var greeting = message + " " + person;
// Жолдарды біріктіру
console.log(greeting.toLowerCase());
// Нәтиже → hello john
```

Шаблондық жолдар, сондай-ақ **шаблондық литералдар (Template Literals)** деп аталады. Бұл - жолдармен жұмыс істеуді ыңғайлы әрі оқуға жеңіл ететін JavaScript мүмкіндігі. Бір немесе қос тырнақшалардың орнына шаблондық жолдарда кері тырнақшалар ` ` қолданылады.

Мысал:

```
let person = "John";
let greeting = `Сәлем, ${person}!`;
console.log(greeting);
// Нәтиже: Сәлем, John!
```

Зертханалық тапсырма – 1

Өзіңіз таңдаған сандық мәндері бар екі айнымалыны - **num1** және **num2** - жариялайтын бағдарлама жазыңыз. Бағдарлама осы екі санның **қосындысын, айырмасын, көбейтіндісін және бөліндісін** есептеп, көрсетуі керек.

Зертханалық тапсырма – 2

Пайдаланушыдан сөйлем қабылдайтын JavaScript бағдарламасын жазыңыз. Содан кейін келесілерді көрсетіңіз:

A. Сөйлемнің ұзындығы.
Б. indexOf әдісін пайдаланып, сөйлемдегі белгілі бір сөздің (мысалы, **the**) алғаш кездесетін орнын анықтаңыз.
В. Сөйлемді кіші әріптермен көрсетіңіз.

Зертханалық тапсырма – 3

Пайдаланушының қазіргі жасын енгізу арқылы қабылдайтын және **90 жасқа дейін қалған уақытты** есептейтін JavaScript бағдарламасын жазыңыз.

Нәтижені келесі форматта көрсетіңіз:

→ X күн, Y апта және Z ай қалды.

Мұнда **X, Y және Z** нақты есептелген мәндермен алмастырылады.

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 11.
JAVASCRIPT ОПЕРАТОРЛАРЫ, ЦИКЛДЕРІ, ФУНКЦИЯЛАРЫ ЖӘНЕ ОБЪЕКТІЛЕРІ

Тәжірибенің тақырыбы: JavaScript тіліндегі операторларға, циклдерге және пайдаланушы функцияларына кіріспе.
Теория: Операторлар әртүрлі математикалық және логикалық есептеулерді орындау үшін қолданылады. Олар шартты операторлармен және циклдермен жұмыс істегенде өте ыңғайлы.

- < → Кіші
- >→ Үлкен
- == → Тең
- != → Тең емес
- <= → Кіші немесе тең
- >= → Үлкен немесе тең

JavaScript тіліндегі **шартты операторлар** белгілі бір шарттарға байланысты код блоктарын орындауға мүмкіндік береді. Егер шарт орындалса, белгілі бір код блогы іске қосылады, ал орындалмаса, басқа код блогы орындалады.
Мысал:

```
let x = 10;

if (x > 5) {
   console.log("x 5-тен үлкен");
} else if (x == 5) {
   console.log("x 5-ке тең");
} else {
   console.log("x 5-тен кіші");
}
```

Циклдер қайталанатын тапсырмаларды тиімді орындау үшін қажет. Олар берілген шарт орындалып тұрған кезде код блогын бірнеше рет орындайды. Циклдер тапсырмаларды автоматтандыруға және кодты оңтайландыруға мүмкіндік береді.
for циклі әдетте қайталану саны алдын ала белгілі болған жағдайда қолданылады. Ол үш бөліктен тұрады: инициализация, шарт және қайталану өрнегі.
Мысал:

```
for (let i = 1; i <= 5; i++) {
   console.log(i);
}
```

i - инициализацияны, i <= 5 - шартты, ал i++ - циклдің әр қайталануындағы өсімді білдіреді.
while циклі көбінесе қайталану саны алдын ала белгісіз болған жағдайда қолданылады.
Мысал:

```
let i = 1;

while (i <= 5) {
   console.log("Қайталану нөмірі: " + i);
   i++;
}
```

JavaScript тіліндегі **функция** - белгілі бір тапсырманы орындайтын код блогы. Ол function кілттік сөзі, функция атауы және параметрлерге арналған жақшалар арқылы анықталады. Параметрлер жақшаның ішінде көрсетіледі, ал функцияның денесі фигуралық жақшалардың { } ішінде жазылады.

Мысал:

```
function sayHello(name) {
   console.log(`Қайырлы таң, ${name}!`);
}

sayHello("Рахим");
```

Функцияларда функцияның қандай мән қайтаратынын көрсететін **return операторы** болуы мүмкін. return операторы функцияның орындалуын аяқтап, көрсетілген мәнді функция шақырылған жерге қайтарады.

Мысал:

```
function addNumbers(a, b) {
   let sum = a + b;
   return sum;
}

const result = addNumbers(10, 20);
console.log(result); // 30
```

JavaScript тіліндегі **объект** деректерді **кілт–мән (key-value)** жұптары арқылы сақтауға және ұйымдастыруға мүмкіндік береді. Әрбір кілт жол болып табылады, ал оған сәйкес мән кез келген деректер типінде болуы мүмкін.

Мысал:

```
let person = {
   fullName: "Мохаммед Надиб Хассан",
   age: 30,

   address: {
      area: "Мурадпур",
      district: "Читтагонг"
   }
};
```

Ескерту: объектінің кез келген қасиетін өзгертуге болады:

```
person.age = 35;
```

JavaScript массиві объектілерді де сақтай алады. **Объектілер массиві** өзара байланысты деректер жиынтығын көрсету үшін пайдалы.

Мысал:

```
var students = [
    { name: "Элис", age: 22, class: "A" },
    { name: "Боб", age: 24, class: "B" },
    { name: "Чарли", age: 23, class: "C" }
];
```

Зертханалық тапсырма – 1

Белгілі бір жылдың **кібісе жыл** екенін тексеретін бағдарлама жазыңыз. Бағдарлама енгізілген жылды қабылдап, оның кібісе жыл екенін немесе кібісе жыл емес екенін көрсететін хабарлама шығаруы керек.

Зертханалық тапсырма – 2

Берілген санның **көбейту кестесін** шығару үшін for циклін қолданатын бағдарлама жазыңыз. Пайдаланушыға көбейту кестесін көргісі келетін санды енгізуге мүмкіндік беріңіз.

Зертханалық тапсырма – 3

Шеңбердің радиусын аргумент ретінде қабылдап, оның ауданын қайтаратын **calculateArea** атты функция құрыңыз.

Зертханалық тапсырма – 4

Кафедраларыңызды сипаттайтын **объектілер массивін** құрыңыз. Әр объектіде departmentName, HOD (**кафедра меңгерушісі**) және студенттер массиві сияқты қасиеттер болуы керек.

Әрбір студент объектісінде name, studentID және currentSemester сияқты қасиеттер болуы керек.

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 12.
JS ТІЛІНДЕ DOM-МЕН ӨЗАРА ӘРЕКЕТТЕСУ ЖӘНЕ МАНИПУЛЯЦИЯ ЖАСАУ

Эксперимент атауы: JavaScript тілінде DOM-мен өзара әрекеттесу және манипуляция жасау.
Теория: Веб-бет жүктелген кезде браузер HTML-кодты оқиды және DOM деп аталатын ағашқа ұқсас құрылым жасайды. Әрбір HTML-элемент (мысалы, тақырыптар, абзацтар және батырмалар) осы ағашта объектіге айналады. JavaScript осы DOM-ағашына қол жеткізе алады және оны басқара алады. Ол элементтерді таба алады, олардың мазмұны мен стилін өзгерте алады, сондай-ақ жаңа элементтерді қоса алады. Бұл процесс веб-беттердің интерактивті болуына мүмкіндік береді. Мысалы, батырманы басқанда немесе форманы толтырғанда JavaScript беттегі көріністі жаңарту немесе деректерді серверге жіберу арқылы әрекет ете алады. Қорытындылай келе, DOM веб-беттің статикалық HTML-контенті мен JavaScript-тің динамикалық функционалдығы арасындағы көпір ретінде қызмет етеді. Ол JavaScript-ке веб-бетті оқуға және өзгертуге мүмкіндік береді, осылайша сайттарды интерактивті және жауап беретін етеді.



Сурет: HTML DOM ағашы
Элементтерді таңдау:

- getElementById() → элементті id бойынша таңдау.
- getElementsByTagName() → элементтерді тег атауы бойынша таңдау.
- getElementsByClassName() → элементтерді класс атауы бойынша таңдау.
- querySelector() → CSS-селектордың көмегімен жеке элементті таңдау.
- querySelectorAll() → бірдей CSS-селекторлары бар барлық элементтерді таңдау.

Мысал:
index.html ішінде:
<body>

```
<p id="message">Мысалы, әдепкі абзац...</p>

<ul>
   <li>Бірінші тармақ</li>
   <li>Екінші тармақ</li>
   <li>Үшінші тармақ</li>
</ul>

<div class="container">Бұл класс берілген div.</div>

<div id="parent">Бұл ата-аналық компонент</div>

<script src="script.js"></script>
<!-- html-файлды js-файлмен байланыстыру -->

</body>
```

script.js ішінде:

getElementById

```
var message = document.getElementById("message");

console.log(message);
// Енді ол id="message" мәні бар <p> элементін қамтиды
```

getElementsByTagName

```
var elementsByTagName = document.getElementsByTagName("li");

for (var i = 0; i < elementsByTagName.length; i++) {
   console.log(elementsByTagName[i].textContent);
}
```

getElementsByClassName

```
var elementsByClassName = document.getElementsByClassName("container");

for (var i = 0; i < elementsByClassName.length; i++) {
   console.log(elementsByClassName[i].textContent);
}
```

querySelector

```
var elementBySelector = document.querySelector("#parent");
console.log(elementBySelector.textContent);
```

Элементтермен манипуляция жасау:

- createElement() → жаңа элемент құру.
- appendChild() → белгілі бір ата-аналық түйінге еншілес түйінді қосу.
- textContent → түйіннің мәтіндік мазмұнын алу және орнату.

- innerHTML → элементтің HTML-мазмұнын алу және орнату.
- appendChild() → түйінді ата-аналық түйіннің соңғы еншілес түйінінен кейін кірістіру.
- removeChild() → түйіннің еншілес элементтерін жою.

Мысал:
document.createElement() көмегімен жаңа элементін құру:
var div = document.createElement("div");
Және div ішіне HTML-фрагментін қосу:
div.innerHTML = ' Жаңадан құрылған элемент ';
div элементін құжатқа тіркеу үшін appendChild() әдісін пайдаланыңыз:
document.body.appendChild(div);
Мысал:
var menu = document.getElementById("menu");
Жаңа li элементін құру:
var li = document.createElement("li");
li.innerText = "Негізгі бөлім";
Оны ul элементінің мәзіріне қосу:
menu.appendChild(li);

Элемент атрибуттары:

- setAttribute() → элементке берілген атрибуттың мәнін орнату.
- getAttribute() → элементтегі атрибуттың мәнін алу.
- removeAttribute() → берілген элементтен атрибутты жою.
- hasAttribute() → элементте белгілі бір атрибуттың бар-жоғын тексеру.

Мысал:
var myInput = document.getElementById("my-input");
myInput.setAttribute("type", "email");
Енгізу өрісі келесі түрге өзгереді: <input id="my-input" type="email" />

addEventListener() көмегімен оқиғалармен жұмыс істеу:

- change → HTML-элемент өзгерген кезде.
- click → пайдаланушы HTML-элементті басқан кезде.
- mouseover → пайдаланушы тінтуірді HTML-элементтің үстіне апарған кезде.
- keydown → пайдаланушы пернетақта пернесін басқан кезде.
- submit → пайдаланушы форма элементін жіберген кезде.
- load → браузер бетті жүктеуді аяқтаған кезде.

Мысал:
var btn = document.getElementById("btn");

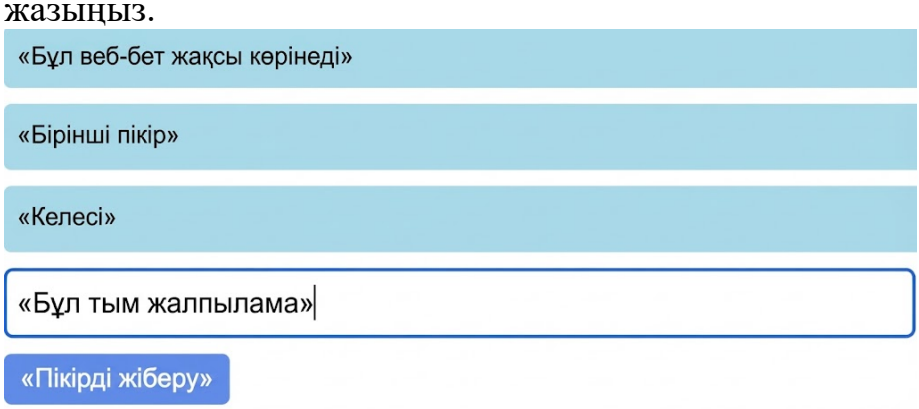
```
btn.addEventListener("click", function () {
   alert("Ол басты");
});
```

Стильдермен жұмыс істеу:
Біз JavaScript DOM манипуляциясын пайдаланып, элементтің стилін өзгерте аламыз.

Мысалы:
Element.style.[property_name] = 'мән';
Мысал:
var styledElement = document.getElementById("container");
styledElement.style.backgroundColor = "orange";

Студенттер көрсететін зертханалық эксперименттер

Зертханалық тапсырма: HTML және CSS көмегімен келесі интерфейсті жасаңыз, содан кейін бұрын қосылған пікірлер көрсетілетін және батырманы басу арқылы жаңа пікірлерді қосуға болатын пікірлер бөлімін құру үшін JavaScript бағдарламасын жазыңыз.



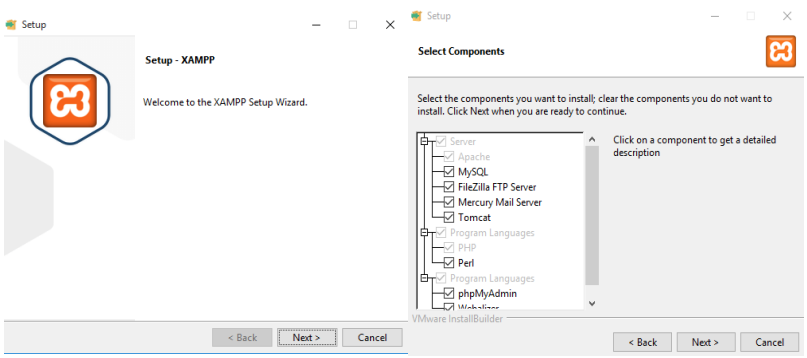
ЗЕРТХАНАЛЫҚ ЖҰМЫС № 13-14.

PHP ЖӘНЕ ВЕБ-СЕРВЕР: КІРІСПЕ ЖӘНЕ БАПТАУ

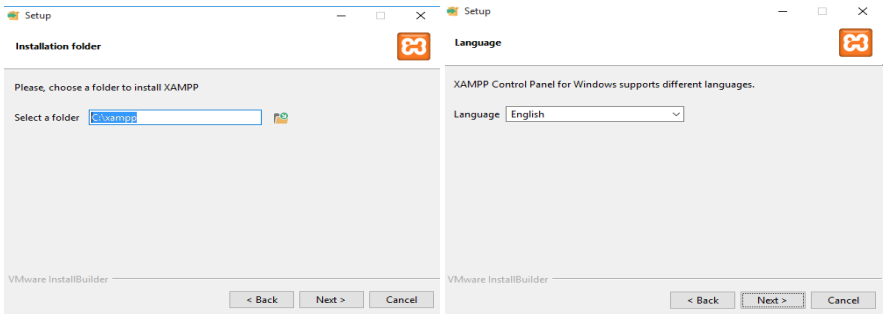
Эксперимент атауы: PHP-ке кіріспе және XAMPP серверін орнату.
Теория: PHP - веб-әзірлеуге бағытталған әмбебап скрипттік тіл. Оны 1993 жылы даниялық-канадалық бағдарламашы Расмус Лердорф жасаған және 1995 жылы шығарған. Қазіргі уақытта PHP-тің эталондық жүзеге асырылуын PHP Group әзірлейді. PHP бастапқыда **Personal Home Page** сөздерінің қысқартылған атауы болған, ал қазір ол рекурсивті инициализм болып табылатын **PHP: Hypertext Preprocessor** дегенді білдіреді. PHP - бэкенд үшін кеңінен қолданылатын ашық бастапқы кодты скрипттік тіл. PHP файлдарының кеңейтімі - **.php**.
Неліктен PHP үйренуіміз керек?

- PHP әртүрлі платформаларда жұмыс істейді (Windows, Linux, Unix, Mac OS X және т.б.).
- PHP қазіргі заманғы серверлердің барлығына дерлік үйлесімді (Apache, IIS және т.б.).
- PHP деректер базаларының кең ауқымын қолдайды.
- PHP тегін және ашық бастапқы кодты.
- PHP-ті үйрену оңай және ол сервер жағында тиімді жұмыс істейді.

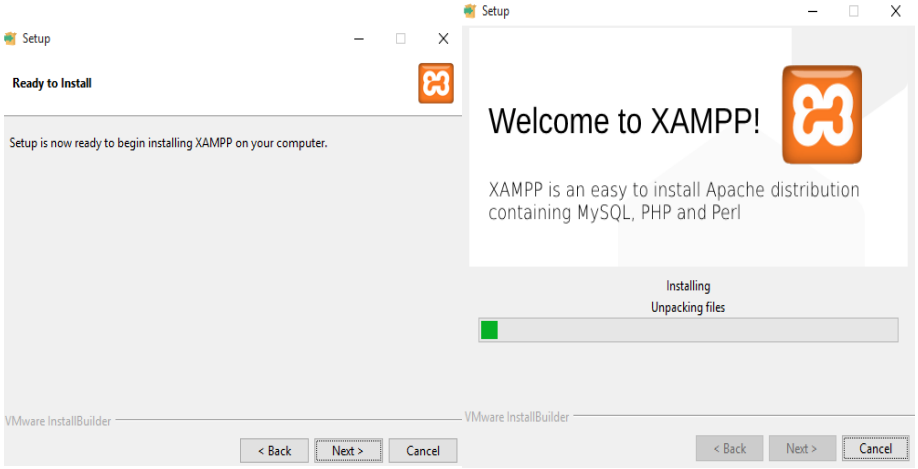
Веб-сервер: Веб-сервер - пайдаланушылардың желілік сұраныстарын өңдейтін және оларға веб-беттерді құрайтын файлдарды ұсынатын бағдарлама. Бұл алмасу гипермәтінді тасымалдау протоколы (**HTTP**) арқылы жүзеге асырылады. XAMPP - Apache Friends компаниясы әзірлеген тегін және ашық бастапқы кодты, кросс-платформалық веб-сервер шешімдерінің пакеті. Ол негізінен **Apache HTTP серверінен, MariaDB деректер базасынан және PHP мен Perl бағдарламалау тілдерінің скрипт интерпретаторларынан** тұрады және қолдануға оңай.
XAMPP орнату
1-қадам: XAMPP бағдарламасын https://www.apachefriends.org/download.html сайтынан жүктеп алыңыз.
2-қадам: Жүктелген .exe файлын екі рет басыңыз, содан кейін қалқымалы терезе пайда болады.
3-қадам: Нұсқауларға сәйкес **«Келесі» (Next)** батырмасын басуды жалғастырыңыз.



Сурет: XAMPP серверін орнату процесі

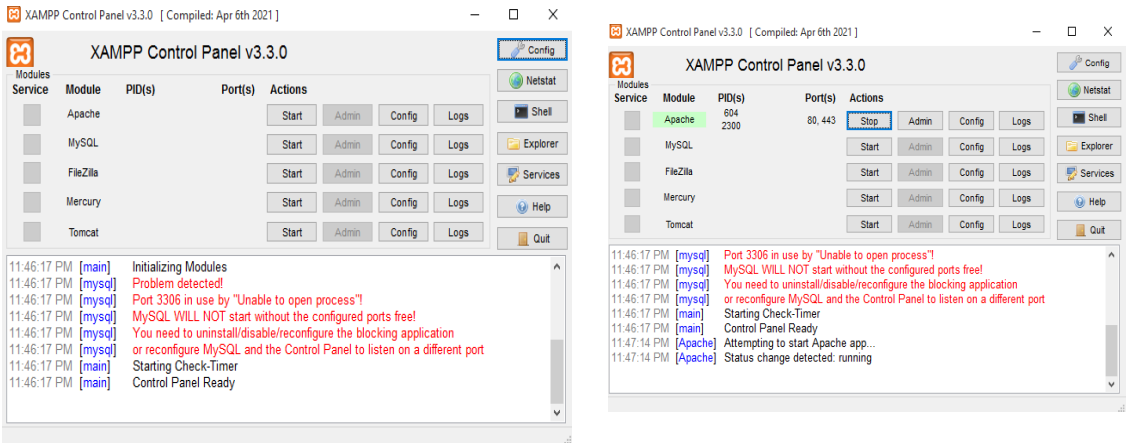


Сурет: XAMPP серверін орнату процесі



Сурет: XAMPP серверін орнату процесі

Орнату аяқталғаннан кейін XAMPP бағдарламалық жасақтамасын ашып, Apache серверін іске қосыңыз.



Сурет: XAMPP басқару панелі

XAMPP сервері сәтті іске қосылған кезде, XAMPP-тағы Apache ерекшеленіп көрсетіледі.

Содан кейін браузерге кіріп, URL өрісіне **http://localhost** деп жазыңыз, сонда сізге келесі интерфейс көрсетіледі.



Сурет: Басқару панелі - XAMPP сервері

[Ескерту: жаңа index.php файлын құрыңыз және оның ішінде HTML шаблонын дайындаңыз.]

PHP тіліндегі echo операторы браузерге мәтінді немесе айнымалыларды шығарады. Ол үтір арқылы бөлінген бірнеше аргументті қабылдай алады.

Мысал:

```
<!DOCTYPE html>
<html lang="kk">
<head>
    <title>PHP-ке кіріспе</title>
</head>
<body>
<?php
echo "ССЕ-ге қош келдіңіз!";
// Бұл жолды браузерге шығарады.
?>
</body>
</html>
```

Айнымалылар: PHP тіліндегі айнымалылар **$ белгісінен** басталады, одан кейін айнымалының атауы жазылады. Олар мәндерді сақтау үшін қолданылады.

Мысал: (<?php тегінің ішінде)

```
$name = "Джон Доу";
$age = 30;

echo "Менің атым $name, мен $age жастамын.";
Бұл форматталған жолды браузерге шығарады.
```

Жолдар: PHP тіліндегі жолдар - символдар тізбегі. Олар бір тырнақша ' немесе қос тырнақша " ішінде жазылуы мүмкін. Қос тырнақшадағы жолдар айнымалыларды интерполяциялауға мүмкіндік береді.

Мысал:

```
$greeting = "Сәлем";

echo '$greeting, әлем!';
```

```
// Нәтиже: $greeting, әлем!

echo "$greeting, әлем!";
// Нәтиже: Сәлем, әлем!
```

Сандар және арифметика: PHP бүтін сандарды, жылжымалы нүктелі сандарды және әртүрлі арифметикалық амалдарды қолдайды.

Мысал:

```
$x = 5;
$y = 3.14;

echo $x + $y; // Нәтиже: 8.14
echo $x - $y; // Нәтиже: 1.86
echo $x * $y; // Нәтиже: 15.7
echo $x / $y; // Нәтиже: 1.59236
echo $x % $y; // Қалдық бойынша нәтиже
```

Массивтер: PHP тіліндегі массивтер әртүрлі типтегі бірнеше мәнді сақтай алады. Олар [] квадрат жақшалары немесе array() функциясы арқылы құрылады.

Мысал:

```
$fruits = ["алма", "банан", "шие"];

echo $fruits[0];
// Нәтиже: алма
```

Массивті жариялаудың басқа тәсілі:

```
$ages = array(25, 30, 35);
```

Массивтің соңына элемент қосу:

```
array_push($fruits, "апельсин");
```

Индекс бойынша жаңа элемент қосу:

```
$fruits[3] = "жүзім";
```

unset() элементті оның индексі бойынша жояды:

```
unset($fruits[2]);
```

Ассоциативті массив - мәндерді сақтау және оларға қол жеткізу үшін сандық индекстердің орнына атаулы кілттерді пайдаланатын PHP массивінің түрі.

Мысал:

```
$person = array(
    "name" => "Джон Доу",
    "age" => 30,
    "city" => "Читтагонг"
);
echo $person["name"];
// Нәтиже: Джон Доу
```

Шартты операторлар:

PHP тіліндегі шартты операторлар белгілі бір шарттарға байланысты кодты орындауға мүмкіндік береді. Кең таралған түрлері - if, else, elseif және switch.

- **if** шарттың ақиқат екенін тексереді және ақиқат болса, өз блогындағы кодты орындайды.
- **else** if шарты жалған болған жағдайда кодты орындайды.
- **elseif** бірінші if шарты жалған болған жағдайда қосымша шарттарды тексереді.

Мысал:

```
<?php
$year = 2024;
if ($year % 400 == 0) {
   echo "$year - кібісе жыл.";
} elseif ($year % 100 == 0) {
   echo "$year - кібісе жыл емес.";
} elseif ($year % 4 == 0) {
   echo "$year - кібісе жыл.";
} else {
   echo "$year - кібісе жыл емес.";
}

?>
```

Циклдер: Басқа бағдарламалау тілдеріндегі сияқты, циклдер кодты бірнеше рет орындау үшін қолданылады. Циклдердің кең таралған түрлеріне for, while және foreach жатады.

Мысал:

PHP тіліндегі for циклі:

```
for ($i = 0; $i < 5; $i++) {
   echo "Қайталану $i\n";
}
```

PHP тіліндегі while циклі:

```
$i = 1;
while ($i <= 5) {
   echo "Нөмір: $i <br>";
   $i++;
}
```

PHP тіліндегі foreach циклі:

```
$colors = array("қызыл", "жасыл", "көк");
foreach ($colors as $color) {
   echo "Түс - $color\n";
}
```

Функция: PHP тіліндегі **пайдаланушы функциясы** - белгілі бір тапсырманы орындау үшін әзірлеуші жасаған функция. Ол қайталанатын тапсырмаларды болдырмау мақсатында пайдаланушы анықтайтын код блогын құру үшін қолданылады.

Мысал:

```
function greet($name) {
   return "Сәлем, $name!";
}
$message = greet("CCE бөліміне қош келдіңіз!");
echo $message;
```

PHP құрамында нақты тапсырмаларды орындау үшін скрипт ішінде тікелей шақыруға болатын **1000-нан астам кірістірілген функция** бар.

Мысал:

```
echo date("Y-m-d H:i:s");
```

Әдепкі функция ағымдағы күн мен уақытты көрсетеді.

```
$date = new DateTime('2024-09-30');
echo $date->format('d-m-Y');
// Нәтиже: 30-09-2024
```

Файлдармен жұмыс істеу: PHP файлдарды оқу және оларға жазу үшін функциялар ұсынады. fopen(), fread(), fwrite() және fclose() функциялары жиі қолданылады.

Мысал:

```
$file = fopen("example.txt", "w");
fwrite($file, "Сәлем, файл мазмұнын тексеру!");
fclose($file);
$content = file_get_contents("example.txt");
echo $content;
```

Нәтижесі:

Сәлем, файл мазмұнын тексеру!

Қателерді өңдеу: PHP тілінде қателерді өңдеу сенімді қосымшаларды әзірлеу үшін өте маңызды. PHP қателерді өңдеудің бірнеше тәсілін ұсынады, соның ішінде ерекшеліктер үшін try, catch және finally блоктары бар.

```
try {
   $result = 10 / 0;
   // Бұл «нөлге бөлу» қатесі
} catch (Exception $e) {
   echo "Ұсталған ерекшелік: " . $e->getMessage();
} finally {
   echo "Бұл блок әрқашан орындалады.";
}
```

Зертханалық тапсырма – 1

Келесі әрекеттерді орындайтын PHP скриптін құрыңыз:

- Екі айнымалыны анықтаңыз: біреуі сіздің атыңыз үшін (жол), екіншісі жасыңыз үшін (сан).
- Осы екі айнымалыны бір сөйлемге біріктіріңіз, мысалы: **«Менің атым ABC, мен 25 жастамын.»**
- Өзіңіздің сүйікті үш түсіңізден массив құрыңыз және әр түсті жаңа жолға шығарыңыз.

Зертханалық тапсырма – 2

Санның жұп немесе тақ екенін тексеретін PHP бағдарламасын жазыңыз. Егер сан 2-ге бөлінсе, **«Жұп»**, ал кері жағдайда **«Тақ»** деп шығарыңыз.

Зертханалық тапсырма – 3

Екі санды аргумент ретінде қабылдап, оларды қосып және қосындысын қайтаратын **addNumbers** атты PHP функциясын жазыңыз.

Зертханалық тапсырма – 4

for циклінің көмегімен **1-ден 10-ға дейінгі сандарды numbers.txt деп аталатын файлға** жаза алатын PHP бағдарламасын құрыңыз. Жазып болғаннан кейін файлды оқып, әрбір санды тексеріңіз. Егер сан жұп болса, **«X жұп»**, ал кері жағдайда **«X тақ»** деп шығарыңыз.

ЗЕРТХАНАЛЫҚ ЖҰМЫС № 15.
ДЕРЕКТЕРДІ ЖИНАУҒА ЖӘНЕ ОЛАРДЫ ДЕРЕКТЕР БАЗАСЫНДА САҚТАУҒА АРНАЛҒАН PHP ФОРМАСЫ

Жұмыс атауы: XAMPP серверін пайдаланып, PHP және MySQL көмегімен HTML-де тіркелу және жүйеге кіру формасын жасау.

Теория: Бэкенд немесе серверлік бөлік - пайдаланушылар көрмейтін бөлік. Ол серверді, қолданбаны және деректер базасын қамтиды. Бэкенд әзірлеу функционалдылыққа, логикаға және деректер базасымен орындалатын операцияларға бағытталған, бұл деректердің дұрыс және тиімді өңделуін қамтамасыз етеді. PHP интернеттегі барлық сайттардың шамамен 80%-ының бэкендін қамтамасыз ету үшін қолданылады, ал олардың шамамен 40%-ы PHP негізіндегі WordPress арқылы әзірленеді.

HTML тіркелу формасын жасау тәртібі

01-қадам: Жұмыс кеңістігін дайындау

- XAMPP бағдарламалық жасақтамасы орналасқан файлды ашыңыз.
- **htdocs** бумасын тауып, ашыңыз.
- Жаңа бума құрыңыз (егер оның атауы бірнеше сөзден тұрса, CamelCase, PascalCase қолданыңыз немесе екі сөздің арасына төменгі сызықша қойыңыз).
- Буманы VS Code бағдарламасында ашыңыз.

02-қадам: Жаңа деректер базасын құру

- XAMPP басқару панелінен phpMyAdmin-ді ашыңыз.
- **New** батырмасын басып, **user_db** деп аталатын деректер базасын құрыңыз.
- **user_db** деректер базасында келесі құрылымы бар **users** деп аталатын кесте құрыңыз:

Name	Type	Length/Values	Default
id	INT	11	None
fullname	VARCHAR	100	None
email	VARCHAR	255	None
phone	VARCHAR	100	None
password	VARCHAR	100	None

Сурет: Деректер базасы кестесінің құрылымы

03-қадам: Тіркелу формасы үшін жаңа **index.php** файлын құрыңыз.

```html
<!DOCTYPE html>
<html lang="kk">
<head>
    <title>Тіркеу формасы</title>
</head>
<body>
    <h2>Тіркеу формасы</h2>
    <form action="register.php" method="POST">
        <label for="fullname">Аты-жөні:</label>
        <input type="text" name="fullname" required><br><br>

        <label for="email">Электронды пошта:</label>
        <input type="email" name="email" required><br><br>

        <label for="phone">Телефон:</label>
        <input type="text" name="phone" required><br><br>

        <label for="password">Құпия сөз:</label>
        <input type="password" name="password" required><br><br>
        <input type="submit" name="submit" value="Қазір тіркелу">
    </form>
</body>
</html>
```

Нәтижесі:



Сурет: Пайдаланушыларды тіркеу беті

04-қадам: register.php деп аталатын тіркелу PHP-скриптін құрыңыз.

```php
<?php
$conn = mysqli_connect('localhost', 'root', 'password', 'user_db');

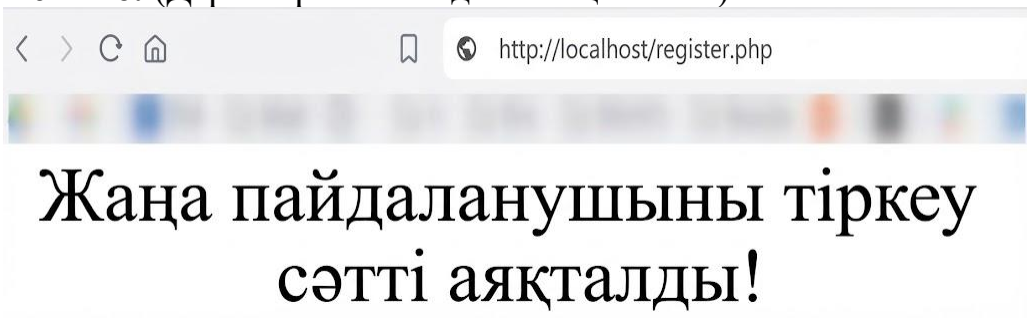
if (!$conn) {
    die("Қосылу сәтсіз аяқталды: " . mysqli_connect_error());
}

if (isset($_POST['submit'])) {
    $fullname = $_POST['fullname'];
    $email = $_POST['email'];
    $phone = $_POST['phone'];
    $password = $_POST['password'];

    $query = "INSERT INTO users (fullname, email, phone, password)
              VALUES ('$fullname', '$email', '$phone', '$password')";

    if (mysqli_query($conn, $query)) {
        echo "Жаңа пайдаланушы сәтті тіркелді!";
    } else {
        echo "Қате: " . mysqli_error($conn);
    }
}
?>
```

Нәтиже: (Деректер базасындағы жаңа жазба)



Сурет: Сәтті орындалғаны туралы хабарлама



Сурет: Деректер базасындағы жаңа жазба

05-қадам: Жүйеге кіру беті үшін жаңа **login.php** файлын құрыңыз:

```html
<!DOCTYPE html>
<html lang="kk">
<head>
    <title>Кіру формасы</title>
</head>
<body>
    <h2>Кіру формасы</h2>
    <form action="process_login.php" method="POST">
        <label for="email">Электронды пошта:</label>
        <input type="email" name="email" required><br><br>

        <label for="password">Құпия сөз:</label>
        <input type="password" name="password" required><br><br>

        <button type="submit" name="login">Қазір кіру</button>
    </form>
</body>
</html>
```

05-қадам: Жүйеге кіру үшін **process_login.php** деп аталатын PHP-скриптін құрыңыз.

```php
<?php
$conn = mysqli_connect('localhost', 'root', 'password', 'user_db');

if (!$conn) {
    die("Қосылу сәтсіз аяқталды: " . mysqli_connect_error());
}

if (isset($_POST['login'])) {
    $email = $_POST['email'];
    $password = $_POST['password'];

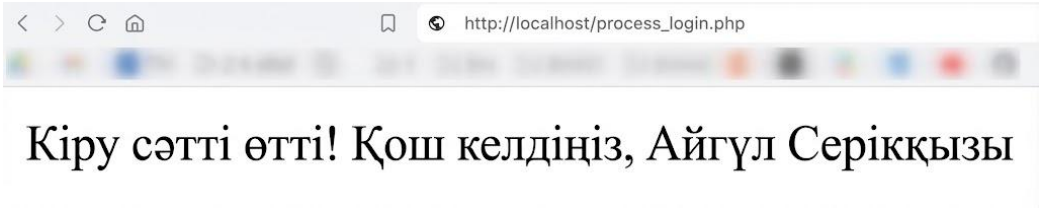
    $query = "SELECT * FROM users WHERE email='$email' AND password='$password'";
    $result = mysqli_query($conn, $query);

    if (mysqli_num_rows($result) == 1) {
        $user = mysqli_fetch_assoc($result);
        echo "Кіру сәтті өтті! Қош келдіңіз, " . $user['fullname'];
    } else {
        echo "Электронды пошта немесе құпия сөз қате!";
    }
}
?>
```

Нәтижесі:



Сурет: Жүйеге кіру бетінің интерфейсі



Сурет: Жүйеге сәтті кіру туралы хабарлама

Зертханалық тапсырма: Студенттер өздерінің аты-жөнін, электрондық поштасын, телефон нөмірін, студенттік билетін, кафедрасын және құпиясөзін пайдаланып тіркеле алатын PHP негізіндегі жүйе құрыңыз. Тіркелгеннен кейін студенттер жүйеге кіріп, өздерінің барлық деректерін HTML-кестеде көре алады.

Баған атауы	Дерек түрі	Ұзындығы	Қосымша
id	INT	11	AUTO_INCREMENT, Негізгі кілт
student_name	VARCHAR	100	Бос емес
email	VARCHAR	100	Бос емес, Бірегей
phone	VARCHAR	15	Бос емес
student_id	VARCHAR	20	Бос емес, Бірегей
department	VARCHAR	100	Бос емес
password	VARCHAR	255	Бос емес (хештелген)

Сурет: Деректер базасындағы «студенттер» кестесінің дизайны

ҚАЗАҚСТАН РЕСПУБЛИКАСЫ АУЫЛ ШАРУАШЫЛЫҒЫ МИНИСТРЛІГІ
«С.СЕЙФУЛЛИН атындағы ҚАЗАҚ АГРОТЕХНИКАЛЫҚ ЗЕРТТЕУ УНИВЕРСИТЕТІ» КеАҚ

«WEB БАҒДАРЛАМАЛАУ»

БІЛІМ АЛУШЫНЫҢ ӨЗІНДІК ЖҰМЫСЫНА АРНАЛҒАН ТАПСЫРМАЛАР.

Астана, 2026ж

СӨЖ №1.
WEB-БАҒДАРЛАМАЛАУ НЕГІЗДЕРІ. WEB СТАНДАРТТАРЫ ЖӘНЕ ОЛАРДЫҢ ДАМУЫ

Мақсаты: Web-бағдарламалаудың негізгі ұғымдарын, Web технологиялардың даму кезеңдерін және стандарттардың маңызын меңгеру.
Оқыту нәтижелері: білім алушы Web, WWW, Web-сайт, Web-қосымша ұғымдарын ажыратады; HTML, CSS, JavaScript қызметтерін түсіндіреді; Web стандарттарының қажеттілігін негіздейді.
Тапсырмалар:

1. Web-бағдарламалау ұғымына анықтама беріңіз.
2. Интернет пен WWW арасындағы айырмашылықты түсіндіріңіз.
3. Web технологиялардың даму кезеңдерін уақыт шкаласы түрінде көрсетіңіз.
4. HTML, CSS және JavaScript технологияларын салыстырыңыз.
5. Web стандарттары не үшін қажет екенін түсіндіріңіз.
6. Web 1.0, Web 2.0 және Web 3.0 ерекшеліктерін салыстырыңыз.

Практикалық тапсырма: «Web технологиялардың даму тарихы» тақырыбында уақыт шкаласын немесе инфографика дайындаңыз.
Бақылау сұрақтары: Web дегеніміз не? WWW дегеніміз не? Web-сайт пен Web-қосымшаның айырмашылығы қандай? HTML қандай қызмет атқарады? CSS не үшін қолданылады? JavaScript-тің міндеті қандай? Web стандарттарының маңызы неде?
Есеп беру: 3–5 бет есеп, салыстырмалы кесте және қорытынды.

СӨЖ №2.
ИНТЕРНЕТ КОММУНИКАЦИЯСЫНЫҢ МОДЕЛІ. PULL ЖӘНЕ PUSH МОДЕЛЬДЕРІ

Мақсаты: Интернеттегі ақпарат алмасудың негізгі модельдерін зерттеу.
Тапсырмалар:

1. Интернет коммуникациясының моделін сипаттаңыз.
2. Клиент-сервер байланысының схемасын жасаңыз.
3. Pull моделінің жұмыс принципін түсіндіріңіз.
4. Push моделінің ерекшеліктерін көрсетіңіз.
5. Pull және Push модельдерін салыстырыңыз.
6. Әр модельге кемінде үш практикалық мысал келтіріңіз.

Практикалық тапсырма: келесі кестені толтырыңыз:

Критерий	**Pull**	**Push**
Сұраныста кім бастайды?		
Деректерді жеткізу тәсілі		
Мысал		
Артықшылығы		
Кемшілігі		

Жеке тапсырма: әлеуметтік желі, электрондық пошта немесе жаңалықтар сервисінің біреуін таңдап, онда Push/Pull механизмдерінің қолданылуын талдаңыз.
Бақылау сұрақтары: коммуникация дегеніміз не? Клиент дегеніміз не? Сервер дегеніміз не? Pull және Push модельдерінің негізгі айырмашылығы қандай?

СӨЖ №3. Қолданушының Интернетке кіруі. «Соңғы миля». Модем

Мақсаты: пайдаланушының Интернетке қосылу технологияларын зерттеу.
Тапсырмалар: «Соңғы миля» ұғымына анықтама беріңіз; ISP қызметін сипаттаңыз; модем мен маршрутизатордың айырмашылығын анықтаңыз; каналдарды коммутациялау принципін түсіндіріңіз.
Практикалық тапсырма: келесі байланыстың схемасын құрыңыз:
Компьютер/смартфон
↓
Wi-Fi / Ethernet
↓
Маршрутизатор
↓
Модем/ONT
↓
Провайдер
↓
Интернет
↓
Web-сервер
Жеке тапсырма: өзіңіз білетін Интернетке қосылу тәсілдерінің артықшылықтары мен кемшіліктерін салыстырыңыз.
Бақылау сұрақтары: ISP дегеніміз не? Модемнің қызметі қандай? Маршрутизатор не үшін қажет? «Соңғы миля» нені білдіреді?

СӨЖ №4. Интернет жүйесіне қатынасу технологиялары. DSL және ISDN

Мақсаты: Интернетке қатынау технологияларының ерекшеліктерін салыстыру.
Тапсырмалар: DSL, ADSL, ISDN, кабельдік, мобильді және оптикалық Интернет туралы ақпарат жинаңыз.
Кестені толтырыңыз:

Технология	Тасымалдау ортасы	Жылдамдық ерекшелігі	Артықшылығы	Кемшілігі
ISDN				
DSL				
ADSL				

Технология	Тасымалдау ортасы	Жылдамдық ерекшелігі	Артықшылығы	Кемшілігі
Fiber				
4G/5G				

Зерттеу сұрағы: DSL және ISDN технологияларының қазіргі жоғары жылдамдықты желілерден қандай айырмашылығы бар?

СӨЖ №5. Интернеттегі Web-серверлер

Тақырып: Web-сервер жұмысының механизмі. Статикалық және динамикалық беттер. CGI технологиясы. Скрипттер.
Мақсаты: Web-сервердің жұмыс принципін және статикалық/динамикалық Web-беттердің айырмашылығын зерттеу.
Тапсырмалар:

1. Web-серверге анықтама беріңіз.
2. Клиент-сервер архитектурасын түсіндіріңіз.
3. HTTP сұранысы мен жауабының қызметін көрсетіңіз.
4. Статикалық және динамикалық Web-беттерді салыстырыңыз.
5. CGI технологиясының қызметін зерттеңіз.
6. Клиенттік және серверлік скрипттердің айырмашылығын көрсетіңіз.

Практикалық тапсырма: Web-сервер жұмысын схема түрінде көрсетіңіз:
Браузер
↓
HTTP Request
↓
Web-сервер
↓
Қосымша / файл / деректер қоры
↓
HTTP Response
↓
Браузер
Жеке тапсырма: бір статикалық және бір динамикалық сайтты таңдап, олардың ерекшеліктерін салыстырыңыз.

СӨЖ №6. Web-ресурстарды алу және Web-парақтарды кэштеу

Мақсаты: Web-ресурстарды алу, кэштеу және прокси-сервер механизмдерін зерттеу.
Тапсырмалар:

1. URL енгізілгеннен кейін Web-парақтың ашылу кезеңдерін сипаттаңыз.
2. DNS қызметін түсіндіріңіз.
3. Web-кэш ұғымын анықтаңыз.

4. Браузерлік кэштің жұмыс принципін түсіндіріңіз.
5. Прокси-сервердің қызметін сипаттаңыз.
6. Cache-Control, Expires, ETag ұғымдарын зерттеңіз

Практикалық тапсырма: браузерден F12 → **Network** құралын ашыңыз. Бір Web-сайтты екі рет жүктеп, жүктелу уақыттарын салыстырыңыз.

Көрсеткіш	**1-жүктеу**	**2-жүктеу**
Сұраныстар саны		
Деректер көлемі		
Жүктелу уақыты		
Кэштен алынған файлдар		

Қорытынды: кэштеу Web-сайттың жылдамдығына қалай әсер ететінін түсіндіріңіз.

СӨЖ №7. ИНТЕРНЕТТІҢ ІЗДЕУ ТЕХНОЛОГИЯЛАРЫ

Мақсаты: іздеу жүйелерінің Web-ресурстарды табу және индекстеу механизмдерін меңгеру.

Негізгі тапсырма: келесі процесті зерттеңіз:

Crawler → Web-парақты табу → Сканерлеу → Индекстеу → Ranking → Іздеу нәтижесі.

Практикалық тапсырма: бір ғылыми немесе кәсіби тақырыпты таңдап, үш түрлі іздеу сұранысын орындаңыз. Қолданылған кілт сөздердің нәтижелерге әсерін салыстырыңыз.

Зерттеу: SEO, robots.txt, sitemap және meta description ұғымдарының іздеу жүйелері үшін маңызын түсіндіріңіз.

СӨЖ №8. ЭЛЕКТРОНДЫҚ ПОШТА СЕРВИСІ

Мақсаты: E-mail сервисінің архитектурасы мен хаттамаларын зерттеу.

Тапсырмалар: SMTP, POP3 және IMAP хаттамаларының қызметін анықтап, салыстырыңыз.

Жіберуші
↓
E-mail клиент
↓
SMTP сервер
↓
Интернет
↓
Қабылдаушының mail-сервері
↓
IMAP / POP3
↓

Алушы
Жеке тапсырма: SMTP, POP3 және IMAP бойынша салыстырмалы кесте құрыңыз.

СӨЖ №9. FTP СЕРВИСІ ЖӘНЕ ФАЙЛДАРМЕН АЛМАСУ

Мақсаты: Интернет арқылы файлдарды тасымалдау технологияларын меңгеру.
Тапсырмалар: FTP, FTP сервері, FTP клиенті, SFTP ұғымдарын түсіндіріңіз. FTP арқылы файлды жүктеу және серверге жіберу механизмін схема түрінде көрсетіңіз.
Салыстыру: FTP, SFTP және HTTP арқылы файл тасымалдаудың ерекшеліктерін кестеге енгізіңіз.
Практикалық бөлім: FTP-клиент бағдарламасының интерфейсін зерттеп, файлдарды жүктеу/жіберу кезеңдерін сипаттаңыз.

СӨЖ №10. ИНТЕРНЕТТЕГІ МӘЛІМЕТТЕРДІ ҚОРҒАУ

Мақсаты: Web ортадағы криптографиялық қорғаныс механизмдерін меңгеру.
Тапсырмалар: криптография, шифрлау, хэштеу, симметриялық және асимметриялық криптография, ашық және жабық кілт, электрондық цифрлық қолтаңба ұғымдарын түсіндіріңіз.
Практикалық тапсырма: ЭЦҚ механизмін схема түрінде көрсетіңіз:
Құжат
↓
Хэш-функция
↓
Хэш
↓
Жабық кілтпен қол қою
↓
Цифрлық қолтаңба
Талдау: фишинг, зиянды бағдарламалар, әлсіз парольдер, деректердің ұрлануы сияқты Web қауіптерді және олардан қорғану тәсілдерін қарастырыңыз.

СӨЖ №11. COOKIES МЕХАНИЗМІ

Мақсаты: Web-қосымшаларда пайдаланушыны идентификациялау механизмдерін зерттеу.
Тапсырмалар: Cookie, Session Cookie, Persistent Cookie ұғымдарына анықтама беріңіз. Cookies-тің авторизация, баптауларды сақтау және аналитикадағы қызметін түсіндіріңіз.
Практикалық тапсырма: браузердің Developer Tools құралындағы **Application/Storage → Cookies** бөлімін зерттеңіз.
Мына мәліметтерді анықтаңыз:

- Name;
- Value;

- Domain;
- Path;
- Expires;
- Secure;
- HttpOnly;
- SameSite.

Қорытынды: Cookies технологиясының артықшылықтары мен құпиялыққа қатысты кемшіліктерін талдаңыз.

СӨЖ №12. HTML МУЛЬТИМЕДИА ЖӘНЕ ФОРМАЛАР

Мақсаты: HTML көмегімен мультимедиалық және интерактивті Web-парақ жасау.
Практикалық тапсырма: «Студентті тіркеу» Web-парағын жасаңыз.
Парақта:

- тақырып;
- сурет;
- аудио немесе видео;
- аты-жөні;
- e-mail;
- телефон;
- мамандығы;
- курс;
- radio;
- checkbox;
- select;
- Submit батырмасы

болуы тиіс.
Қолданылатын негізгі элементтер:
<img>
<audio>
<video>
<iframe>
<form>
<input>
<select>
<textarea>
<button>

СӨЖ №13. CSS – СТИЛЬДЕРДІҢ САТЫЛЫ КЕСТЕЛЕРІ

Мақсаты: CSS көмегімен Web-парақ интерфейсін безендіру және элементтерді орналастыру.
Тапсырмалар: Inline, Internal және External CSS тәсілдерін салыстырыңыз. Selector, property және value түсініктерін зерттеңіз.

Практикалық тапсырма: №12 СӨЖ-де құрылған Web-парақты CSS арқылы безендіріңіз.
Міндетті түрде:
color
background
font-size
font-family
width
height
margin
padding
border
position
z-index
қасиеттерінің бірқатарын қолданыңыз.
Қосымша тапсырма: CSS Box Model сызбасын жасап, content → padding → border → margin байланысын түсіндіріңіз.

СӨЖ №14. JavaScript тілі

Мақсаты: JavaScript арқылы Web-параққа интерактивті функционал қосу.
Тапсырмалар: айнымалылар, деректер типтері, операторлар, шарттар, циклдер, массивтер, функциялар, объектілер және оқиғаларды зерттеңіз.
Практикалық тапсырма: пайдаланушының бағасын анықтайтын бағдарлама жасаңыз.

```
let score = 85;

if (score >= 90) {
   console.log("Өте жақсы");
} else if (score >= 75) {
   console.log("Жақсы");
} else if (score >= 50) {
   console.log("Қанағаттанарлық");
} else {
   console.log("Қанағаттанарлықсыз");
}
```

Одан кейін HTML батырмасына click оқиғасын қосып, пайдаланушы әрекетіне қарай Web-парақтағы мәтінді өзгертіңіз.
Жеке тапсырма: қарапайым калькулятор, тест, баға есептеу немесе To-Do List қосымшасының бірін әзірлеңіз.

СӨЖ №15. WEB-ҚОСЫМШАЛАР ИНТЕРФЕЙСІ. HTTP ЖӘНЕ PHP НЕГІЗДЕРІ

Мақсаты: клиент-серверлік Web-қосымшалардың жұмыс принципін және серверлік бағдарламалау негіздерін меңгеру.

Тапсырмалар:

1. HTTP хаттамасының қызметін сипаттаңыз.
2. HTTP Request және Response түсініктерін зерттеңіз.
3. GET және POST әдістерін салыстырыңыз.
4. PHP тілінің ерекшеліктерін түсіндіріңіз.
5. PHP айнымалылары мен операторларын қарастырыңыз.

Практикалық тапсырма: HTML форма құрып, енгізілген ақпаратты PHP көмегімен өңдеңіз.

```
<?php
$name = $_POST["name"];
echo "Қош келдіңіз, " . $name;
?>
```

Қорытынды тапсырма: HTML + CSS + JavaScript + PHP технологияларын біріктіріп, қарапайым **«Студенттерді тіркеу жүйесі»** Web-қосымшасының құрылымын жобалаңыз.

ҚАЗАҚСТАН РЕСПУБЛИКАСЫ АУЫЛ ШАРУАШЫЛЫҒЫ МИНИСТРЛІГІ
«С.СЕЙФУЛЛИН атындағы ҚАЗАҚ АГРОТЕХНИКАЛЫҚ ЗЕРТТЕУ УНИВЕРСИТЕТІ» КеАҚ

«WEB-БАҒДАРЛАМАЛАУ»

АҒЫМДЫҚ ЖӘНЕ ҚОРЫТЫНДЫ БАҚЫЛАУҒА АРНАЛҒАН МАТЕРИАЛДАР.

\

Астана, 2026ж

1-ТАҚЫРЫП. WEB-БАҒДАРЛАМАЛАУ НЕГІЗДЕРІ. WEB СТАНДАРТТАРЫ ЖӘНЕ ОЛАРДЫҢ ДАМУЫ

Бақылау сұрақтары

1. Web-бағдарламалау дегеніміз не?
2. Интернет және World Wide Web ұғымдарының айырмашылығы қандай?
3. Web-сайт дегеніміз не?
4. Web-сайт пен Web-қосымшаның айырмашылығы қандай?
5. Web-парақ дегеніміз не?
6. Web-браузер қандай қызмет атқарады?
7. Web-сервер дегеніміз не?
8. Клиент-сервер архитектурасының жұмыс принципі қандай?
9. HTML, CSS және JavaScript қандай қызмет атқарады?
10. Web стандарттары не үшін қажет?
11. W3C ұйымының Web технологияларындағы рөлі қандай?
12. Web 1.0 және Web 2.0 арасындағы айырмашылықтарды атаңыз.

Практикалық тапсырмалар

1. Клиент-сервер архитектурасының схемасын құрыңыз.
2. HTML, CSS және JavaScript технологияларын қызметтері бойынша кестеде салыстырыңыз.
3. Қарапайым HTML құжатын құрып, браузерде іске қосыңыз.
4. Web технологиялардың даму кезеңдерін уақыт шкаласы түрінде көрсетіңіз.

2-ТАҚЫРЫП. ИНТЕРНЕТ КОММУНИКАЦИЯСЫНЫҢ МОДЕЛІ. PULL ЖӘНЕ PUSH МОДЕЛЬДЕРІ

Бақылау сұрақтары

1. Интернет коммуникациясы дегеніміз не?
2. Клиент пен сервердің міндеттері қандай?
3. Pull моделі дегеніміз не?
4. Push моделі дегеніміз не?
5. Pull моделінде коммуникацияны кім бастайды?
6. Push моделінде ақпарат қалай жеткізіледі?
7. Pull моделіне мысал келтіріңіз.
8. Push моделіне мысал келтіріңіз.
9. Push технологиясының артықшылықтары қандай?
10. Pull және Push модельдерінің айырмашылығы неде?

Практикалық тапсырмалар

1. Pull моделінің жұмыс схемасын салыңыз.
2. Push моделінің жұмыс схемасын салыңыз.
3. Электрондық пошта, жаңалықтар сайты және хабарлама жүйесін Pull/Push тұрғысынан талдаңыз.
4. Pull және Push модельдерін салыстыратын кесте жасаңыз.

3-тақырып. Қолданушының Интернетке кіруі. «Соңғы миля». Модем

Бақылау сұрақтары

1. Интернетке қосылу дегеніміз не?
2. Интернет провайдері дегеніміз не?
3. ISP қандай қызмет атқарады?
4. «Соңғы миля» ұғымы нені білдіреді?
5. Модем дегеніміз не?
6. Модемнің негізгі қызметі қандай?
7. Маршрутизатор дегеніміз не?
8. Модем мен маршрутизатордың айырмашылығы қандай?
9. Каналдарды коммутациялау дегеніміз не?
10. Пайдаланушы құрылғысы Интернетке қандай кезеңдер арқылы қосылады?

Практикалық тапсырмалар

1. **Компьютер → маршрутизатор → провайдер → Интернет → Web-сервер** схемасын жасаңыз.
2. Модем мен маршрутизаторды қызметтері бойынша салыстырыңыз.
3. Үй желісінің қарапайым құрылымдық схемасын құрыңыз.
4. Кабельдік және сымсыз Интернет қосылымдарын салыстырыңыз.

4-тақырып. Интернетке қатынасу технологиялары. DSL және ISDN

Бақылау сұрақтары

1. Интернетке қатынасу технологиясы дегеніміз не?
2. DSL дегеніміз не?
3. ADSL дегеніміз не?
4. DSL қалай жұмыс істейді?
5. ISDN дегеніміз не?
6. DSL және ISDN арасындағы айырмашылық қандай?
7. Оптикалық Интернеттің ерекшелігі қандай?
8. Мобильді Интернет қалай жұмыс істейді?
9. Интернет жылдамдығына қандай факторлар әсер етеді?
10. Қазіргі уақытта DSL/ISDN орнына қандай технологиялар кең қолданылады?

Практикалық тапсырма

Кестені толтырыңыз:

Технология	Байланыс ортасы	Артықшылығы	Кемшілігі
ISDN			
DSL			
ADSL			
Fiber			
4G/5G			

Қай технологияны қазіргі Web-қосымшалар үшін тиімді деп есептейтініңізді негіздеңіз.

5-тақырып. Интернеттегі Web-серверлер. Статикалық және динамикалық беттер. CGI

Бақылау сұрақтары

1. Web-сервер дегеніміз не?
2. Web-сервер қандай қызмет атқарады?
3. Браузер мен Web-сервер қалай байланысады?
4. Статикалық Web-парақ дегеніміз не?
5. Динамикалық Web-парақ дегеніміз не?
6. Статикалық және динамикалық парақтардың айырмашылығы қандай?
7. Серверлік сценарий дегеніміз не?
8. CGI дегеніміз не?
9. CGI технологиясының қызметі қандай?
10. Динамикалық Web-парақтар қандай жағдайда қолданылады?

Практикалық тапсырмалар

1. Статикалық index.html парағын жасаңыз.
2. Статикалық және динамикалық сайттардың жұмыс схемаларын салыңыз.
3. Web-сервердің сұранысты өңдеу алгоритмін схема түрінде көрсетіңіз.
4. Браузердің Developer Tools құралынан бір HTTP сұранысын тауып, оның негізгі параметрлерін сипаттаңыз.

6-тақырып. Web-ресурстарды алу. Кэштеу және прокси-сервер

Бақылау сұрақтары

1. Web-ресурс дегеніміз не?
2. URL қандай қызмет атқарады?
3. Браузер Web-ресурсты қалай алады?
4. DNS қандай қызмет атқарады?
5. Web-кэш дегеніміз не?
6. Кэштеу не үшін қажет?
7. Браузерлік кэш қалай жұмыс істейді?
8. Cache-Control дегеніміз не?
9. Expires қандай қызмет атқарады?
10. ETag дегеніміз не?
11. Прокси-сервер дегеніміз не?
12. Прокси-сервердің негізгі қызметтері қандай?
13. Кэштеу Web-парақтың жылдамдығына қалай әсер етеді?

Практикалық тапсырмалар

1. Браузерде **F12 → Network** бөлімін ашыңыз.
2. Бір сайтты бірінші рет жүктеп, уақытын белгілеңіз.
3. Сол парақты қайта жүктеп, нәтижесін салыстырыңыз.
4. Кэштен алынған ресурстарды анықтаңыз.
5. **Браузер → Proxy → Web-сервер** схемасын құрыңыз.

7-тақырып. Интернеттің іздеу технологиялары

Бақылау сұрақтары

1. Іздеу жүйесі дегеніміз не?
2. Web crawler дегеніміз не?
3. Web-парақтарды индекстеу дегеніміз не?
4. Іздеу индексі қандай мақсатта құрылады?
5. Ranking дегеніміз не?
6. Кілт сөз дегеніміз не?
7. SEO дегеніміз не?
8. robots.txt қандай қызмет атқарады?
9. Sitemap дегеніміз не?
10. Іздеу нәтижелерінің реті қандай факторларға байланысты болуы мүмкін?

Практикалық тапсырмалар

1. Бір тақырыпты үш түрлі кілт сөзбен іздеп, нәтижелерді салыстырыңыз.
2. site: іздеу операторын қолданып көріңіз.
3. Іздеу жүйесінің **Crawler → Index → Ranking → Result** схемасын жасаңыз.
4. Қарапайым Web-параққа title және meta description жазыңыз.

8-ТАҚЫРЫП. ЭЛЕКТРОНДЫҚ ПОШТА ЖӘНЕ E-MAIL ХАТТАМАЛАРЫ

Бақылау сұрақтары

1. Электрондық пошта дегеніміз не?
2. E-mail мекенжайының құрылымы қандай?
3. SMTP дегеніміз не?
4. SMTP қандай қызмет атқарады?
5. POP3 дегеніміз не?
6. IMAP дегеніміз не?
7. POP3 пен IMAP айырмашылығы қандай?
8. Mail-сервер дегеніміз не?
9. Электрондық хат серверлер арасында қалай жеткізіледі?
10. Электрондық поштадағы спам және фишинг дегеніміз не?

Практикалық тапсырмалар

1. Электрондық хаттың жіберушіден алушыға жету схемасын жасаңыз.
2. SMTP, POP3 және IMAP хаттамаларын салыстырыңыз.
3. Электрондық хаттың негізгі өрістерін анықтаңыз: From, To, Subject, CC, BCC.
4. Фишингтік хаттың белгілерін анықтайтын қысқаша ереже құрыңыз.

9-ТАҚЫРЫП. FTP СЕРВИСІ ЖӘНЕ ФАЙЛДАРМЕН АЛМАСУ

Бақылау сұрақтары

1. FTP дегеніміз не?
2. FTP қандай мақсатта қолданылады?
3. FTP сервер дегеніміз не?

4. FTP клиент дегеніміз не?
5. FTP арқылы файл қалай жіберіледі?
6. Upload және Download дегеніміз не?
7. FTP және SFTP айырмашылығы қандай?
8. FTP-дің қауіпсіздік мәселелері қандай?
9. FTP клиентінің қызметі қандай?
10. FTP мен HTTP арасындағы айырмашылық қандай?

Практикалық тапсырмалар

1. FTP клиентінің интерфейсін зерттеңіз.
2. FTP арқылы файл жіберудің алгоритмін құрыңыз.
3. FTP, SFTP және HTTP технологияларын кестеде салыстырыңыз.
4. Файл жүктеу және серверге жіберу процестерінің схемасын жасаңыз.

10-тақырып. Интернеттегі мәліметтерді қорғау. Криптография және цифрлық қолтаңба

Бақылау сұрақтары

1. Ақпараттық қауіпсіздік дегеніміз не?
2. Криптография дегеніміз не?
3. Шифрлау дегеніміз не?
4. Симметриялық шифрлау қалай жұмыс істейді?
5. Асимметриялық шифрлау дегеніміз не?
6. Ашық кілт дегеніміз не?
7. Жабық кілт дегеніміз не?
8. Симметриялық және асимметриялық шифрлаудың айырмашылығы қандай?
9. Хэш-функция дегеніміз не?
10. Цифрлық қолтаңба дегеніміз не?
11. Цифрлық қолтаңба қалай тексеріледі?
12. HTTPS қандай қорғаныс береді?
13. Фишинг дегеніміз не?
14. Интернеттегі негізгі қауіптерді атаңыз.

Практикалық тапсырмалар

1. Симметриялық және асимметриялық шифрлаудың схемасын жасаңыз.
2. Цифрлық қолтаңбаны қалыптастыру және тексеру процесін көрсетіңіз.
3. HTTP және HTTPS байланысын салыстырыңыз.
4. Берілген қауіптер үшін қорғаныс тәсілдерін ұсыныңыз: әлсіз пароль, фишинг, зиянды файл, ашық Wi-Fi.

11-тақырып. Cookies және пайдаланушыларды идентификациялау

Бақылау сұрақтары

1. Пайдаланушыны идентификациялау дегеніміз не?
2. Идентификация мен аутентификация айырмашылығы қандай?

3. Авторизация дегеніміз не?
4. Cookie дегеніміз не?
5. Cookies не үшін қолданылады?
6. Session Cookie дегеніміз не?
7. Persistent Cookie дегеніміз не?
8. First-party Cookie дегеніміз не?
9. Secure атрибутының қызметі қандай?
10. HttpOnly не үшін қажет?
11. SameSite қандай қызмет атқарады?
12. Cookie мен Session айырмашылығы қандай?
13. Cookies технологиясының кемшіліктері қандай?

Практикалық тапсырмалар

1. **F12 → Application/Storage → Cookies** бөлімін ашыңыз.
2. Бір Cookie-дің Name, Domain, Path, Expires, Secure, HttpOnly, SameSite мәндерін анықтаңыз.
3. JavaScript арқылы Cookie жасаңыз:

```
document.cookie = "language=kk; path=/";
```

4. Cookie, LocalStorage және SessionStorage технологияларын салыстырыңыз.

12-ТАҚЫРЫП. HTML ФРЕЙМДЕРІ, МУЛЬТИМЕДИА ЖӘНЕ ФОРМАЛАР

Бақылау сұрақтары

1. iframe дегеніміз не?
2. iframe қандай жағдайда қолданылады?
3. HTML-де сурет қалай орналастырылады?
4. alt атрибуты не үшін қажет?
5. <audio> элементінің қызметі қандай?
6. <video> элементі не үшін қолданылады?
7. HTML форма дегеніміз не?
8. <form> элементінің қызметі қандай?
9. action және method атрибуттары не үшін қажет?
10. <input> элементінің қандай түрлері бар?
11. <textarea> қандай жағдайда қолданылады?
12. <select> және <option> қызметтері қандай?
13. GET және POST әдістерінің айырмашылығы қандай?
14. required атрибутының қызметі қандай?

Практикалық тапсырма

«Студентті тіркеу» формасын жасаңыз.

Формада:
Аты-жөні
E-mail
Телефон
Туған күні

Мамандық
Курс
Жынысы
Қызығушылығы
Қосымша ақпарат
Жіберу батырмасы
болуы тиіс.
Қосымша ретінде Web-параққа сурет, аудио, видео және iframe орналастырыңыз.

13-ТАҚЫРЫП. CSS – СТИЛЬДЕРДІҢ САТЫЛЫ КЕСТЕЛЕРІ

Бақылау сұрақтары

1. CSS дегеніміз не?
2. CSS не үшін қолданылады?
3. CSS ережесінің синтаксисі қандай?
4. Селектор дегеніміз не?
5. Tag selector дегеніміз не?
6. Class selector қалай жазылады?
7. ID selector қалай жазылады?
8. Inline CSS дегеніміз не?
9. Internal CSS дегеніміз не?
10. External CSS дегеніміз не?
11. External CSS-тің артықшылығы қандай?
12. Box Model дегеніміз не?
13. margin және padding айырмашылығы қандай?
14. position қасиеті не үшін қажет?
15. relative және absolute айырмашылығы қандай?
16. fixed қандай қызмет атқарады?
17. z-index не үшін қолданылады?

Практикалық тапсырма

Келесі HTML элементін:

```
<div class="card">
   <h2>Web бағдарламалау</h2>
   <p>HTML, CSS және JavaScript</p>
</div>
```

CSS көмегімен карточка түрінде безендіріңіз.
Міндетті түрде:
width
padding
margin
border
border-radius
font-size

background
position
қасиеттерін пайдаланыңыз.
Қосымша: бір-бірімен қабаттасатын екі элемент жасап, z-index көмегімен орналасу ретін өзгертіңіз.

14-ТАҚЫРЫП. JAVASCRIPT ТІЛІ

Бақылау сұрақтары

1. JavaScript дегеніміз не?
2. JavaScript Web-парақта қандай қызмет атқарады?
3. JavaScript HTML құжатына қалай қосылады?
4. Литерал дегеніміз не?
5. Айнымалы дегеніміз не?
6. let және const айырмашылығы қандай?
7. JavaScript деректер типтерін атаңыз.
8. Өрнек дегеніміз не?
9. Шартты операторлар не үшін қажет?
10. for циклі қалай жұмыс істейді?
11. while циклі дегеніміз не?
12. Функция дегеніміз не?
13. Параметр және аргумент дегеніміз не?
14. Массив дегеніміз не?
15. Объект дегеніміз не?
16. Мұралау дегеніміз не?
17. DOM дегеніміз не?
18. Браузер объектілерін атаңыз.
19. Оқиға дегеніміз не?
20. addEventListener() не үшін қолданылады?

Практикалық тапсырмалар

1. Екі санның қосындысын есептеңіз:

```
function sum(a, b) {
   return a + b;
}
```

2. Студенттің бағасын анықтайтын бағдарлама жазыңыз.

Шарт:
90–100 → Өте жақсы
75–89 → Жақсы
50–74 → Қанағаттанарлық
0–49 → Қанағаттанарлықсыз

3. Батырманы басқанда мәтінді өзгертіңіз.

```
document
   .getElementById("btn")
```

```
  .addEventListener("click", function () {
    document
      .getElementById("text")
      .textContent = "Мәтін өзгерді!";
  });
```

4. Массивтегі студенттердің аттарын цикл арқылы экранға шығарыңыз.

15-ТАҚЫРЫП. WEB-ҚОСЫМШАЛАР. HTTP. PERL ЖӘНЕ PHP НЕГІЗДЕРІ

Бақылау сұрақтары

1. Web-қосымша дегеніміз не?
2. Клиент-сервер архитектурасы қалай жұмыс істейді?
3. HTTP дегеніміз не?
4. HTTPS дегеніміз не?
5. HTTP Request дегеніміз не?
6. HTTP Response дегеніміз не?
7. GET әдісі не үшін қолданылады?
8. POST әдісі қандай мақсатта қолданылады?
9. GET пен POST айырмашылығы қандай?
10. HTTP күй коды дегеніміз не?
11. 200 OK нені білдіреді?
12. 404 Not Found дегеніміз не?
13. 500 Internal Server Error нені білдіреді?
14. Серверлік бағдарламалау дегеніміз не?
15. CGI дегеніміз не?
16. Perl тілінің Web бағдарламалаудағы тарихи рөлі қандай?
17. PHP дегеніміз не?
18. PHP коды қандай тегтер арасында жазылады?
19. PHP айнымалысы қалай жарияланады?
20. PHP-де шартты оператор қалай жазылады?
21. for, while, foreach циклдерінің қызметі қандай?
22. PHP функциясы қалай құрылады?
23. HTML формасындағы мәлімет PHP-ге қалай жіберіледі?

Практикалық тапсырмалар

1. PHP арқылы экранға мәтін шығарыңыз:

```
<?php
echo "Web бағдарламалау";
?>
```

2. Екі санның қосындысын есептеңіз:

```
<?php
$a = 10;
$b = 20;
```

```
echo $a + $b;
?>
```

3. Студенттің бағасын шартты оператор арқылы анықтаңыз.

4. HTML форма құрыңыз:

```
<form action="result.php" method="post">

    <label>Аты-жөні:</label>

    <input
        type="text"
        name="student"
        required>

    <button type="submit">
        Жіберу
    </button>

</form>
```

5. PHP арқылы формадағы ақпаратты өңдеңіз:

```
<?php

$name = $_POST["student"] ?? "";

echo "Қош келдіңіз, " .
     htmlspecialchars($name);

?>
```

Қорытынды практикалық бақылау